



CompTIA

DeepDumps

Premium exam material

Get certification quickly with the **Deep Dumps** Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily.

<https://www.DeepDumps.com>

About DeepDumps

At DeepDumps, we finally had enough of the overpriced, paywall-ridden exam prep industry. Our team of experienced IT professionals spent years navigating the certification landscape, only to realize that most prep materials came with a price tag bigger than the exam itself. We saw aspiring professionals spending more on study guides than on rent, and frankly, it was ridiculous. So, we decided to change the game. DeepDumps was born with one goal: to provide high-quality exam preparation without draining your wallet.

With DeepDumps, you get expert-vetted study materials, real-world insights, and a fair shot at success—without the financial headache..

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://DeepDumps.com>

[Mail us on - support@deepdumps.com](mailto:support@deepdumps.com)



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John



Thank you so much for your help. I scored 972 in my exam today. More than 30 questions were there from your PDFs!

Dana



Thanks a lot for this updated AZ-900 Q&A. I just finished my exam and got 374. I followed both of your AZ-900 videos and the 6 PDF, the PDFs are very accurate, all answers are correct. Could you please create a similar video/PDF for DP900, your content-/PDF's is really awesome. The tem'd a really good job. Thank You 🙏

Ahamod Shibly



Customer support is really fast and friendly, I just finished with me exam I passed it along with the 6 PDF's you

Passed my exam today with 845 marks



Passed my exam today with 845 marks! Out of 52 questions, 51 were from your DeepDumps PDFs including Contoso case study. I will recommend you to my friends. Thank you DeepDumps team!

These questions are real and 100 % valid.



Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria



Simple, easy to understand explanations. To anyone who is writing the exam of AZ900,

Microsoft

(DP-750)

Implementing Data Engineering Solutions Using Azure Databricks

Total: **162 Questions**

Link: <https://deepdumps.com/papers/microsoft/dp-750>

Question: 1

Deep Dumps

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Overview -

Company Information -

Contoso, Inc. is a renewable energy provider that operates solar and wind farms across North America.

Existing Environment -

Azure Environment -

Contoso has a single Azure Databricks workspace named Workspace1 in the West US Azure region. Workspace1 is enabled for Unity Catalog.

Workspace1 contains all-purpose clusters for both development and production workloads.

The company's Azure environment contains:

In the West US, Central US, and East US Azure regions, Azure event hubs that stream telemetry data and an Azure Data Lake Storage Gen2 account in each region for each hub

A single Azure SQL database in the West US region that hosts enterprise resource planning (ERP) data

An Azure Database for PostgreSQL server in the West US region that stores operational maintenance data

Data Environment -

Contoso ingests the following operational and business data:

Telemetry data: More than 40,000 IoT sensors across 28 sites emit JSON telemetry events every few seconds. Each site sends the events to the nearest event hub, which writes the data into the corresponding Data Lake Storage Gen2 account. These files frequently experience schema drift.

Maintenance logs: Maintenance systems generate historical repair logs, daily incremental updates, technician notes, and unstructured attachments that are stored in the Data Lake Storage Gen2 accounts.

Operational maintenance data: Structured operational maintenance data is stored on the Azure Database for PostgreSQL server.

External weather data: Hourly weather forecasts are retrieved from a REST API and written to the Data Lake Storage Gen2 accounts.

ERP data: Daily CSV extracts of 50 to 100 GB contain equipment metadata, work orders, and purchase order information.

Problem Statements -

The company's existing analytics environment has several issues:

Ingestion -

Telemetry pipelines fall behind during peak loads.

Telemetry ingestion fails when schema drift occurs.

Streaming pipelines reprocess events after a pipeline restarts.

Compute -

Production and development workloads run on the same all-purpose clusters.

Production and development workloads do NOT support autoscaling or workload isolation.

Governance -

The ERP data is duplicated across systems and development teams.

Naming conventions are inconsistent across development teams, regions, and products.

Ownership of the IoT sensors changes over time, and analysts must track the full history of the ownership.

Occasionally, equipment manufacturers must correct data-entry mistakes in equipment names. Historical values are NOT required.

Pipeline operations -

Pipelines lack resiliency, alerting, and centralized scheduling.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement scalable data pipeline orchestration.

Create a managed analytics catalog in Unity Catalog.
 Implement a consistent approach to creating curated datasets.
 Establish a centralized governance model across ingestion, cleansed, and curated layers.
 Grant data engineers access to the ERP tables by using minimal development effort.
 Adopt a compute strategy that isolates production workloads and supports autoscaling.
 Adopt a slowly changing dimension (SCD) approach to address current data modeling issues.

Technical Requirements -
 Contoso identifies the following environment and compute requirements:
 Ensure that production ingestion workloads run on compute clusters that can scale automatically during telemetry spikes.
 Provide fast and consistent performance for business intelligence (BI) workloads.
 Prevent development activity from affecting production pipelines.
 Production ingestion workloads must run as scheduled, non-interactive pipelines rather than on shared interactive development clusters.

Contoso identifies the following data ingestion and processing requirements:
 Auto-scale ingestion pipelines to handle bursty workloads.
 Handle schema drift for the maintenance and telemetry data.
 Ingest file-based telemetry data by using minimal operational effort.
 Store all the ingested data in a format that supports incremental processing.
 Support the continuous ingestion of telemetry data from the event hubs by using exactly-once semantics.
 Support the ingestion of the structured maintenance data from the Azure Database for PostgreSQL server.
 Build a new telemetry pipeline that ingests raw events from the event hubs, cleanses the data, and publishes curated tables to Unity Catalog.

Ensure that the Apache Spark Structured Streaming pipelines reading from the event hubs write the data into a managed Delta table named telemetry.raw_events. The pipelines must support schema drift and resume processing after failures without reprocessing the data.

Contoso identifies the following data modeling and optimization requirements:
 Build curated tables that standardize business logic.
 Overwrite equipment metadata attributes, such as name, manufacturer, model, and commissioning date, when the attributes change. Historical values are NOT required.

Contoso identifies the following pipeline deployment and operation requirements:
 Orchestrate multi-step ingestion and transformation workflows.
 Define a clear execution order and dependencies.
 Automatically retry failed steps and notify operators.
 Schedule ingestion and transformation workloads consistently.

Governance Requirements -
 Contoso identifies the following governance requirements:
 Centralize the metadata catalog.
 Provide isolated development areas that follow standard naming conventions.
 Establish a consistent structure for organizing raw, cleansed, and curated data.
 Provide a read-only mechanism to reference the ERP data through a foreign catalog.

Business Requirements -
 Contoso identifies the following business requirements:
 Improve ingestion reliability and reduce operational effort.
 Standardize data definitions across development teams.

You need to configure compute for the ingestion of telemetry data. The solution must meet the data ingestion and processing requirements.

What should you do?

- A. Move the ingestion pipelines to shared compute.
- B. Enable Photon acceleration for a job compute cluster.
- C. Increase an all-purpose cluster to a larger fixed node type.
- D. Disable autoscaling for a job compute cluster.

Answer: B

Explanation:

Correct Answer: B. Enable Photon acceleration for a job compute cluster

Why B is the best choice:

Scalability and Autoscaling: The requirement specifies that production ingestion workloads must run on compute clusters that can scale automatically during telemetry spikes. Job compute clusters in Azure Databricks are designed for this purpose, and enabling **Photon acceleration** further enhances the performance of these workloads by providing a native, in-memory query processing engine that accelerates Spark workloads without requiring any code changes.

Workload Isolation and Autoscaling Support: Job clusters are ideal for scheduled, non-interactive pipelines (as required), isolating production workloads from development activities. Photon acceleration on a job cluster ensures fast and consistent performance, crucial for BI workloads and meeting the exactly-once semantics for telemetry data ingestion.

Meeting Specific Requirements:

Auto-scale ingestion pipelines: Supported by job clusters.

Handle schema drift: While not directly related to compute, the overall architecture (e.g., using Delta tables with merge capabilities) can handle this, and Photon doesn't hinder this capability.

Supports incremental processing and exactly-once semantics: Photon-accelerated job clusters can efficiently handle these requirements.

Why Other Options are Less Suitable:

A. Move the ingestion pipelines to shared compute:

Contradicts Requirements: Shared compute would not isolate production workloads from development, potentially affecting pipeline performance and reliability.

Lacks Autoscaling for Production Spikes: Shared clusters might not scale as effectively for sudden production workload increases.

C. Increase an all-purpose cluster to a larger fixed node type:

Inefficient Scaling: Fixed node sizes do not offer the dynamic scaling needed for handling bursty, spike-based workloads efficiently.

No Workload Isolation: All-purpose clusters mix development and production workloads, contrary to the requirements.

D. Disable autoscaling for a job compute cluster:

Directly Opposes Requirements: Disabling autoscaling would prevent the cluster from scaling automatically during telemetry spikes, contradicting a primary technical requirement.

Inefficient Resource Utilization: Would lead to under or over provisioning, affecting cost and performance.

References

[Azure Databricks: Job Compute Clusters](#)

[Azure Databricks: Photon Acceleration](#)

Question: 2

Deep Dumps

You have an Azure Databricks workspace.

You are creating a Lakeflow Spark Declarative Pipelines (SDP) pipeline that scales automatically.

You need to configure compute for the pipeline. The solution must minimize operational costs and effort.

What should you use?

- A. the existing SQL warehouse
- B. an all-purpose cluster that uses autoscaling
- C. a job cluster that uses autoscaling
- D. a single-node, all-purpose cluster

Answer: C

Explanation:

Requirement Recap: Configure compute for a Lakeflow Spark Declarative Pipelines (SDP) pipeline in an Azure Databricks workspace, ensuring automatic scaling to minimize operational costs and effort.

Why Option C (a job cluster that uses autoscaling) is the Best Choice:

Cost Efficiency and Minimal Operational Effort: Job clusters are designed for workload-specific operations like running jobs or pipelines. Since they are ephemeral and scale automatically, they only incur costs when actively processing the pipeline, minimizing idle resource expenses.

Autoscaling Capability: Autoscaling ensures the cluster adjusts its size based on the pipeline's demands in real-time, optimizing resource utilization.

Alignment with SDP Workload: SDP pipelines are suited for batch or scheduled workloads. Job clusters, with their on-demand scaling, match this workload pattern perfectly, unlike always-on configurations.

Why Other Options are Less Suitable:

A. The Existing SQL Warehouse:

Inefficient Resource Utilization: SQL warehouses are optimized for interactive queries, not pipeline workloads, leading to potential resource mismatches.

Cost Inefficiency: Dedicated warehouses incur costs even when idle, contradicting the cost-minimization requirement.

B. An All-Purpose Cluster with Autoscaling:

Over provisioning for Pipeline Needs: All-purpose clusters are designed for a mix of interactive and batch workloads, potentially leading to over-provisioning for a single pipeline's needs.

Higher Baseline Costs: While autoscaling helps, the broader capability set may increase baseline costs compared to a job cluster.

D. A Single-Node, All-Purpose Cluster:

Scalability Limitation: Single-node clusters cannot scale out to handle increased pipeline workload, leading to potential processing bottlenecks.

No Autoscaling: Contradicts the requirement for automatic scaling to manage costs and effort effectively.

References:

[Azure Databricks Documentation - Job Clusters](#)

[Azure Databricks Documentation - Autoscaling](#)

Question: 3

Deep Dumps

DRAG DROP -

You have an Azure Databricks workspace that contains an all-purpose compute cluster named Cluster1. Cluster1 is used for interactive development.

You need to configure Cluster1 to meet the following requirements:

Automatically add and remove worker nodes based on workload demand.

Automatically shut down when the cluster has been idle for a specific period.

What should you configure for each requirement? To answer, drag the appropriate options to the correct requirements. Each option may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Options

- Auto termination
- Autoscaling
- Max workers
- Min workers
- Photon acceleration
- Spot instances
- The driver node type

Answer Area

Automatically add and remove worker nodes:

Option

Automatically shut down:

Option

Answer:

Options

- Auto termination
- Autoscaling
- Max workers
- Min workers
- Photon acceleration
- Spot instances
- The driver node type

Answer Area

Automatically add and remove worker nodes:

Autoscaling

Automatically shut down:

Auto termination

Explanation:

Autoscaling: This configuration enables Azure Databricks to dynamically provision more worker nodes when computing workloads spike, and automatically release them when the workload drops. This aligns resources directly to current needs, saving infrastructure costs.

Auto termination: This setting allows you to specify a period of inactivity (in minutes). If a cluster stays idle with no active commands or jobs for that duration, Databricks automatically terminates (shuts down) the cluster compute resources to prevent unnecessary cloud compute billing.

Incorrect answer

Max workers / Min workers: These are parameters used within the autoscaling configuration to set absolute numeric boundaries (caps), but they do not actively manage or trigger the dynamic adding/removing mechanism on their own.

Photon acceleration: This is a high-performance, native C++ vectorized query engine that speeds up data processing workloads; it does not manage cluster scaling or shutdown states.

Spot instances: This allows you to use idle cloud capacity at a steep discount for workers, which helps minimize costs but does not govern cluster termination or resource scaling logic.

The driver node type: This selects the specific hardware/virtual machine instance size dedicated to hosting the Spark driver program orchestrating the cluster; it handles control-plane memory and coordination, not

Question: 4**Deep Dumps**

You have an Azure Databricks workspace that is attached to a Unity Catalog metastore named metastore1, metastore1 contains a catalog named catalog1.

You need to create a new schema named schema2 that meets the following requirements:

Is contained in catalog1 -

Uses abfss:///data as the managed location

Which SQL statement should you execute?

- A. CREATE SCHEMA catalog1.schema2 -
LOCATION 'abfss:///data';
- B. CREATE SCHEMA catalog1.schema2 -
MANAGED LOCATION 'abfss:///data';
- C. CREATE CATALOG schema2 -
MANAGED LOCATION 'abfss:///data';
- D. CREATE SCHEMA catalog1.schema2 -
WITH DBPROPERTIES (LOCATION='abfss:///data');

Answer: B

Explanation:

Technical Justification for Correct Answer: B

To create a new schema named schema2 within catalog1 on Azure Databricks with Unity Catalog, meeting the specified requirements, the correct SQL statement is **B. CREATE SCHEMA catalog1.schema2 -MANAGED LOCATION 'abfss://@[storage_account_name].blob.core.windows.net/data';**. Here's why:

Correct Answer (B) Justification

CREATE SCHEMA catalog1.schema2: Correctly specifies the creation of a new schema named schema2 within the existing catalog1, fulfilling the "contained in catalog1" requirement.

MANAGED LOCATION 'abfss://@[storage_account_name].blob.core.windows.net/data':

MANAGED LOCATION is the correct keyword for specifying the managed location for the schema in Unity Catalog. This setting ensures that the schema's metadata and data location are managed by Unity Catalog, which is crucial for unified governance and security.

The provided abfss path correctly points to the Azure Blob Storage location as required, indicating where the schema's associated data will be managed.

Why Other Options Are Less Suitable

A. CREATE SCHEMA catalog1.schema2 -LOCATION '...';

LOCATION (without "MANAGED") might imply an external location not managed by Unity Catalog, lacking the governance and security benefits provided by Unity Catalog's management. This could lead to inconsistencies in data management and security.

C. CREATE CATALOG schema2 -MANAGED LOCATION '...';

This statement attempts to create a **CATALOG**, not a **SCHEMA**, and outside of catalog1, directly violating the requirement that schema2 must be contained within catalog1.

D. CREATE SCHEMA catalog1.schema2 -WITH DBPROPERTIES (LOCATION-'...');

WITH DBPROPERTIES is used for setting custom properties, not for specifying the managed location of a schema in Unity Catalog.

The syntax **LOCATION-** (with a hyphen) is incorrect for this context; even if the syntax were corrected, this method does not effectively designate a managed location as required.

Key Technical Points

Unity Catalog requires explicit management specifications for schemas and catalogs to ensure unified security and governance.

MANAGED LOCATION is crucial for Unity Catalog to manage the schema's data location effectively. Syntax accuracy (e.g., MANAGED LOCATION, correct catalog/schema notation) is vital for successful execution in Azure Databricks SQL.

Recommendations for Study (Related to Question)

Review Unity Catalog concepts, especially schema and catalog management.

Focus on the differences between managed and unmanaged resources in Azure Databricks.

Practice SQL statements for creating and managing schemas and catalogs in Unity Catalog.

References

1. **Azure Databricks - Unity Catalog Documentation:** <https://docs.databricks.com/governance/unity-catalog/index.html>
2. **Azure Databricks - Managing Schemas with Unity Catalog:** <https://docs.databricks.com/governance/unity-catalog/schemas.html>

Question: 5

Deep Dumps

You have an Azure Databricks workspace named Workspace1.

You create a compute cluster named Cluster1 that will be used to ingest data.

You need to install the required libraries on Cluster1. The solution must use Unity Catalog for access control. What should you do?

- A. Install the libraries by using pip3.
- B. Create a custom dependency management script and run the script from a Databricks notebook.
- C. Upload the libraries to Workspace1 and install the libraries on Cluster1.
- D. Install the libraries on Cluster1 and manually restart the cluster.

Answer: C

Explanation:

Technical Justification for Correct Answer (C)

Why C is the best option: Installing libraries by uploading them to the Workspace and then installing on the Cluster (Option C) is the most suitable approach for several reasons, especially when Unity Catalog is involved for access control:

Centralized Management: Uploading libraries to the Workspace facilitates centralized management, making it easier to track and update libraries across multiple clusters.

Security and Access Control via Unity Catalog: Since the solution must use Unity Catalog for access control, uploading libraries to the Workspace aligns with managing resources in a unified manner. This approach ensures that access to libraries can be controlled and audited similarly to other resources managed by Unity

Catalog.

Cluster Management Efficiency: Installing libraries from a centralized workspace location reduces the overhead of managing different library versions across clusters and ensures consistency.

Automation and Scalability: This method is more amenable to automation, particularly when integrating with CI/CD pipelines for library updates, enhancing the overall scalability of the data engineering workflow.

Why Other Options are Less Suitable:

A. Install the libraries by using pip3:

Lack of Central Control: Installing directly via pip3 on the cluster doesn't leverage Workspace-level management, making version control and access management more challenging.

Security Concerns: Direct internet access for pip3 might not be desirable in all environments due to security policies.

B. Create a custom dependency management script and run from a Databricks notebook:

Complexity: Introduces unnecessary complexity for library installation, which can be managed more straightforwardly through Workspace uploads.

Dependency on Notebook Execution: Ties library installation to notebook runtime, which might not be ideal for cluster provisioning workflows.

D. Install the libraries on Cluster1 and manually restart the cluster:

Inefficient: Manual intervention for installation and restart is inefficient and prone to human error, especially in automated deployment scenarios.

Lacks Integration with Unity Catalog's Access Control: Does not align with the requirement for unified access control through Unity Catalog.

References

[Azure Databricks: Managing Libraries](#)

[Azure Databricks: Unity Catalog](#)

Question: 6

Deep Dumps

DRAG DROP -

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a catalog named finance, finance contains two schemas named default and procurement.

You need to create a table named assets in the procurement schema, assets must contain the following columns: asset_id asset_type asset_name

How should you complete the SQL statement? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

assets

CATALOG finance

CATALOG hive_metastore

finance

SCHEMA procurement

TABLE assets

Answer Area

```
USE [Value] ;  
  
USE [Value] ;  
  
CREATE [Value] (  
  
    asset_id INT,  
    asset_name STRING,  
    asset_type STRING  
);
```

Answer:

Values

assets

CATALOG finance

CATALOG hive_metastore

finance

SCHEMA procurement

TABLE assets

Answer Area

```
USE CATALOG finance ;  
  
USE SCHEMA procurement ;  
  
CREATE TABLE assets (  
  
    asset_id INT,  
    asset_name STRING,  
    asset_type STRING  
);
```

Explanation:

CATALOG finance: Unity Catalog requires a three-tier namespace.

SCHEMA procurement: This sets the middle tier (also called a database). It groups related tables, views, and volumes together inside the selected catalog.

TABLE assets: This creates the standard structured data object with the defined schema columns inside the specific path (finance.procurement.assets).

Incorrect answer:

CATALOG hive_metastore: This is the default legacy workspace-level metastore. It does not utilize standard multi-tier isolation architectures or Unity Catalog advanced security frameworks.

finance / assets: These are raw entity names. They lack the mandatory declarative SQL keywords (CATALOG / TABLE) needed to make the query statement syntactically valid.

SCHEMA procurement : You cannot use a schema before a catalog is selected. This would cause a compilation error because Databricks wouldn't know which catalog contains that schema.

Question: 7

You have an Azure Databricks workspace that contains an all-purpose cluster named Cluster1.

You need to configure Cluster1 to meet the following requirements:

Scale up automatically when workloads increase

Scale down automatically when workloads decrease

Minimize costs -

Which two actions should you perform? Each correct answer presents part of the solution.

NOTE: Each correct selection is worth one point.

- A. Disable Photon acceleration.
- B. Enable autoscaling for Cluster1.
- C. Apply a compute policy that enables users to manage the cluster settings.
- D. Specify a fixed number of workers.
- E. Configure Cluster1 to terminate after 30 minutes of inactivity.

Answer: BE

Explanation:

Technical Justification for Correct Answer: BE

To meet the requirements of scaling up/down automatically based on workload and minimizing costs for Cluster1 in an Azure Databricks workspace, the following actions are justified:

B. Enable Autoscaling for Cluster1

Why It's Correct: Enabling autoscaling allows Cluster1 to dynamically adjust the number of workers in response to changes in workload. When workloads increase, the cluster scales up to handle the demand, and when workloads decrease, it scales down. This directly addresses the first two requirements.

Technical Basis: Autoscaling in Azure Databricks is based on the current workload, ensuring resources are allocated efficiently. It leverages Databricks' internal metrics to decide when to add or remove nodes.

Cost Minimization: By only provisioning additional resources when necessary and reducing them during lulls, autoscaling plays a crucial role in minimizing costs.

E. Configure Cluster1 to Terminate After 30 Minutes of Inactivity

Why It's Correct: Configuring the cluster to terminate after a period of inactivity ensures that no resources are consumed (and thus, no costs incurred) when the cluster is not in use. This significantly contributes to minimizing costs.

Technical Basis: Azure Databricks clusters, especially those not configured with manual termination settings, can incur unnecessary costs if left idle. Setting an inactivity timeout aligns with a cost-optimized strategy.

Alignment with Requirements: While not directly scaling, this action ensures that when there's no workload (the ultimate decrease), costs are fully eliminated for that cluster.

Why Other Options Are Less Suitable

A. Disable Photon Acceleration:

Irrelevant to Scaling and Cost: Photon acceleration is related to query performance optimization, not scaling or cost management. Disabling it would not impact the ability to scale or reduce costs.

C. Apply a Compute Policy to Enable Users to Manage Cluster Settings:

Indirect and Potentially Counterproductive: While useful for governance, this doesn't directly achieve the scaling or cost objectives. It might even lead to inefficient scaling if users aren't optimized for cost.

D. Specify a Fixed Number of Workers:

Contradicts Scaling Requirements: Fixing the number of workers prevents both scaling up and down, directly opposing the primary requirements.

References

1. **Azure Databricks Autoscaling Documentation:** <https://docs.databricks.com/clusters/databricks-cluster-manage.html#autoscaling>
2. **Azure Databricks Cluster Lifecycle Policies:** <https://docs.databricks.com/clusters/lifecycle-policies.html>

Question: 8

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.
You have an Apache Spark Structured Streaming job that writes data to a Delta table.
After the cluster restarts, the streaming job reprocesses previously ingested data.
You need to prevent the streaming job from reprocessing the data after the cluster restarts.
What should you do?

- A. Configure a checkpoint location for the streaming query.
- B. Increase the trigger interval of the streaming query.
- C. Enable change data feed (CDF) for the target table.
- D. Configure a watermark for the streaming query.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Why A is the Best Option:Configuring a checkpoint location for the streaming query (Option A) is the most suitable solution to prevent reprocessing of previously ingested data after a cluster restart. Here's why:

Checkpointing in Apache Spark Structured Streaming allows the job to save its progress (e.g., the last offset processed) to a designated location. Upon restart, the job can resume from the last checkpoint, avoiding redundant processing of already ingested data.

This approach directly addresses the issue by maintaining the stream's processing state, ensuring continuity post-restart without altering the job's logical behavior or dependencies.

Why Other Options are Less Suitable:

B. Increase the trigger interval of the streaming query:

Increasing the trigger interval might reduce the frequency of processing but does not prevent reprocessing of previously ingested data upon restart. It's more about throughput control than state persistence.

C. Enable change data feed (CDF) for the target table:

Change Data Feed (CDF) on Delta tables is useful for capturing changes (inserts, updates, deletes) made to the table. However, it doesn't directly solve the problem of a streaming job restarting and reprocessing data from its source; CDF is more relevant for tracking changes within the Delta table itself.

D. Configure a watermark for the streaming query:

Watermarks are used to handle late-arriving data in event-time processing, ensuring that the query can distinguish between late arrivals and the end of the stream. While crucial for certain streaming scenarios,

watermarks do not prevent the reprocessing of already consumed data upon a cluster restart.

Correct Action for the Scenario: To prevent the streaming job from reprocessing data after a cluster restart, **configure a checkpoint location for the streaming query**. This ensures the job resumes from the last processed point, maintaining efficiency and data integrity.

References:

[Apache Spark Documentation - Checkpointing](#)

[Azure Databricks - Checkpoints and IO Bits](#)

Question: 9

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a managed Delta table named Table1.

Table1 is written by batch jobs every hour and is queried frequently by filtering two columns named CustomerId and EventDate.

You expect Table1 to grow significantly over time.

The rows in Table1 are frequently updated and deleted to support compliance requests.

You need to keep query performance consistent as Table1 grows. The solution must minimize update and deletion effort.

What should you include in the solution? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

To optimize query performance, implement:

Liquid clustering

Partitioning

Z-ordering

To optimize updates and deletions:

Disable deletion vectors.

Enable deletion vectors.

Run the VACUUM command after each batch load.

Answer:

Answer Area

To optimize query performance, implement:

Liquid clustering

Partitioning

Z-ordering

To optimize updates and deletions:

Disable deletion vectors.

Enable deletion vectors.

Run the VACUUM command after each batch load.

Explanation:

Liquid clustering: This is the modern, recommended data layout technique in Delta Lake and Azure Databricks. Unlike legacy partitioning or static Z-ordering, Liquid clustering dynamically groups data based on high-cardinality or frequently queried columns without creating fragmented small files. It ensures **consistent query performance** automatically as the table size grows over time without requiring repetitive manual tuning or optimization re-runs.

Enable deletion vectors: [Deletion vectors](#) drastically **minimize update and deletion effort** by utilizing a "merge-on-read" mechanism. When a row is modified or deleted, Databricks writes a tiny, highly compressed bitmap file indicating which rows were removed, rather than executing a heavy "copy-on-write" operation that forces the entire data file to be rewritten. This reduces write amplification and accelerates transaction execution.

Incorrect answer:

Partitioning: This legacy directory-based strategy frequently results in the "small-file problem" or uneven data distribution when applied to high-cardinality fields, degrading query performance as tables expand.

Z-ordering: While excellent for multi-dimensional data skipping, Z-ordering is a static operation. It requires running periodic, high-overhead OPTIMIZE commands to maintain its performance benefits as new data is continually appended.

Disable deletion vectors: Disabling this feature forces Databricks to fall back to traditional copy-on-write execution, significantly increasing latency, infrastructure compute usage, and storage costs for frequent updates or deletes.

Run the VACUUM command after each batch load: The VACUUM command physically purges stale data files that are no longer referenced by the current state of the transaction log. Running it continuously after every batch load adds severe, unnecessary storage engine overhead and limits your ability to use "Time Travel" queries.

Question: 10

Deep Dumps

You have an Azure Databricks workspace that contains a cluster named Cluster1. Performance monitoring shows that Cluster1 is consistently overprovisioned for its batch workload: CPU utilization remains below 20 percent, including peak processing periods. The workload is highly predictable and does not spike. The current node type already meets the workload requirements. You need to reduce compute costs without increasing job duration. What should you do?

- A. Enable Photon acceleration.
- B. Configure Cluster1 to use a larger node type.
- C. Decrease the autotermination timeout of Cluster1.
- D. Disable autoscaling and reduce the number of worker nodes.

Answer: D

Explanation:

Technical Justification for Correct Answer: D

Why D is the Best Option: Reducing compute costs without increasing job duration for an overprovisioned Azure Databricks cluster (Cluster1) with a predictable, non-spiking batch workload can be most effectively achieved by **D. Disable autoscaling and reduce the number of worker nodes.**

Cost Reduction: Directly decreasing the number of worker nodes reduces the overall compute resources utilized, leading to lower costs.

No Increase in Job Duration: Since the workload is highly predictable and the current node type meets requirements, reducing the number of nodes (not changing the node type) won't impact job processing time, assuming the reduced node count still sufficiently handles the workload's peak (which, at <20% CPU utilization, is likely).

Autoscaling Disablement: Given the predictability of the workload, autoscaling (which is designed for variable or unpredictable workloads) is unnecessary and disabling it ensures the cluster doesn't inadvertently scale up, incurring additional costs.

Why Other Options are Less Suitable:

A. Enable Photon acceleration:

Purpose: Photon is designed to accelerate query performance by improving processing speeds, not directly reducing costs.

Impact on Cost: Enabling Photon might reduce job durations, potentially leading to indirect cost savings if nodes can be reduced afterward. However, it does not directly address the overprovisioning issue described.

Conclusion: Not the primary solution for reducing compute costs in this scenario.

B. Configure Cluster1 to use a larger node type:

Impact: Upgrading node types would increase costs due to the larger resource footprint, contrary to the objective.

Conclusion: Directly opposes the goal of reducing compute costs.

C. Decrease the autotermination timeout of Cluster1:

Purpose: Reduces the time a cluster stays idle before shutting down, which can save costs if the cluster is frequently idle.

Limitation: The question implies the cluster is consistently overprovisioned during peak processing periods, not just idle times. Thus, the cost savings would be minimal compared to reducing node count.

Conclusion: Less effective for the described scenario focusing on overprovisioning during active processing.

References

[Azure Databricks: Cluster Configuration Best Practices](<https://docs.microsoft.com/en-us/azure/databricks/manage-clusters/strcat#configure-cluster-size-and-autoscaling>)
[Azure Databricks Pricing](#)

Question: 11

Deep Dumps

You have an Azure Databricks workspace.

You are creating a Lakeflow Spark Declarative Pipelines (SDP) pipeline that scales automatically.

You need to configure compute for the pipeline. The solution must minimize operational costs and administrative effort.

What should you use?

- A. serverless compute
- B. a single-node, all-purpose cluster
- C. an all-purpose cluster that uses autoscaling

- D. an existing SQL warehouse
- E. a job cluster that uses autoscaling

Answer: A

Explanation:

Technical Justification for Correct Answer: A. Serverless Compute

Why A (Serverless Compute) is the Best Option:

Minimizes Operational Costs: Serverless compute in Azure Databricks eliminates the need to provision or manage clusters, reducing costs associated with idle resources. With serverless, you only pay for the compute resources consumed by your Lakeflow Spark Declarative Pipelines (SDP) pipeline.

Reduces Administrative Effort: Serverless compute automates cluster management, including scaling, patching, and upgrading, thereby minimizing administrative overhead. This aligns perfectly with the requirement for automatic scaling and reduced management effort.

Why Other Options are Less Suitable:

B. Single-node, All-purpose Cluster:

Inflexibility: Fails to meet the "scales automatically" requirement due to its fixed single-node configuration.

Cost Inefficiency: Incurs costs for the entire node regardless of utilization, potentially leading to underutilization.

C. All-purpose Cluster with Autoscaling:

Higher Operational Overhead: Requires managing cluster configuration, node types, and scaling rules.

Potential for Higher Costs: Autoscaling might lead to over-provisioning during peaks, increasing costs compared to serverless.

D. Existing SQL Warehouse:

Purpose Misalignment: Optimized for analytical queries, not suited for the compute demands of SDP pipelines.

Cost Structure: Might not offer the same cost optimization as serverless compute designed for variable workloads.

E. Job Cluster with Autoscaling:

Near Miss but Not Optimal: While it offers autoscaling and is designed for jobs, it still requires some cluster management overhead compared to serverless.

Cost Efficiency: Similar to C, might not be as cost-effective as serverless due to the need for managing minimum node configurations.

Conclusion: Serverless compute (A) is the most appropriate choice for configuring compute in an Azure Databricks Lakeflow SDP pipeline aiming to minimize operational costs and administrative effort while scaling automatically.

References:

1. **Azure Databricks Serverless Compute Documentation**
<https://docs.microsoft.com/en-us/azure/databricks/verifier/serverless>
2. **Lakehouse Spark Declarative Pipelines (SDP) on Azure Databricks**
<https://docs.microsoft.com/en-us/azure/databricks/lakehouse/spark-declarative-pipelines>

Question: 12

Deep Dumps

You have an Azure Databricks solution that was developed by multiple engineers and deployed to development, test, and production environments.

You need to implement an application lifecycle management (ALM) process that meets the following requirements:

Deploys and provides source control for Databricks notebooks, workflows, and jobs in a consistent and repeatable way

Parameterizes the target deployment environment

What should you include in the solution?

- A. the export and import of Databricks workspaces
- B. Databricks resources managed by using the Databricks Terraform provider
- C. Git folders and manual promotion
- D. Declarative Automation Bundles in Git

Answer: D

Explanation:

Technical Justification for Correct Option (D) and Analysis of Other Options

Correct Option: D. Declarative Automation Bundles in Git

Why D is the best choice:

Declarative Automation Bundles in Git directly address the requirements by providing a consistent and repeatable way to deploy Databricks notebooks, workflows, and jobs.

Source Control Integration: Storing these bundles in Git (e.g., Azure DevOps, GitHub) inherently provides source control for all components, ensuring versioning and collaboration among multiple engineers.

Environment Parameterization: Declarative Automation Bundles can be easily parameterized for target environments (development, test, production) using variables in the Git repository, which can be switched or overridden based on the deployment target.

Automation and Consistency: Automation Bundles automate the deployment process, reducing manual errors and ensuring consistency across environments.

Why Other Options are Less Suitable:

A. Export and Import of Databricks Workspaces

Limitation: This approach is more manual and not scalable for managing individual notebooks, workflows, and jobs across multiple environments efficiently.

Lack of Native Parameterization: Doesn't inherently support environment parameterization without additional, potentially complex scripting.

Source Control: While workspaces can be exported/imported, this method doesn't natively integrate with source control for the deployment process itself.

B. Databricks Resources Managed by using the Databricks Terraform Provider

Partial Solution: Excellent for provisioning Databricks infrastructure but less focused on the deployment of notebooks, workflows, and jobs themselves.

Additional Complexity for Deployment Logic: Would require integrating Terraform with another deployment tool/script for the actual deployment of Databricks artifacts, adding complexity.

Parameterization and Source Control: While Terraform configs can be parameterized and stored in Git, this option doesn't directly address the deployment of Databricks content in a simple, repeatable manner.

C. Git Folders and Manual Promotion

Highly Manual: Involves too much manual intervention for deployment, increasing the risk of human error.

Lack of Automation: Does not provide a automated, repeatable process for deployments.

Parameterization Challenges: Environment parameterization would require additional, potentially error-prone steps.

References

[Azure Databricks Documentation: Declarative Automation Bundles](#)

[Databricks Blog: Implementing CI/CD for Databricks with Declarative Automation Bundles](#)

Question: 13

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that contains a cluster named Cluster1.

Cluster1 has the following characteristics:

Configured as a single node cluster

Uses a general purpose virtual machine node type

The cluster runtime environment has the following configurations:

Uses Databricks Runtime for Machine Learning (Databricks Runtime ML)

Includes common machine learning libraries

Supports Python workloads -

For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Cluster1 can be used to train deep learning models that require GPU acceleration.	☐	☐
Cluster1 can distribute machine learning workloads across multiple nodes.	☐	☐
Cluster1 can run Python workloads that rely on preinstalled machine learning libraries.	☐	☐

Answer:

Answer Area

Statements	Yes	No
Cluster1 can be used to train deep learning models that require GPU acceleration.	☐	☒
Cluster1 can distribute machine learning workloads across multiple nodes.	☒	☐
Cluster1 can run Python workloads that rely on preinstalled machine learning libraries.	☒	☐

Explanation:

Statement 1: No

Reasoning: The STANDARD_DS12_V2 is a standard general-purpose **CPU-only** virtual machine size. It does not provide GPU hardware acceleration.

Statement 2: Yes

Reasoning: The parameter max_nodes=4 specifies a multi-node capability, allowing the cluster to scale up and distribute workloads across up to 4 distinct computing nodes.

Statement 3: Yes

Reasoning: Azure Machine Learning compute clusters automatically execute scripts within a chosen Environment container, which typically has essential packages (like scikit-learn or pandas) pre-installed.

Question: 14

Deep Dumps

You have an Azure Databricks workspace that contains the objects shown in the following table.

Name	Type
Catalog1	Catalog
Schema1	Schema
Sales1	Table
Notebook1	Notebook
Space1	AI/BI Genie space

Users often use the following words to refer to a sale: transaction, event, order, and invoice. You need to create a knowledge store. The solution must ensure that when the users use any of the words in Space1, Genie queries the Sales1 table. Any other Genie spaces must remain unaffected. To which object should you add the instructions?

- A. Sales1
- B. Schema1
- C. Space1
- D. Catalog1
- E. Notebook1

Answer: C

Explanation:

C. Space1.

In **Databricks AI/BI Genie**, a knowledge store is used to add custom business terms, synonyms, and specific text instructions to guide the AI assistant in querying data correctly.

Space-level Scoping: All knowledge store configurations and text instructions are strictly **scoped to a specific Genie Space** (in this case, Space1).

Isolation: Modifying Space1 directly ensures that the specific synonyms (transaction, event, order, invoice) map

to the Sales1 table only within that workspace, leaving all other Genie spaces in the organization completely unaffected. Adding metadata or definitions at the Catalog, Schema, or Table level would apply globally across multiple spaces.

Question: 15

Deep Dumps

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the “Next” button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the “Question” button to return to the question.

Overview -

Company Information -

Contoso, Inc. is a renewable energy provider that operates solar and wind farms across North America.

Existing Environment -

Azure Environment -

Contoso has a single Azure Databricks workspace named Workspace1 in the West US Azure region. Workspace1 is enabled for Unity Catalog.

Workspace1 contains all-purpose clusters for both development and production workloads.

The company's Azure environment contains:

In the West US, Central US, and East US Azure regions, Azure event hubs that stream telemetry data and an Azure Data Lake Storage Gen2 account in each region for each hub

A single Azure SQL database in the West US region that hosts enterprise resource planning (ERP) data

An Azure Database for PostgreSQL server in the West US region that stores operational maintenance data

Data Environment -

Contoso ingests the following operational and business data:

Telemetry data: More than 40,000 IoT sensors across 28 sites emit JSON telemetry events every few seconds. Each site sends the events to the nearest event hub, which writes the data into the corresponding Data Lake Storage Gen2 account. These files frequently experience schema drift.

Maintenance logs: Maintenance systems generate historical repair logs, daily incremental updates, technician notes, and unstructured attachments that are stored in the Data Lake Storage Gen2 accounts.

Operational maintenance data: Structured operational maintenance data is stored on the Azure Database for PostgreSQL server.

External weather data: Hourly weather forecasts are retrieved from a REST API and written to the Data Lake Storage Gen2 accounts.

ERP data: Daily CSV extracts of 50 to 100 GB contain equipment metadata, work orders, and purchase order information.

Problem Statements -

The company's existing analytics environment has several issues:

Ingestion -

Telemetry pipelines fall behind during peak loads.

Telemetry ingestion fails when schema drift occurs.

Streaming pipelines reprocess events after a pipeline restarts.

Compute -

Production and development workloads run on the same all-purpose clusters.

Production and development workloads do NOT support autoscaling or workload isolation.

Governance -

The ERP data is duplicated across systems and development teams.

Naming conventions are inconsistent across development teams, regions, and products.
 Ownership of the IoT sensors changes over time, and analysts must track the full history of the ownership.
 Occasionally, equipment manufacturers must correct data-entry mistakes in equipment names. Historical values are NOT required.

Pipeline operations -
 Pipelines lack resiliency, alerting, and centralized scheduling.

Requirements -
 Planned Changes -
 Contoso plans to implement the following changes:
 Implement scalable data pipeline orchestration.
 Create a managed analytics catalog in Unity Catalog.
 Implement a consistent approach to creating curated datasets.
 Establish a centralized governance model across ingestion, cleansed, and curated layers.
 Grant data engineers access to the ERP tables by using minimal development effort.
 Adopt a compute strategy that isolates production workloads and supports autoscaling.
 Adopt a slowly changing dimension (SCD) approach to address current data modeling issues.

Technical Requirements -
 Contoso identifies the following environment and compute requirements:
 Ensure that production ingestion workloads run on compute clusters that can scale automatically during telemetry spikes.
 Provide fast and consistent performance for business intelligence (BI) workloads.
 Prevent development activity from affecting production pipelines.
 Production ingestion workloads must run as scheduled, non-interactive pipelines rather than on shared interactive development clusters.

Contoso identifies the following data ingestion and processing requirements:
 Auto-scale ingestion pipelines to handle bursty workloads.
 Handle schema drift for the maintenance and telemetry data.
 Ingest file-based telemetry data by using minimal operational effort.
 Store all the ingested data in a format that supports incremental processing.
 Support the continuous ingestion of telemetry data from the event hubs by using exactly-once semantics.
 Support the ingestion of the structured maintenance data from the Azure Database for PostgreSQL server.
 Build a new telemetry pipeline that ingests raw events from the event hubs, cleanses the data, and publishes curated tables to Unity Catalog.
 Ensure that the Apache Spark Structured Streaming pipelines reading from the event hubs write the data into a managed Delta table named `telemetry.raw_events`. The pipelines must support schema drift and resume processing after failures without reprocessing the data.

Contoso identifies the following data modeling and optimization requirements:
 Build curated tables that standardize business logic.
 Overwrite equipment metadata attributes, such as name, manufacturer, model, and commissioning date, when the attributes change. Historical values are NOT required.

Contoso identifies the following pipeline deployment and operation requirements:
 Orchestrate multi-step ingestion and transformation workflows.
 Define a clear execution order and dependencies.
 Automatically retry failed steps and notify operators.
 Schedule ingestion and transformation workloads consistently.

Governance Requirements -
 Contoso identifies the following governance requirements:
 Centralize the metadata catalog.
 Provide isolated development areas that follow standard naming conventions.
 Establish a consistent structure for organizing raw, cleansed, and curated data.
 Provide a read-only mechanism to reference the ERP data through a foreign catalog.

Business Requirements -
 Contoso identifies the following business requirements:
 Improve ingestion reliability and reduce operational effort.
 Standardize data definitions across development teams.
 You need to organize Unity Catalog. The solution must meet the governance requirements.
 What should you do?

- A. Use a single shared schema for all the development teams and rely on table-level permissions for isolation.
- B. Enable the development teams to create objects directly in the default catalog and schema.
- C. Create a separate catalog for each development team and enable each team to choose its own schema names.
- D. Create a shared development catalog, enforce a standardized naming convention, and assign each development team its own schema.

Answer: D

Explanation:

Technical Justification for Correct Answer D

Why D is the Best Option:

Governance Requirements Alignment: Option D directly addresses Contoso's governance requirements by providing a "consistent structure for organizing raw, cleansed, and curated data" through a standardized naming convention.

Isolation with Efficiency: By assigning each development team its own schema within a shared development catalog, D balances isolation (preventing naming conflicts and ensuring team ownership) with efficiency (avoiding catalog proliferation).

Centralized Metadata Catalog: Although D suggests a "shared development catalog," the correct interpretation in the context of Unity Catalog's capabilities and the question's intent is that this shared catalog exists under the unified management of Unity Catalog, thereby centralizing metadata management as required.

Scalability and Management: This approach is scalable for managing multiple teams and easier to administer than managing multiple catalogs or lacking any structure.

Why Other Options are Less Suitable:

A. Single Shared Schema with Table-Level Permissions:

Inadequate Isolation: Relying solely on table-level permissions for isolation can become cumbersome to manage across large teams and does not enforce naming conventions.

Naming Conflicts: A single schema increases the risk of naming conflicts among development teams.

B. Development Teams Creating Objects in the Default Catalog and Schema:

Lack of Governance: This approach fails to establish any form of isolation or naming convention, leading to potential chaos in the metadata catalog.

Security and Organization Concerns: Insecure and disorganized, contradicting governance requirements.

C. Separate Catalog for Each Development Team:

Over-Complication: Managing multiple catalogs for each team would overly complicate the metadata landscape, contrary to the goal of centralizing governance.

Scalability Issues: Less scalable for large numbers of teams and does not inherently enforce a consistent naming structure across teams.

Correct Action for Organizing Unity Catalog (Answer D): Create a shared development catalog, enforce a standardized naming convention, and assign each development team its own schema.

References(Two valid and working documentation links for further reading on Unity Catalog and Azure Databricks governance)

1. **Unity Catalog Documentation:** <https://docs.microsoft.com/en-us/azure/databricks/data-governance/unified-metadata>
2. **Azure Databricks Governance Best Practices:** <https://docs.microsoft.com/en-us/azure/databricks/administration-guide/cloud-governance-best-practices>

Question: 16

DRAG DROP -

You have an Azure Databricks workspace named Workspace1 that is attached to a Unity Catalog metastore named metastore1.

You need to register an Azure Storage account named account1 that has a hierarchical namespace enabled as an external location. The external location must use a managed identity to authenticate to account1 and the solution must follow the principle of least privilege.

Which three actions should you perform in sequence? To answer, move the appropriate actions from the list of actions to the answer area and arrange them in the correct order.

Actions**Answer Area**

Assign the Storage Blob Data Owner role to the access connector.

Assign Workspace1 the Storage Blob Data Contributor role for account1.

Register the access connector as a storage credential in metastore1.

Create a Databricks access connector.

Assign the Storage Blob Data Contributor role to the access connector.

Register the access connector as a storage credential in Workspace1.



Answer:

Actions**Answer Area**

Assign the Storage Blob Data Owner role to the access connector.

Assign Workspace1 the Storage Blob Data Contributor role for account1.

Register the access connector as a storage credential in metastore1.

Create a Databricks access connector.

Assign the Storage Blob Data Contributor role to the access connector.

Register the access connector as a storage credential in Workspace1.

Create a Databricks access connector.

Assign the Storage Blob Data Contributor role to the access connector.

Register the access connector as a storage credential in metastore1.



Explanation:

Create a Databricks access connector.

Why: This provides the managed identity that Azure Databricks uses to connect securely with your Azure

storage accounts.

Assign the Storage Blob Data Contributor role to the access connector.

Why: Following the principle of least privilege, the Storage Blob Data Contributor role grants read, write, and delete permissions to blob data without granting full administrative ownership (which Storage Blob Data Owner would do).

Register the access connector as a storage credential in metastore1.

Why: Unity Catalog manages storage credentials at the metastore level (metastore1) rather than individual workspace levels, allowing central data governance across workspaces.

Question: 17

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You need to ensure that data lineage is captured and can be reviewed for tables accessed by Databricks notebooks and jobs. The solution must minimize administrative effort.

Which compute configuration should you use to capture the data lineage and what should you use to review the data lineage? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Capture:

A serverless cluster
A single-user cluster
An interactive cluster
A Unity Catalog-enabled cluster

Review:

Log Analytics
Catalog Explorer
Azure Data Explorer
Directed Acyclic Graph (DAG)

Answer:

Answer Area

Capture:

A serverless cluster
A single-user cluster
An interactive cluster
A Unity Catalog-enabled cluster

Review:

Log Analytics
Catalog Explorer
Azure Data Explorer
Directed Acyclic Graph (DAG)

Explanation:

Capture: A serverless cluster

Reasoning: Databricks serverless compute automatically integrates with Unity Catalog to capture end-to-end table and column-level data lineage automatically. It completely eliminates the administrative overhead of configuring, upgrading, and managing custom clusters or runtime versions manually.

Review: Catalog Explorer

Reasoning: [Catalog Explorer](#) is the built-in UI interface within Databricks used to inspect database objects and visually review automatically captured data lineage maps, upstream sources, and downstream impacts.

Question: 18

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace.

You have an Azure key vault named kv-secure that stores a secret named storageKey. The value of storageKey is managed and updated by the cloud security team at your company.

You need to enable a Databricks notebook named Notebook1 to retrieve the value of storageKey securely at runtime. The solution must follow the principle of least privilege and always retrieve the latest value.

What should you do? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Expose storageKey to Databricks by using:

A Databricks access connector
A Databricks-backed secret scope
A Key Vault-backed secret scope

Retrieve storageKey from within Notebook1 by using:

dbutils.secrets.get
dbutils.secrets.list
spark.conf.get

Answer:

Answer Area

Expose storageKey to Databricks by using:

A Databricks access connector
A Databricks-backed secret scope
A Key Vault-backed secret scope

Retrieve storageKey from within Notebook1 by using:

dbutils.secrets.get
dbutils.secrets.list
spark.conf.get

Explanation:

Expose storageKey to Databricks by using: A Key Vault-backed secret scope

Reasoning: In Azure environments, secrets like storage account keys (storageKey) are securely managed in an **Azure Key Vault**. Creating a Key Vault-backed secret scope allows Azure Databricks to securely reference those secrets without hardcoding them.

Retrieve storageKey from within Notebook1 by using: dbutils.secrets.get

Reasoning: The Databricks Utilities library provides the `dbutils.secrets.get(scope, key)` method specifically to fetch the plaintext value of a secret within a notebook securely.

Why other options are incorrect:

A Databricks access connector: This is used specifically for managed identity integration with Unity Catalog external locations, rather than handling raw key string properties.

A Databricks-backed secret scope: While this does allow secret storage, the secrets are stored and encrypted natively inside the Databricks database itself. This creates decentralized secret management across platforms instead of utilizing Azure's dedicated native Key Vault infrastructure.

dbutils.secrets.list: This method is purely informational; it returns a list of secret names available within a

scope but **redacts and masks the actual values** so it cannot be used to supply the authentication key.

spark.conf.get: This method is strictly intended to fetch standard configuration settings from a Spark session environment (e.g., file block formats or shuffle partition tuning sizes) and cannot reach out to retrieve security tokens or Key Vault secrets.

Question: 19

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a catalog named Catalog1. Catalog1 contains a schema named Schema1 and a table named Table1.

You need to ensure that access to the data in Table1 is controlled by using attribute-based access control (ABAC). What should you apply to Table1, and how should you control access for users? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Apply to Table1:

▼

A governed tag

A table privilege

A workspace permission

To control user access:

▼

Create a dynamic view to restrict access.

Create a Unity Catalog access policy.

Grant the users the SELECT permission for Table1.

Answer:

Answer Area

Apply to Table1:

▼

A governed tag

A table privilege

A workspace permission

To control user access:

▼

Create a dynamic view to restrict access.

Create a Unity Catalog access policy.

Grant the users the SELECT permission for Table1.

Explanation:

Apply to Table1: A governed tag

Reasoning: Attribute-Based Access Control (ABAC) relies on assigning metadata attributes — known as **governed tags** in Unity Catalog — directly to tables or columns to classify the data (e.g., tagging a table as Sensitivity=High).

To control user access: Create a Unity Catalog access policy.

Reasoning: Once the data is classified using tags, you create a central **Unity Catalog access policy** to enforce permissions automatically based on those tags (e.g., allowing only specific groups to access any table carrying the Sensitivity=High attribute).

Incorrect Choices:

A table privilege: This represents traditional Role-Based Access Control (RBAC) where privileges (like SELECT or MODIFY) are granted directly to a user/group rather than evaluating object attributes dynamically.

A workspace permission: This manages user access to top-level workspace files, folders, or compute clusters rather than database-level data governance or catalog attributes.

Create a dynamic view to restrict access: While dynamic views can mask or filter rows, they are a legacy data-protection method that does not utilize the centralized tag-and-policy paradigm of ABAC.

Grant the users the SELECT permission for Table1: This is a direct static grant (RBAC) and does not implement attribute-based logic.

Question: 20

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You need to implement a data lifecycle and expiration solution that meets the following requirements:

Transaction logs and deleted data files that are older than 90 days must be removed from Delta tables to reclaim storage.

All the tables must remain available for querying during the cleanup process.

Administrative effort must be minimized.

What should you do for each requirement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Data lifecycle control:

▼

Set timeUntilArchived to 90.
Set logRetentionDuration to 90.
Set deletedFileRetentionDuration to 90.
Tag tables that contain retention metadata.

Expiration cleanup:

▼

Run the VACUUM command.
Run the OPTIMIZE command.
Drop and recreate the tables.
Use scheduled DELETE operations in jobs.

Answer:

Answer Area

Data lifecycle control:

▼

Set timeUntilArchived to 90.
Set logRetentionDuration to 90.

Set deletedFileRetentionDuration to 90.

Tag tables that contain retention metadata.

Expiration cleanup:

▼

Run the VACUUM command.

Run the OPTIMIZE command.
Drop and recreate the tables.
Use scheduled DELETE operations in jobs.

Explanation:

Data lifecycle control: Set deletedFileRetentionDuration to 90.

Reasoning: In Delta Lake, the table property `delta.deletedFileRetentionDuration` explicitly controls the minimum amount of time a file must remain in the log after being deleted or updated before it becomes eligible for physical removal. Setting this to 90 ensures history is maintained for the required duration.

Expiration cleanup: Run the VACUUM command.

Reasoning: The VACUUM command is used to permanently delete files that are no longer referenced by a Delta table and are older than the retention threshold. Crucially, running VACUUM does not lock the table, meaning **all tables remain available for querying** during the background cleanup process.

Question: 21

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a managed Delta table named Table1. Table1 stores customer data.

You need to implement a data retention solution that meets the following requirements:

Deleted data must be retained for 30 days to support audits.

Deleted data that is older than 30 days must be removed permanently.

The solution must minimize administrative effort

Which two properties should you configure? Each correct answer presents part of the solution.

NOTE: Each correct selection is worth one point.

- A. delta.timeUntilArchived
- B. delta.enableDeletionVectors
- C. delta.logRetentionDuration
- D. delta.deletedFileRetentionDuration
- E. delta.autoOptimize.autoCompact

Answer: BC

Explanation:

Technical Justification for Correct Options (B and C)

To implement a data retention solution for the managed Delta table (Table1) in Azure Databricks with Unity Catalog, meeting the specified requirements, the correct properties to configure are **B**.

delta.enableDeletionVectors and **C. delta.logRetentionDuration**. Here's why these are the best choices and why the other options are less suitable:

Correct Options:

B. delta.enableDeletionVectors: This property is crucial for efficient data retention and deletion. Enabling deletion vectors allows Delta Lake to keep track of deleted files without having to scan the entire table for deletes during compactions. This feature minimizes administrative effort by optimizing the deletion process, especially in a scenario where data needs to be retained for audits before permanent removal. **This directly supports the requirement for minimized administrative effort.**

C. delta.logRetentionDuration (set to 30 days): Configuring this property ensures that transaction logs (which include delete operations) are retained for the specified duration (30 days in this case). Since Delta Lake uses these logs for point-in-time recovery and to efficiently manage deletions, setting this to 30 days **directly meets the requirement for retaining deleted data for 30 days**. After this period, the logs are deleted, and permanently removing data older than 30 days becomes feasible.

Why Other Options Are Less Suitable:

A. delta.timeUntilArchived: This property is related to archiving files, which does not directly address the retention of deleted data for audit purposes or the automatic removal of old deleted data. Archiving is more about moving less accessed data to a colder storage tier, not about deletion retention periods.

D. delta.deletedFileRetentionDuration: While this might seem relevant, **delta.deletedFileRetentionDuration** specifically controls the retention of deleted files in the file system, not the logs necessary for auditing or the efficient management of deletions through deletion vectors. It's more about the physical file deletion timing

rather than the logical retention for audit purposes managed through transaction logs.

E. `delta.autoOptimize.autoCompact`: This property automates the compaction process for Delta tables, which is beneficial for performance but does not directly control data retention durations or the auditing requirement. Compaction is about consolidating files for query efficiency, not about retaining or removing deleted data based on a time threshold.

References

For deeper understanding of the configuration properties mentioned:

1. **Delta Lake Configuration** - <https://docs.databricks.com/delta/lake-configuration.html>
2. **Delta Lake Data Management** - <https://docs.databricks.com/delta/data-management.html>

Question: 22

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two catalogs named Catalog1 and Catalog2.

An external application uses a service principal named SP1 to connect to a SQL warehouse.

You need to ensure that SP1 can query the data in Catalog1 and Catalog2. The solution must follow the principle of least privilege.

Which permissions should you grant to SP1 for the catalogs?

- A. USE SCHEMA and SELECT
- B. USE CATALOG and SELECT
- C. USE CATALOG, USE SCHEMA, and SELECT
- D. USE CATALOG and USE SCHEMA

Answer: C

Explanation:

Technical Justification for Correct Answer C

To ensure the service principal SP1 can query the data in Catalog1 and Catalog2 within an Azure Databricks workspace enabled for Unity Catalog, following the principle of least privilege, the permissions granted must be precisely tailored to enable the required actions without over-privileging. Here's why option **C** is the best choice and why the others are less suitable:

Correct Answer C: USE CATALOG, USE SCHEMA, and SELECT

USE CATALOG: Necessary for SP1 to access the catalogs (Catalog1 and Catalog2) themselves. Without this, SP1 cannot even view the existence of the catalogs, let alone their contents.

USE SCHEMA: Required for SP1 to access the schemas within the catalogs. Since the question implies accessing specific data (implying specific schemas), this permission is essential. It allows SP1 to see and interact with the schema structure without granting broader catalog-level permissions that might not be necessary.

SELECT: Enables SP1 to query (read) the data, which is the core requirement. This is the least privilege needed for reading data, aligning with the principle of least privilege.

Why Other Options are Less Suitable

A. USE SCHEMA and SELECT:

Missing: USE CATALOG. Without this, SP1 cannot access the catalogs, making the other permissions useless in this context.

Insufficient for Catalog Access

B. USE CATALOG and SELECT:

Missing: USE SCHEMA. While SP1 can access the catalog, without USE SCHEMA, it cannot interact with the schema level, which is necessary for querying specific data structures.

Overly Broad without Schema Access: Might imply broader permissions than necessary if not paired with explicit schema access controls.

D. USE CATALOG and USE SCHEMA:

Missing: SELECT. Grants access to the catalog and its schemas but does not allow the actual querying of data, failing to meet the core requirement.

Incomplete for Data Access

Conclusion

Option **C** (USE CATALOG, USE SCHEMA, and SELECT) is the only choice that fully meets the requirements by:

1. Allowing access to the specified catalogs.
2. Enabling interaction with the necessary schemas within those catalogs.
3. Granting the specific permission to query (read) the data, all while following the principle of least privilege.

References

[Azure Databricks - Unity Catalog Permissions](#)

[Azure Databricks - Access Control for Unity Catalog](#)

Question: 23

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a catalog named Catalog1. Catalog1 contains a table named Transactions. Transactions contains the following columns: transaction_id customer_name email_address credit_card_number transaction_amount

You need to ensure that business analysts can query all the rows in the Transactions table. The solution must meet the following requirements:

Prevent the analysts from seeing the full values in the email_address and credit_card_number columns.

Ensure that the analysts can see only the values after the @ character in each email address.

Ensure that the analysts can see only the last four digits of each credit card number.

Enable the analysts to query the table without errors.

Follow the principle of least privilege.

What should you do?

- A. Grant the analysts the SELECT permission for the Transactions table and apply column-level encryption.
- B. Grant the analysts the SELECT permission for columns that do NOT contain sensitive data.
- C. Grant the analysts the SELECT permission for the Transactions table and implement row-level filters.
- D. Grant the analysts the SELECT permission for the Transactions table and apply column masks to email_address and credit_card_number.

Answer: D

Explanation:

Technical Justification for Correct Answer D

Why D is the Best Option: Granting the analysts SELECT permission for the Transactions table and applying column masks to email_address and credit_card_number directly addresses all the requirements.

Meet Query Requirements: Analysts can query all rows.

Data Protection:

Email Address: Column masking can be configured to show only characters after the @ symbol.

Credit Card Number: Column masking can reveal only the last four digits.

Least Privilege: SELECT permission is the least privilege required for querying.

Error-Free Querying: Properly applied masks do not prevent querying; they just obscure sensitive data.

Why Other Options are Less Suitable:

A. Column-Level Encryption:

Inconvenient for Querying: Encrypted data would require decryption keys to view, complicating the query process for analysts and not directly meeting the specific masking requirements.

Does Not Meet Specific Masking Needs: Encryption is an all-or-nothing approach, not flexible for partial data exposure as required.

B. SELECT on Non-Sensitive Columns:

Insufficient Access: Analysts need access to all columns (albeit with some masked) to perform comprehensive queries.

Inflexible: Does not allow for future needs if sensitive data columns must be accessed in a masked form.

C. Row-Level Filters:

Misaligned with Requirements: Row-level filters control access to rows based on conditions, not column content visibility.

Does Not Address Column Masking Needs: Would not mask email_address and credit_card_number as required.

Question: 24

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a catalog named catalog1.

You have a group named group1.

You plan to create a schema named schema1 in catalog1.

You need to ensure that group1 meets the following requirements:

Can create tables in schema1 -

Can modify and query tables -

Cannot grant permissions for the schema and its objects

How should you complete the SQL statements? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE SCHEMA catalog1.schema1;
```

```
GRANT  ON SCHEMA catalog1.schema1 
```

USAGE
USAGE, CREATE TABLE
USAGE, MODIFY

TO CATALOG catalog1
TO ROLE group1
TO SCHEMA schema1

Answer:

Answer Area

```
CREATE SCHEMA catalog1.schemal;
```

```
GRANT  ON SCHEMA catalog1.schemal 
```

USAGE

USAGE,CREATE TABLE

USAGE,MODIFY

TO CATALOG catalog1

TO ROLE group1

TO SCHEMA schemal

Explanation:

Dropdown 1: USAGE, CREATE TABLE

Reasoning: In Unity Catalog, granting a principal the ability to look up a schema and build data structures inside it requires both the USAGE privilege (to read the schema's metadata) and the object-creation privilege (such as CREATE TABLE).

Dropdown 2: TO ROLE group1

Reasoning: The GRANT statement syntax requires specifying a valid target principal (a user, group, or account role). group1 is the only valid principal entity listed in the options, whereas granting a schema permission to another schema or catalog is syntactically invalid.

Question: 25

Deep **Dumps**

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You need to create an external volume named Volume1 in an existing schema. Volume1 must expose files from an Azure Storage container. The solution must meet the following requirements:

Ensure that authentication does NOT require storing credentials in Databricks.

Ensure that users can access the files, but NOT modify the files.

Follow the principle of least privilege.

Which type of authentication should you configure, and which permission should you grant to the users? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Authentication type:



A service principal

A Databricks access connector

A shared access signature (SAS)

Permission:



BROWSE

READ VOLUME

WRITE VOLUME

Answer:

Answer Area

Authentication type:

A service principal
A Databricks access connector
A shared access signature (SAS)

Permission:

BROWSE
READ VOLUME
WRITE VOLUME

Explanation:

Authentication type

Correct Choice: A Databricks access connector

Reasoning: Azure Databricks strongly recommends using a Databricks Access Connector (Azure Managed Identity) for authentication. It eliminates the administrative overhead of rotating keys/certificates and enforces the principle of least privilege natively without exposing raw secrets.

Incorrect Choices:

A shared access signature (SAS): SAS tokens have expiration constraints and risk exposure if hardcoded or leaked into user logs, violating security best practices.

A service principal: While secure, service principals require client secrets or certificates that necessitate credential rotation pipelines, introducing higher administrative overhead than managed identities.

Permission

Correct Choice: READ VOLUME

Reasoning: Unity Catalog volumes manage non-tabular datasets. To allow users to fully access and read file data without giving them authorization to alter, append, or delete anything, **READ VOLUME** is the correct permission level.

Incorrect Choices:

BROWSE: This permission only allows a user to inspect metadata and list folder contents within the directory; it prevents them from actually opening or reading the file data.

WRITE VOLUME: This grants full read and write capabilities, which directly violates the requirement that users must not modify files.

Question: 26

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace named Workspace1 that contains a lakehouse and is enabled for Unity Catalog.

You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

The schemas and tables can be queried in Databricks.

The schemas and tables appear alongside other Unity Catalog objects.

The data is NOT copied into Databricks-managed storage.

Solution: You create a Lakeflow Connect pipeline and connect it to DB1.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification

Solution Evaluation: Creating a Lakeflow Connect pipeline to connect to DB1

Goal Requirements:

1. Query schemas and tables of DB1 in Databricks
2. Integrate schemas and tables alongside other Unity Catalog objects
3. No data copying into Databricks-managed storage

Analysis of the Proposed Solution (Lakeflow Connect pipeline):

Querying in Databricks: Lakeflow Connect primarily facilitates data ingestion and orchestration. While it enables data to be brought into Databricks for processing, **it does not directly expose the remote database's schemas and tables for ad-hoc querying** within Databricks without first ingesting the data.

Unity Catalog Integration: Lakeflow Connect is more focused on data movement and workflows rather than integrating metadata seamlessly into Unity Catalog for direct discovery and querying alongside other objects.

No Data Copying: This requirement is **met** as Lakeflow Connect, by design, can move data without necessitating a copy for mere querying purposes (though for processing, data might temporarily reside in Databricks).

Why the Correct Answer is B. No:

The solution **does not fully meet the goals** because it fails to directly expose DB1's schemas and tables for querying within Databricks without data ingestion and does not seamlessly integrate these into Unity Catalog for immediate discovery and querying alongside other objects.

Why Other Options (Implicitly Considered) Might Be More Suitable:

Alternative Solution (Not Listed but for Context):

Using Databricks' External Tables with Unity Catalog:

Meets Requirement 1: Direct querying of DB1 schemas and tables in Databricks.

Meets Requirement 2: Schemas and tables appear alongside other Unity Catalog objects.

Meets Requirement 3: No data copying into Databricks-managed storage.

Why It's More Suitable: Directly addresses all stated requirements without the need for data ingestion pipelines solely for querying and integration purposes.

References

[Azure Databricks - External Tables](#)

[Azure Databricks - Unity Catalog](#)

Question: 27

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace named Workspace1 that contains a lakehouse and is enabled for Unity Catalog.

You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

The schemas and tables can be queried in Databricks.

The schemas and tables appear alongside other Unity Catalog objects.

The data is NOT copied into Databricks-managed storage.

Solution: You create a Databricks access connector.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification for Solution Evaluation

Scenario Recap: Expose Microsoft SQL Server (DB1) schemas and tables in Azure Databricks Workspace1 (enabled for Unity Catalog) without copying data into Databricks-managed storage, ensuring queryability alongside other Unity Catalog objects.

Evaluated Solution: Creating a Databricks Access Connector

Evaluation Outcome: B. No (Does not meet the goal)

Rationale for Outcome:

Databricks Access Connector primarily facilitates secure, authenticated access to Azure services from Databricks, streamlining connectivity but not directly exposing external database schemas and tables within Unity Catalog.

Querying Capability: While a connector enables querying, it doesn't natively integrate the schema visibility within Unity Catalog as required, treating the source as an external data source rather than unified with internal objects.

No Data Copying: This aspect is technically met since the connector approach doesn't involve data replication. However, this single criterion is insufficient to fulfill all requirements.

Unity Catalog Integration: The key shortfall; merely creating an access connector does not automatically expose DB1's schemas and tables **alongside** other Unity Catalog objects in the unified metadata layer as explicitly required.

Why Other Hypothetical Options Might Be More Suitable (for context, assuming they were presented):

Alternative Option (Hypothetical): Register DB1 as an External Data Source within Unity Catalog

Meets Goal: Yes

Rationale:

Queryability: Enabled through the registered connection.

Schema Visibility: Schemas and tables are visible within Unity Catalog.

No Data Copying: Data remains in DB1.

Unity Catalog Integration: Directly integrates external schemas with internal objects.

Actual Evaluated Solution Reiteration: Given the provided solution (Databricks Access Connector) does not fully meet the requirements, especially concerning unified schema visibility within Unity Catalog, the correct answer is **B. No**.

References

[Azure Databricks - Unity Catalog](#)

[Azure Databricks - External Data Sources](#)

Question: 28

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace named Workspace1 that contains a lakehouse and is enabled for Unity Catalog.

You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

- The schemas and tables can be queried in Databricks.
- The schemas and tables appear alongside other Unity Catalog objects.
- The data is NOT copied into Databricks-managed storage.

Solution: You create a foreign catalog in Catalog Explorer.

Does this meet the goal?

- A. Yes
- B. No

Answer: A

Explanation:

Technical Justification for Solution A. Yes

The solution of creating a foreign catalog in Catalog Explorer within Azure Databricks workspace (Workspace1) effectively meets the stated requirements for exposing the schemas and tables of Microsoft SQL Server database (DB1). Here's why this option is correct and why other implied options (though not listed) might be less suitable:

Why A. Yes (Creating a Foreign Catalog) is Correct:

Querying Capabilities: By establishing a foreign catalog, users can query the schemas and tables of DB1 directly from Azure Databricks. This is because a foreign catalog allows for the integration of external data sources without data duplication, enabling querying capabilities through Unity Catalog.

Visibility Alongside Unity Catalog Objects: The foreign catalog integrates the metadata of DB1 into Unity Catalog, making the schemas and tables appear alongside other Unity Catalog managed objects. This meets the requirement for unified visibility.

No Data Copying into Databricks-Managed Storage: Since a foreign catalog only integrates metadata and provides a query interface to the source, the actual data remains in DB1. Thus, no data is copied into Databricks-managed storage, fulfilling this critical requirement.

Why Other Implied Options Might Be Less Suitable (assuming common alternatives):

Alternative: Creating a Database in Databricks and Copying Data

Inadequacy Reason: This would involve copying data into Databricks-managed storage, directly violating one of the primary requirements.

Alternative: Using Only a JDBC Connector without Catalog Integration

Inadequacy Reason: While this allows querying, it might not fully integrate the metadata alongside other Unity Catalog objects as seamlessly as a foreign catalog does, potentially falling short on the visibility requirement.

Conclusion: Creating a foreign catalog in Catalog Explorer (Solution A. Yes) is the best approach as it meets all specified requirements without the drawbacks of alternative methods.

References

1. **Azure Databricks - Unity Catalog and Foreign Catalogs:** <https://docs.microsoft.com/en-us/azure/databricks/security/unified-metadata/unified-catalog-intro>
2. **Azure Databricks - Querying External Data Sources:** <https://docs.microsoft.com/en-us/azure/databricks/data-sources/index>

Question: 29

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace named Workspace1 that contains a lakehouse and is enabled for Unity Catalog.

You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

The schemas and tables can be queried in Databricks.

The schemas and tables appear alongside other Unity Catalog objects.

The data is NOT copied into Databricks-managed storage.

Solution: You create a new native catalog in Unity Catalog.

Does this meet the goal?

- A. Yes
- B. No

Answer: B

Explanation:

Technical Justification for Answer B: No

The proposed solution of creating a new native catalog in Unity Catalog does not fully meet the stated goals for the following reasons:

Querying Schemas and Tables in Databricks: While creating a new native catalog in Unity Catalog allows for the management of metadata, it does not inherently expose external databases like Microsoft SQL Server (DB1) for direct querying within Databricks without additional configuration (e.g., creating external tables).

Insufficient for this goal as presented.

Appearance Alongside Other Unity Catalog Objects: A new native catalog would indeed allow schemas and tables to be managed alongside other Unity Catalog objects, meeting this requirement. However, this does not address the external data source integration aspect.

No Data Copy into Databricks-managed Storage: This requirement is met by the nature of the solution since a native catalog focuses on metadata management. However, the primary issue of seamlessly exposing DB1's schemas and tables for querying in Databricks without data copying is not directly addressed by just creating a new native catalog.

Why the Solution Falls Short (Why B: No is Correct)

Lack of Direct Integration Mechanism: The solution overlooks the need for an explicit integration mechanism (like External Tables or a similar concept) to expose DB1's schemas and tables for querying within Databricks without copying data.

Misinterpretation of "Native Catalog" Purpose: A native catalog in Unity Catalog is primarily for managing metadata within the Databricks ecosystem, not for exposing external databases for querying without additional setup.

More Suitable Approach (Not Listed but for Context)

Correct Approach Would Involve:

Creating **External Tables** in Unity Catalog pointing to DB1, allowing querying without data copying. Optionally, configuring a **Native Catalog** for metadata management of Databricks-native assets, alongside the externally defined tables for a unified view.

References

[External Tables in Unity Catalog - Azure Databricks](<https://docs.databricks.com/data/engineering/external-tables.html>)

[Unity Catalog Native Catalogs - Azure Databricks](#)

Question: 30

Deep Dumps

Your company has sales, finance, and HR departments.

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You need to implement Unity Catalog to meet the following requirements:

Access to Unity Catalog for each department must be managed independently from that of the other departments.

The data assets of each department must be isolated logically from those of the other departments.

The solution must support centralized governance.

What should you do for each department?

A. Create a separate Azure Data Lake Storage Gen2 account.

B. Create dynamic views.

- C. Create a new workspace.
- D. Create a new catalog.

Answer: D

Explanation:

Technical Justification for Implementing Unity Catalog Across Departments

Requirement Summary:

Independent access management for each department
Logical isolation of departmental data assets
Centralized governance support

Recommended Solution: D. Create a new catalog for each department

Why D is the Best Option:

Independent Access Management: Creating a new catalog for each department allows for granular, independent management of access controls through Unity Catalog's fine-grained access control capabilities. This ensures that access to each department's data assets can be managed separately without affecting other departments.

Logical Isolation: A separate catalog for each department provides a clear, logical separation of data assets, aligning with the requirement for isolation without necessitating physical storage separation.

Centralized Governance: Unity Catalog, with multiple catalogs, still operates under a unified metadata layer, facilitating centralized governance across all departments while maintaining departmental isolation.

Why Other Options are Less Suitable:

A. Separate Azure Data Lake Storage Gen2 account:

Overkill for Logical Isolation: Meets isolation but at a higher infrastructure complexity and cost level, without inherently solving access management independence or necessarily simplifying governance.

Lacks Unified Metadata Layer for Governance: Separate storage accounts would require additional integration for unified governance.

B. Create Dynamic Views:

Access Control Limitations: Dynamic views do not inherently provide the independent access management required across departments.

No Isolation of Underlying Data Assets: Views are virtual and do not logically isolate the underlying data.

C. Create a new workspace:

Excessive Resource Allocation: Creates unnecessary operational overhead and cost for each department, impacting centralized governance complexity.

Isolation Overkill: Provides more isolation than required (environmental isolation rather than just logical/data asset isolation).

Conclusion: Creating a new catalog for each department (Option D) strikes the optimal balance between independent access management, logical data asset isolation, and support for centralized governance, making it the most suitable solution for the given requirements.

References:

1. **Azure Databricks Unity Catalog Documentation:** <https://docs.databricks.com/security/unified-catalog/index.html>

Question: 31

Deep Dumps

DRAG DROP -

You have an Azure Databricks workspace named Workspace1.

You have a user named User1 that is a non-admin user for Workspace1.

You need to ensure that User1 can perform the following tasks:

Provision clusters of any size.

Run Databricks jobs.

The solution must follow the principle of least privilege.

What should you do?

To answer, drag the appropriate options to the correct requirements. Each option may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Options

Answer Area

Assign the workspace admins role.

Assign the Consumer access entitlement.

Grant the CAN RUN permission for the jobs.

Assign the Contributor role for a resource group.

Assign the Allow unrestricted cluster creation entitlement.

To ensure that User1 can provision clusters:

Option

To ensure that User1 can run jobs:

Option

Answer:

Options

Answer Area

Assign the workspace admins role.

Assign the Consumer access entitlement.

Grant the CAN RUN permission for the jobs.

Assign the Contributor role for a resource group.

Assign the Allow unrestricted cluster creation entitlement.

To ensure that User1 can provision clusters:

Assign the Allow unrestricted cluster creation entitlement.

To ensure that User1 can run jobs:

Grant the CAN RUN permission for the jobs.

Explanation:

To ensure that User1 can provision clusters: Assign the Allow unrestricted cluster creation entitlement.

Reasoning: Databricks uses explicit workspace entitlements to delegate resource deployment. The Allow unrestricted cluster creation entitlement specifically authorizes a non-admin account to deploy, configure, and scale new compute clusters manually.

To ensure that User1 can run jobs: Grant the CAN RUN permission for the jobs.

Reasoning: Databricks Jobs access control lists (ACLs) use fine-grained permissions. Granting the CAN RUN permission provides the exact access needed to trigger or execute an existing automated workflow pipeline manually or via a schedule without giving administrative modification privileges.

Question: 32

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.
You have a CSV file stored in an Azure Data Lake Storage Gen2 container.
You plan to ingest the data into an existing table by running the following SQL statement.
COPY INTO Customer -
FROM 'abfss:///data/customer'
FILEFORMAT = CSV -
You need to ensure that the statement can access the data in the container.
What should you configure?

- A. an external table named Customer in the CSV file format
- B. a Lakeflow ingestion gateway
- C. an external location that uses a storage credential
- D. a volume in a catalog

Answer: C

Explanation:

Technical Justification for Correct Option (C)

To ensure the SQL statement COPY INTO Customer FROM 'abfss:///data/customer' FILEFORMAT = CSV can access the data in the Azure Data Lake Storage Gen2 container within an Azure Databricks workspace enabled for Unity Catalog, the correct configuration is **option C: an external location that uses a storage credential**. Here's why:

Why Option C is Correct:

Unity Catalog and Access Control: With Unity Catalog, access to data sources is managed through fine-grained permissions. An external location configured with a storage credential allows you to securely connect to the Azure Data Lake Storage Gen2 container.

Secure Credential Management: Using a storage credential (e.g., a service principal or managed identity with appropriate permissions on the ADLS Gen2 container) ensures that the connection to the storage is secure and auditable, aligning with best practices for managing sensitive information.

Prerequisite for COPY INTO with External Data Sources: For COPY INTO to work with external data sources like ADLS Gen2, especially in a Unity Catalog-enabled workspace, defining an external location with the proper credential is a prerequisite. This setup informs Databricks where the data is located and how to access it securely.

Why Other Options are Less Suitable:

A. An external table named Customer in the CSV file format:

Reason: While an external table might seem relevant, the immediate issue is accessing the storage location securely, not defining the table's format. The COPY INTO statement implies the table already exists or will be created based on the source data format (in this case, inferred as CSV). **Lack of credential setup** makes this option incomplete for the task at hand.

Misconception Addressed: Defining an external table does not inherently grant access to the underlying storage without proper credential configuration.

B. A Lakeflow ingestion gateway:

Reason: Lakeflow is more suited for orchestrating and managing the lifecycle of data pipelines across various sources and destinations, rather than configuring secure access to a specific storage container for a one-time (or periodic) COPY INTO operation.

Technical Limitation: Lakeflow does not replace the need for credential management when accessing external storage services.

D. A volume in a catalog:

Reason: Volumes in Unity Catalog are primarily for managing metadata and permissions around databases, tables, and views, not for configuring access to external storage locations. This option does not address the security and access requirement for the ADLS Gen2 container.

Contextual Misalignment: Volumes are more about metadata management rather than storage access credentials.

References

1. **Azure Databricks - Unity Catalog:** <https://docs.databricks.com/security/access-control/unity-catalog.html>
2. **Azure Databricks - External Locations:** <https://docs.databricks.com/security/access-control/unity-catalog/external-locations.html>

Question: 33

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a catalog named catalog1. You have a group named group1. Group1 already has the USE CATALOG privilege on catalog1.

You create a schema named schema1 in catalog1.

You need to ensure that group1 can create tables in schema1. Group1 must not be able to grant permissions on the schema or its objects. The solution must follow the principle of least privilege.

How should you complete the SQL statement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

GRANT	CREATE TABLE, USE SCHEMA MODIFY, SELECT USE CATALOG, CREATE SCHEMA	ON SCHEMA catalog1.schema1	TO CATALOG catalog1 TO group1 TO SCHEMA schema1
-------	---	----------------------------	--

Answer:

Answer Area

GRANT	CREATE TABLE, USE SCHEMA MODIFY, SELECT USE CATALOG, CREATE SCHEMA	ON SCHEMA catalog1.schema1	TO CATALOG catalog1 TO group1 TO SCHEMA schema1
-------	---	----------------------------	--

Explanation:

CREATE TABLE, USE SCHEMA

Reasoning: In modern Databricks Unity Catalog permissions, the legacy privilege USAGE on schemas has been updated to **USE SCHEMA**. To allow a user or group to both view a schema and construct new data objects within it, they require both USE SCHEMA and CREATE TABLE granted directly at the schema level (ON SCHEMA catalog1.schema1).

Incorrect Choices:

MODIFY, SELECT: While valid privileges, these are typically applied at the table or view level for data reading and manipulation rather than schema creation or definition lookup tasks.

USE CATALOG, CREATE SCHEMA: These privileges are hierarchically scoped to a **Catalog** level, meaning they must be granted ON CATALOG catalog1 rather than ON SCHEMA.

Target Principal

TO group1

Reasoning: The GRANT statement syntax requires specifying a valid destination principal identity (such as a user, group, or service principal name). Here, **group1** is the only logical principal.

Incorrect Choices:

TO CATALOG catalog1: Granting object schema-level privileges to an entirely separate parent catalog container is syntactically invalid.

TO SCHEMA schema1: You cannot grant permission on a schema to another schema object.

Question: 34

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is attached to a Unity Catalog metastore named metastore1.metastore1 contains:

A catalog named Sales -

A schema named Customers in the Sales catalog

A table named Customer_details in the Customers schema

You need to ensure that a user named User1 can update the data in Customer_details. The solution must meet the following requirements:

Ensure that User1 cannot create new tables.

Follow the principle of least privilege.

Which permission should you grant to User1 for each object? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

The Sales catalog:

▼

BROWSE
USE CATALOG
USE SCHEMA

The Customers schema:

▼

CREATE TABLE
SELECT
USE SCHEMA

The Customer_details table:

▼

MANAGE
MODIFY
SELECT

Answer:

Answer Area

The Sales catalog:

▼

BROWSE
USE CATALOG
USE SCHEMA

The Customers schema:

▼

CREATE TABLE
SELECT
USE SCHEMA

The Customer_details table:

▼

MANAGE
MODIFY
SELECT

Explanation:

The Sales catalog: USE CATALOG

Reasoning: A user must first have permission to traverse the top-level catalog container. USE CATALOG does not grant access to anything inside it automatically, but it acts as a required entry gate to access child

objects.

The Customers schema: USE SCHEMA

Reasoning: Similar to the catalog level, the user needs permission to enter the namespace schema container before they can reference or execute actions on any concrete data structures within it.

The Customer_details table: SELECT

Reasoning: The SELECT privilege provides the user with the exact capability required to read and fetch data records from the specific table without granting unnecessary write (MODIFY) or administrative (MANAGE) permissions.

Question: 35

Deep Dumps

HOTSPOT -

You have an Azure Databricks account that contains a single workspace named Workspace1. Workspace1 is enabled for Unity Catalog.

You discover that data access events for Unity Catalog tables fail to appear in the logs.

You need to ensure that all the data access events are captured centrally for auditing purposes. The log data must be available for analysis as quickly as possible.

What should you do?

To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Enable audit logging for:

▼

Individual Unity Catalog tables
The Azure subscription
The Databricks account
Workspace1

Send audit logs to:

▼

An Azure Storage account
Azure Event Hubs
Azure Monitor metrics
Log Analytics

Answer:

Answer Area

Enable audit logging for:

▼

Individual Unity Catalog tables
The Azure subscription
The Databricks account
Workspace1

Send audit logs to:

▼

An Azure Storage account
Azure Event Hubs
Azure Monitor metrics
Log Analytics

Explanation:

Enable audit logging for: Workspace1

Reason: Standard Azure diagnostic settings are applied directly to the individual Azure Databricks workspace resource (Workspace1) to capture notebooks, clusters, and job activities.

Send audit logs to: Log Analytics

Reason: Sending the telemetry to an Azure [Log Analytics workspace](#) is required whenever the scenario mentions writing KQL queries, setting up alerts, or viewing interactive dashboards in the Azure portal.

Question: 36

Deep **Dumps**

DRAG DROP -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Overview -

Company Information -

Contoso, Inc. is a renewable energy provider that operates solar and wind farms across North America.

Existing Environment -

Azure Environment -

Contoso has a single Azure Databricks workspace named Workspace1 in the West US Azure region. Workspace1 is enabled for Unity Catalog.

Workspace1 contains all-purpose clusters for both development and production workloads.

The company's Azure environment contains:

In the West US, Central US, and East US Azure regions, Azure event hubs that stream telemetry data and an Azure Data Lake Storage Gen2 account in each region for each hub

A single Azure SQL database in the West US region that hosts enterprise resource planning (ERP) data

An Azure Database for PostgreSQL server in the West US region that stores operational maintenance data

Data Environment -

Contoso ingests the following operational and business data:

Telemetry data: More than 40,000 IoT sensors across 28 sites emit JSON telemetry events every few seconds.

Each site sends the events to the nearest event hub, which writes the data into the corresponding Data Lake Storage Gen2 account. These files frequently experience schema drift.

Maintenance logs: Maintenance systems generate historical repair logs, daily incremental updates, technician notes, and unstructured attachments that are stored in the Data Lake Storage Gen2 accounts.

Operational maintenance data: Structured operational maintenance data is stored on the Azure Database for PostgreSQL server.

External weather data: Hourly weather forecasts are retrieved from a REST API and written to the Data Lake Storage Gen2 accounts.

ERP data: Daily CSV extracts of 50 to 100 GB contain equipment metadata, work orders, and purchase order information.

Problem Statements -

The company's existing analytics environment has several issues:

Ingestion -

Telemetry pipelines fall behind during peak loads.

Telemetry ingestion fails when schema drift occurs.

Streaming pipelines reprocess events after a pipeline restarts.

Compute -

Production and development workloads run on the same all-purpose clusters.

Production and development workloads do NOT support autoscaling or workload isolation.

Governance -

The ERP data is duplicated across systems and development teams.

Naming conventions are inconsistent across development teams, regions, and products.

Ownership of the IoT sensors changes over time, and analysts must track the full history of the ownership.

Occasionally, equipment manufacturers must correct data-entry mistakes in equipment names. Historical values are NOT required.

Pipeline operations -

Pipelines lack resiliency, alerting, and centralized scheduling.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement scalable data pipeline orchestration.

Create a managed analytics catalog in Unity Catalog.

Implement a consistent approach to creating curated datasets.

Establish a centralized governance model across ingestion, cleansed, and curated layers.

Grant data engineers access to the ERP tables by using minimal development effort.

Adopt a compute strategy that isolates production workloads and supports autoscaling.

Adopt a slowly changing dimension (SCD) approach to address current data modeling issues.

Technical Requirements -

Contoso identifies the following environment and compute requirements:

Ensure that production ingestion workloads run on compute clusters that can scale automatically during telemetry spikes.

Provide fast and consistent performance for business intelligence (BI) workloads.

Prevent development activity from affecting production pipelines.

Production ingestion workloads must run as scheduled, non-interactive pipelines rather than on shared interactive development clusters.

Contoso identifies the following data ingestion and processing requirements:

Auto-scale ingestion pipelines to handle bursty workloads.

Handle schema drift for the maintenance and telemetry data.

Ingest file-based telemetry data by using minimal operational effort.

Store all the ingested data in a format that supports incremental processing.

Support the continuous ingestion of telemetry data from the event hubs by using exactly-once semantics.
Support the ingestion of the structured maintenance data from the Azure Database for PostgreSQL server.
Build a new telemetry pipeline that ingests raw events from the event hubs, cleanses the data, and publishes curated tables to Unity Catalog.
Ensure that the Apache Spark Structured Streaming pipelines reading from the event hubs write the data into a managed Delta table named telemetry.raw_events. The pipelines must support schema drift and resume processing after failures without reprocessing the data.
Contoso identifies the following data modeling and optimization requirements:
Build curated tables that standardize business logic.
Overwrite equipment metadata attributes, such as name, manufacturer, model, and commissioning date, when the attributes change. Historical values are NOT required.
Contoso identifies the following pipeline deployment and operation requirements:
Orchestrate multi-step ingestion and transformation workflows.
Define a clear execution order and dependencies.
Automatically retry failed steps and notify operators.
Schedule ingestion and transformation workloads consistently.

Governance Requirements -
Contoso identifies the following governance requirements:
Centralize the metadata catalog.
Provide isolated development areas that follow standard naming conventions.
Establish a consistent structure for organizing raw, cleansed, and curated data.
Provide a read-only mechanism to reference the ERP data through a foreign catalog.

Business Requirements -
Contoso identifies the following business requirements:
Improve ingestion reliability and reduce operational effort.
Standardize data definitions across development teams.

Which SCD type should you use to support the planned data modeling changes? To answer, drag the appropriate types to the correct issues. Each type may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

SCD types

- Type 0
- Type 1
- Type 2
- Type 3
- Type 4

Answer Area

Data-entry mistakes by the equipment manufacturers: SCD type

Changes to IoT sensor ownership: SCD type

Answer:

SCD types

- Type 0
- Type 1
- Type 2
- Type 3
- Type 4

Answer Area

Data-entry mistakes by the equipment manufacturers: Type 1

Changes to IoT sensor ownership: Type 2

Explanation:

Data-entry mistakes by the equipment manufacturers: Type 1

Reasoning: Data-entry mistakes require correcting the erroneous data by overwriting the old record with the accurate text. There is no business value or requirement to preserve a historical log of typographical errors.

Changes to IoT sensor ownership: Type 2

Reasoning: Tracking changes in asset ownership requires preserving the full historical timeline. Using an SCD Type 2 ensures that a new row is created with effective start/end dates to show exactly who owned the sensor during any historical window.

Question: 37

Deep Dumps

DRAG DROP -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the “Next” button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the “Question” button to return to the question.

Overview -

Company Information -

Contoso, Inc. is a renewable energy provider that operates solar and wind farms across North America.

Existing Environment -

Azure Environment -

Contoso has a single Azure Databricks workspace named Workspace1 in the West US Azure region. Workspace1 is enabled for Unity Catalog.

Workspace1 contains all-purpose clusters for both development and production workloads.

The company's Azure environment contains:

In the West US, Central US, and East US Azure regions, Azure event hubs that stream telemetry data and an Azure Data Lake Storage Gen2 account in each region for each hub

A single Azure SQL database in the West US region that hosts enterprise resource planning (ERP) data

An Azure Database for PostgreSQL server in the West US region that stores operational maintenance data

Data Environment -

Contoso ingests the following operational and business data:

Telemetry data: More than 40,000 IoT sensors across 28 sites emit JSON telemetry events every few seconds. Each site sends the events to the nearest event hub, which writes the data into the corresponding Data Lake Storage Gen2 account. These files frequently experience schema drift.

Maintenance logs: Maintenance systems generate historical repair logs, daily incremental updates, technician notes, and unstructured attachments that are stored in the Data Lake Storage Gen2 accounts.

Operational maintenance data: Structured operational maintenance data is stored on the Azure Database for PostgreSQL server.

External weather data: Hourly weather forecasts are retrieved from a REST API and written to the Data Lake Storage Gen2 accounts.

ERP data: Daily CSV extracts of 50 to 100 GB contain equipment metadata, work orders, and purchase order information.

Problem Statements -

The company's existing analytics environment has several issues:

Ingestion -

Telemetry pipelines fall behind during peak loads.

Telemetry ingestion fails when schema drift occurs.

Streaming pipelines reprocess events after a pipeline restarts.

Compute -

Production and development workloads run on the same all-purpose clusters.

Production and development workloads do NOT support autoscaling or workload isolation.

Governance -

The ERP data is duplicated across systems and development teams.

Naming conventions are inconsistent across development teams, regions, and products.

Ownership of the IoT sensors changes over time, and analysts must track the full history of the ownership.

Occasionally, equipment manufacturers must correct data-entry mistakes in equipment names. Historical values are NOT required.

Pipeline operations -

Pipelines lack resiliency, alerting, and centralized scheduling.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement scalable data pipeline orchestration.

Create a managed analytics catalog in Unity Catalog.

Implement a consistent approach to creating curated datasets.

Establish a centralized governance model across ingestion, cleansed, and curated layers.

Grant data engineers access to the ERP tables by using minimal development effort.

Adopt a compute strategy that isolates production workloads and supports autoscaling.

Adopt a slowly changing dimension (SCD) approach to address current data modeling issues.

Technical Requirements -

Contoso identifies the following environment and compute requirements:

Ensure that production ingestion workloads run on compute clusters that can scale automatically during telemetry spikes.

Provide fast and consistent performance for business intelligence (BI) workloads.

Prevent development activity from affecting production pipelines.

Production ingestion workloads must run as scheduled, non-interactive pipelines rather than on shared interactive development clusters.

Contoso identifies the following data ingestion and processing requirements:

Auto-scale ingestion pipelines to handle bursty workloads.

Handle schema drift for the maintenance and telemetry data.

Ingest file-based telemetry data by using minimal operational effort.

Store all the ingested data in a format that supports incremental processing.

Support the continuous ingestion of telemetry data from the event hubs by using exactly-once semantics.

Support the ingestion of the structured maintenance data from the Azure Database for PostgreSQL server.

Build a new telemetry pipeline that ingests raw events from the event hubs, cleanses the data, and publishes curated tables to Unity Catalog.

Ensure that the Apache Spark Structured Streaming pipelines reading from the event hubs write the data into a managed Delta table named telemetry.raw_events. The pipelines must support schema drift and resume processing after failures without reprocessing the data.

Contoso identifies the following data modeling and optimization requirements:

Build curated tables that standardize business logic.

Overwrite equipment metadata attributes, such as name, manufacturer, model, and commissioning date, when the attributes change. Historical values are NOT required.

Contoso identifies the following pipeline deployment and operation requirements:

Orchestrate multi-step ingestion and transformation workflows.

Define a clear execution order and dependencies.

Automatically retry failed steps and notify operators.

Schedule ingestion and transformation workloads consistently.

Governance Requirements -

Contoso identifies the following governance requirements:

Centralize the metadata catalog.

Provide isolated development areas that follow standard naming conventions.

Establish a consistent structure for organizing raw, cleansed, and curated data.

Provide a read-only mechanism to reference the ERP data through a foreign catalog.

Business Requirements -

Contoso identifies the following business requirements:

Improve ingestion reliability and reduce operational effort.

Standardize data definitions across development teams.

Which ingestion option should you recommend for each data source? To answer, drag the appropriate options to the correct data sources. Each option may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Ingestion options

Auto Loader

A Databricks notebook

Azure Data Factory

Lakeflow Connect managed connector

Answer Area

Telemetry data:

Ingestion option

Operational maintenance data:

Ingestion option

Maintenance logs:

Ingestion option

External weather data:

Ingestion option

Answer:

Ingestion options

Auto Loader

A Databricks notebook

Azure Data Factory

Lakeflow Connect managed connector

Answer Area

Telemetry data:

Auto Loader

Operational maintenance data:

Lakeflow Connect managed connector

Maintenance logs:

Azure Data Factory

External weather data:

A Databricks notebook

Explanation:

Telemetry data: Auto Loader

Reasoning: The telemetry events arrive as JSON files in Data Lake Storage Gen2 and experience frequent schema drift. **Auto Loader** is explicitly designed to ingest files from cloud storage incrementally, efficiently, and with native support for automatic **schema evolution and schema drift rescue**.

Operational maintenance data: Lakeflow Connect managed connector

Reasoning: This data is structured and resides inside an **Azure Database for PostgreSQL** server. **Lakeflow Connect** provides turnkey, managed database connectors specifically optimized for native relational database replication into the Unity Catalog.

Maintenance logs: Azure Data Factory

Reasoning: Maintenance logs combine complex structures (historical records, incremental updates, unstructured technician notes, and attachments). **Azure Data Factory (ADF)** excels at orchestrating these hybrid bulk/incremental copy operations and moving multi-format relational/unstructured logs securely.

External weather data: A Databricks notebook

Reasoning: Hourly weather forecasts are fetched from an external **REST API**. Running a lightweight Python notebook with HTTP libraries (such as requests) to query APIs, parse JSON payloads, and append them directly to target tables is a standard, low-overhead pattern in Databricks.

Question: 38

Deep Dumps

You have an Azure Databricks workspace that uses serverless compute. You need to ingest data by using Lakeflow Jobs. New records must be processed as soon as they become available. Which type of job trigger should you use for the ingestion?

- A. manual
- B. file arrival
- C. scheduled
- D. continuous

Answer: D

Explanation:

Technical Justification for Correct Answer: D. Continuous

To ingest data in an Azure Databricks workspace using Lakeflow Jobs, with the requirement that new records must be processed as soon as they become available, utilizing a **serverless compute** environment, the most appropriate trigger type is **Continuous (D)**. Here's why:

D. Continuous: This trigger type is best suited for real-time or near-real-time data processing scenarios where data arrival is unpredictable but needs immediate processing. Continuous jobs in Lakeflow are designed to run in an always-on manner, efficiently handling new data as it arrives, aligning perfectly with the serverless compute's on-demand scaling nature. This ensures low latency and immediate processing of new records, fulfilling the requirement for timely data ingestion.

Why Other Options are Less Suitable:

A. Manual: Requires human intervention to trigger the job, which contradicts the automation requirement for immediate processing of new records.

B. File Arrival: While this might seem suitable for triggering on new data, it's more suited for batch processing scenarios where the arrival of a specific file triggers a job. It may not offer the same level of immediacy or efficiency for continuous streams of data as Continuous jobs do, especially in a serverless context.

C. Scheduled: Implies processing data at fixed intervals (e.g., every hour), which does not guarantee that new records are processed "as soon as they become available," leading to potential delays in processing newly arrived data.

Key Technical Points Supporting Continuous Triggers in This Scenario:

Real-Time Processing Need: Immediate processing of new records.

Serverless Compute: Aligns with the on-demand, efficient scaling of continuous jobs.

Lakeflow Jobs Capability: Designed to support continuous processing for low-latency data pipelines.

References

1. **Azure Databricks - Lakeflow Jobs Triggers:**
<https://docs.databricks.com/workflows/lakeflow/jobs.html#trigger-types>
2. **Azure Databricks - Serverless Compute for Lakehouse:**
<https://docs.databricks.com/workflows/lakehouse/serverless-compute.html>

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a managed Delta table named Sales. Sales stores transaction data and contains the following columns: transaction_id (string) transaction_date (date) amount (decimal). You need to implement the following data quality requirements by using table-level data quality enforcement: amount must be greater than 0. transaction_id must never be null. Invalid records must be rejected when data is written to the Sales table. What should you do?

- A. Use a SELECT statement with WHERE conditions to validate the data before querying.
- B. Create a view that filters out rows where transaction_id is null or amount is less than or equal to 0.
- C. Add a NOT NULL constraint to transaction_id and a CHECK constraint to amount.
- D. Configure row-level security (RLS) where transaction_id is null or amount is less than or equal to 0.

Answer: C

Explanation:

Technical Justification with Rationale

Correct Answer: C. Add a NOT NULL constraint to transaction_id and a CHECK constraint to amount.

Why C is the best option:

Table-level data quality enforcement is required, and constraints (NOT NULL, CHECK) are enforced at the table level, ensuring data integrity upon insertion.

NOT NULL constraint on transaction_id directly addresses the requirement that transaction_id must never be null, preventing null values from being inserted.

CHECK constraint on amount (amount > 0) ensures that only valid (positive) amounts are stored, fulfilling the second requirement.

Automatic rejection of invalid records upon write operations is a native behavior when constraints are violated, aligning with the need to reject invalid data.

Why Other Options are Less Suitable:

A. Use a SELECT statement with WHERE conditions:

Validation only at query time, not at data insertion, failing to enforce data quality during writes. Does not reject invalid records upon insertion, only filters them out when querying.

B. Create a view that filters out invalid rows:

Similar to A, **enforcement at query time**, not at insertion, thus not preventing invalid data from being stored. **Views do not enforce data integrity** on the underlying table; they merely provide a filtered view of the data.

D. Configure row-level security (RLS):

RLS controls access to data, not the integrity of the data being inserted. It can hide invalid records from certain users but does not prevent invalid data from being stored.

Does not reject invalid records upon insertion; it's a security, not a data validation, mechanism.

References

[Azure Databricks - Working with Delta Tables: Constraints](#)

[Azure Databricks - Unity Catalog: Table and Column Access Control](#) Note: While the second link focuses on

access control, the first directly supports the correct answer. The Unity Catalog link is included for context on the environment's capabilities, though it doesn't directly address the question's solution.

Question: 40

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two Delta tables named Table1 and Table2 of the same data type.

Table1 contains a column named Column1. Table2 contains a column named Column2.

You run the following query.

```
SELECT Column1 -  
FROM Table1 -  
GROUP BY Column1 -  
HAVING COUNT(*) > 1 -  
INTERSECT -  
SELECT Column2 -  
FROM Table2 -  
GROUP BY Column2 -  
HAVING COUNT(*) > 1;
```

What occurs when you run the query?

- A. Values appear in both tables more than once.
- B. Values appear in either table more than once.
- C. Values appear in Table2 but NOT Table1.
- D. Values appear in Table1 more than once.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Running the provided query on Azure Databricks with Unity Catalog enabled, let's break down the query's components and analyze the outcomes for each option:

Query Breakdown:

Two separate SELECT statements, each querying a different table (Table1 and Table2) but similar structure:

Selecting a specific column (Column1 from Table1, Column2 from Table2)

Grouping by the selected column

Applying a HAVING clause to filter groups with more than one occurrence (COUNT(*) > 1)

INTERSECT operator used to combine the results of the two SELECT statements

Analysis of Options:

A. Values appear in both tables more than once.

Correctness: The INTERSECT operator returns only the values that are common to both result sets. For a value to be in the final result, it must satisfy the HAVING COUNT(*) > 1 condition in **both** tables. Thus, values in the output **must** appear more than once in **both** Table1 (in Column1) and Table2 (in Column2).

Why it's the best choice: Directly aligns with the semantics of INTERSECT and the conditions applied in both queries.

B. Values appear in either table more than once.

Suitability: Less Suitable. This describes a scenario better suited for a UNION operator (without the EXCEPT or INTERSECT modifier), not INTERSECT. INTERSECT requires presence in **both** sets.

C. Values appear in Table2 but NOT Table1.

Suitability: Not Suitable. The presence of INTERSECT and the structure of both queries require commonality between the two result sets, making this outcome impossible for the given query.

D. Values appear in Table1 more than once.

Suitability: Partially Suitable but Incorrect for the Question's Scope. While true that values in the result appear more than once in Table1, this option neglects the requirement for these values to also appear more than once in Table2 as dictated by the INTERSECT operator.

Conclusion: Given the query's structure and the operational semantics of the INTERSECT operator in conjunction with the HAVING COUNT(*) > 1 clause in both queries, **Option A** is the correct answer because it accurately reflects the condition that values must appear more than once in **both** Table1 and Table2 to be included in the query's output.

References

[Azure Databricks: SQL Syntax - INTERSECT](#)

[Azure Databricks: Data Catalog \(Unity Catalog\)](#)

Question: 41

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two managed Delta tables named sales.schema1.table1 and sales.schema1.table2. sales.schema1.table1 contains sales data from the current year. sales.schema1.table2 contains historical data.

You need to load all the rows from sales.schema1.table1 into sales.schema1.table2. The solution must preserve any existing data in sales.schema1.table2 and minimize processing effort.

Which command should you run?

- A. INSERT OVERWRITE sales.schema1.table2 SELECT * FROM sales.schema1.table1;
- B. CREATE TABLE sales.schema1.table2 AS SELECT * FROM sales.schema1.table1;
- C. INSERT INTO sales.schema1.table2 SELECT * FROM sales.schema1.table1;
- D. CREATE OR REPLACE TABLE sales.schema1.table2 AS SELECT * FROM sales.schema1.table1;

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct command to load all rows from sales.schema1.table1 into sales.schema1.table2 while preserving existing data in sales.schema1.table2 and minimizing processing effort is:

C. INSERT INTO sales.schema1.table2 SELECT * FROM sales.schema1.table1;

Why C is the Best Option:

Preserves Existing Data: Unlike options A and D, which overwrite or replace the table (potentially losing existing historical data), INSERT INTO adds new rows to the existing table, preserving its current content.

Minimizes Processing Effort: Since it only appends new data, it avoids the overhead of recreating the entire

table (as in options B and D) or overwriting it (option A), which would require additional storage and processing resources to temporarily store the combined data before replacing the original table.

Why Other Options are Less Suitable:

A. INSERT OVERWRITE ...: Overwrites the target table, **losing existing historical data** in sales.schema1.table2. Contrary to the requirement.

B. CREATE TABLE ... AS SELECT ...: **Recreates the table**, losing existing data and potentially altering schema or permissions not specified in the query. Does not preserve the table's identity.

D. CREATE OR REPLACE TABLE ... AS SELECT ...: Similar to B, **replaces the table**, which could lead to data loss (if the table had data not captured by the SELECT statement) and alter unintended aspects of the table's configuration.

Key Technical Points:

Unity Catalog Enablement: Not directly influencing the choice here since the question focuses on DML (Data Manipulation Language) operations compatibility with Delta tables, which Unity Catalog supports.

Delta Tables: Support ACID transactions, making INSERT INTO a safe, efficient choice for appending data without overwriting.

References

1. **Apache Spark Documentation - Delta Lake:** <https://spark.apache.org/docs/latest/delta-lake/getting-started-intro.html> (See "Data Manipulation" section for INSERT INTO syntax and behavior)
2. **Azure Databricks Documentation - Unity Catalog:** <https://docs.databricks.com/governance/unity-catalog/index.html> (Overview of Unity Catalog capabilities, though not directly applicable to the DML choice, confirms compatibility with Delta tables)

Question: 42

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog.
You plan to run the following PySpark code.

```
(df.writeStream
  .format("delta")
  .option("checkpointLocation", CHECKPOINT_LOCATION)
  .outputMode("append")
  .option("mergeSchema", "true")
  .trigger(availableNow=True)
  .toTable(BRONZE_TABLE)
)
```

Answer Area

Statements

Yes

No

New columns from the incoming JSON payload will be added automatically to the target Delta table.

☐☐

Previously processed data will **NOT** be reprocessed if the pipeline fails.

☐☐

Each new batch of telemetry data will overwrite the existing data in the target Delta table.

☐☐

Answer:

Answer Area

Statements

New columns from the incoming JSON payload will be added automatically to the target Delta table.

Yes

☒

No

☐

Previously processed data will **NOT** be reprocessed if the pipeline fails.

☒☐

Each new batch of telemetry data will overwrite the existing data in the target Delta table.

☐☒

Explanation:

Statement 1: Yes

Reasoning: Auto Loader provides built-in, automatic **schema evolution**. By default, when new fields or columns appear in incoming JSON files, Auto Loader updates the target Delta table schema dynamically (addNewColumns mode) without failing the pipeline.

Statement 2: Yes

Reasoning: Auto Loader maintains an internal state using directory checkpoints (checkpointLocation). If the streaming pipeline crashes or fails, it resumes exactly from the last recorded state and **will not reprocess** files that were already committed to the target table.

Statement 3: No

Reasoning: Auto Loader functions as an incremental or streaming reader that **appends** new data files to the target Delta table. It does not wipe out or overwrite your historical data with each batch.

Question: 43

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You plan to ingest data from CSV files stored in Azure Data Lake Storage Gen2. New rows are appended frequently.

You need to implement a data ingestion solution that meets the following requirements:

New data must be available in near-real-time (NRT).

The data must be stored in managed Delta tables.

The solution must minimize custom code and maintenance effort.

What should you include in the solution?

- A. scheduled Apache Spark batch jobs
- B. an Azure Data Factory pipeline
- C. Auto Loader
- D. an external table that references the CSV files

Answer: C

Explanation:

Technical Justification for Choosing C: Auto Loader

Why C (Auto Loader) is the Best Option:Auto Loader is the most suitable choice for this scenario due to the following reasons:

Near-Real-Time (NRT) Capability: Auto Loader is designed to ingest data in near-real-time, aligning perfectly with the requirement for new data to be available promptly.

Integration with Managed Delta Tables: Auto Loader seamlessly integrates with Delta Lake, allowing for the efficient storage of ingested data in managed Delta tables, meeting the second requirement without additional configuration overhead.

Minimal Custom Code and Maintenance: Auto Loader is a low-code solution that automatically handles the ingestion process, including detecting new files and loading them into Delta tables, thus minimizing custom code and maintenance effort.

Why Other Options are Less Suitable:

A. Scheduled Apache Spark Batch Jobs:

Inadequate for NRT: Scheduled batch jobs cannot provide near-real-time data ingestion as they process data in predefined intervals, leading to delays.

More Custom Code: Requires writing custom Spark code to detect new files and load them into Delta tables, increasing maintenance.

B. An Azure Data Factory (ADF) Pipeline:

Additional Complexity: While ADF can trigger Spark jobs, it introduces an extra layer of complexity and does not inherently support NRT ingestion without significant setup.

Not Optimized for NRT Delta Table Updates: Less efficient for continuous, real-time updates compared to Auto Loader.

D. An External Table Referencing the CSV Files:

Does Not Meet Storage Requirement: The data would not be stored in managed Delta tables but rather referenced in its original CSV format.

No Ingestion Mechanism: Simply defines a table over existing data; does not provide a mechanism for ingesting new data into a managed table format.

Correct Answer: C. Auto Loader

References:

[Azure Databricks - Auto Loader](#)

[Delta Lake - Real-Time Data Ingestion with Auto Loader](#)

Question: 44

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Sales_orders.

Sales_orders stores historical sales data.

You receive a daily CSV file daily that contains new sales records only. The file does NOT contain updates to existing rows.

You need to load the daily data into Sales_orders. The solution must meet the following requirements:

Preserve the existing data.

Add only the new records.

Minimize processing effort.

Which command should include in the loading strategy?

A. UPDATE

B. INSERT OVERWRITE

C. INSERT INTO

Answer: C

Explanation:

Technical Justification for Choosing Option C: INSERT INTO

To load daily CSV files containing new sales records into the existing Delta table Sales_orders in an Azure Databricks workspace enabled for Unity Catalog, while preserving existing data, adding only new records, and minimizing processing effort, **INSERT INTO** is the most suitable command. Here's why:

Why C: INSERT INTO is the Best Choice

Preserve Existing Data: INSERT INTO adds new records to the table without affecting the existing data, thereby meeting the requirement of preserving historical sales data.

Add Only New Records: This command is designed to insert new data, aligning perfectly with the scenario where only new sales records are added daily.

Minimize Processing Effort: Since INSERT INTO only processes the new data to be inserted (without needing to overwrite the entire table or update existing rows), it minimizes the processing effort compared to other options.

Why Other Options are Less Suitable

A. UPDATE:

Inapplicable for Adding New Records: UPDATE is used to modify existing rows, which does not apply since the daily file contains only new records with no updates to existing data.

Processing Overhead: Even if theoretically possible to use with a workaround (e.g., inserting into a temp table and then merging), the primary intent of UPDATE does not align with the requirement, making it less efficient.

B. INSERT OVERWRITE:

Destructive to Existing Data: INSERT OVERWRITE would replace the entire table with the new data, erasing the historical sales data, which violates the requirement to preserve existing data.

Excessive Processing for No Benefit: Overwriting the entire table each day would involve significantly more processing than necessary, given that only a small set of new records needs to be added.

Conclusion

Given the requirements, **INSERT INTO** is the optimal choice for loading daily new sales records into the Sales_orders Delta table in Azure Databricks, as it preserves existing data, efficiently adds new records, and minimizes processing effort.

References

[Azure Databricks - INSERT INTO \(Delta Lake\)](#)

[Azure Databricks - Loading Data into Delta Lake](#)

Question: 45

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace.

You need to ingest streaming data from Azure Event Hubs by using Apache Spark Structured Streaming. The solution must authenticate to Event Hubs and read the event payload.

How should you complete the PySpark code segment? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
events = (  
    spark.readStream  
        .format("eventhubs")  
        .option( , "<auth_value>")  
  
        .load()  
    )  
payload = events.selectExpr( )
```

"eventhubs.connectionString"
"eventhubs.consumerGroup"
"eventhubs.startingPosition"

"CAST(body AS STRING) AS payload"
"CAST(value AS STRING) AS payload"
"CAST(partition AS STRING) AS payload"

Answer:

Answer Area

```
events = (  
    spark.readStream  
        .format("eventhubs")  
        .option( , "<auth_value>")  
  
        .load()  
    )  
payload = events.selectExpr( )
```

"eventhubs.connectionString"
"eventhubs.consumerGroup"
"eventhubs.startingPosition"

"CAST(body AS STRING) AS payload"
"CAST(value AS STRING) AS payload"
"CAST(partition AS STRING) AS payload"

Explanation:

"eventhubs.connectionString."

The `.option()` method expects a key string that specifies the authorization credential format. In the Spark Event Hubs connector, the credential containing the target namespace authorization key is explicitly passed using the "eventhubs.connectionString" configuration parameter.

"CAST(body AS STRING) AS payload"

When Azure Event Hubs ingests streaming data into Spark, the raw binary message payload itself is stored within the body column of the streaming DataFrame (unlike Kafka, which stores it in the value column). To extract and deserialize this stream into a readable text format, you must cast the body column to a string type.

Question: 46**Deep Dumps****HOTSPOT -**

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named db1.sales_orders. db1.sales_orders is updated nightly and has change data feed (CDF) enabled.

You need to ingest all the changes from the db1.sales_orders table, including inserts, updates, and deletes, into a downstream pipeline.

How should you complete the PySpark code segment? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
spark.readStream \  
  .format("delta") \  
  .option(  , "true") \  
      
      
      
  .option(  , "false") \  
      
      
      
  .table("db1.sales_orders")
```

Answer:**Answer Area**

```
spark.readStream \  
  .format("delta") \  
  .option(  , "true") \  
      
      
      
  .option(  , "false") \  
      
      
      
  .table("db1.sales_orders")
```

Explanation:**"readChangeFeed"**

To enable Delta Lake Change Data Feed (CDF) to track row-level changes (inserts, updates, and deletes) over time, the property "readChangeFeed" must be explicitly set to "true".

ignoreChanges .

When "readChangeFeed" is enabled, Delta Lake natively handles updates and deletions through the change data feed protocol. Therefore, supplemental legacy safety options like "ignoreChanges" or "ignoreDeletes" are explicitly set to "false" because the stream does not need to blindly ignore or bypass those file modifications anymore.

Question: 47**Deep Dumps**

You have an Azure Databricks workspace that uses Unity Catalog.

You have a Lakeflow Spark Declarative Pipelines (SDP) pipeline that ingests data into a managed Delta table named Table1. Table1 is used for analytics.

New columns are added to the source data, causing pipeline failures during writes to Table1.

You need to prevent the pipeline failures. The solution must ensure that schema changes are detected and handled.

What should you do?

- A. Disable schema enforcement for Table1.
- B. Use row filters to exclude records that have new columns.
- C. Enable schema evolution.
- D. Create a separate table for each schema version.

Answer: C**Explanation:****Technical Justification for Correct Answer: C (Enable Schema Evolution)**

Why C is the Best Option: Enabling schema evolution is the most suitable solution because it allows the managed Delta table (Table1) to adapt to schema changes dynamically. This feature, introduced in Delta Lake, permits the addition of new columns without causing pipeline failures. When schema evolution is enabled, Apache Spark (the engine powering Azure Databricks) can automatically detect and handle new columns during writes, ensuring the pipeline's continuity for analytics purposes. This approach maintains data integrity, supports flexible schema management, and aligns with best practices for handling evolving data sources in a unified data catalog like Unity Catalog.

Why Other Options are Less Suitable:**A. Disable Schema Enforcement for Table1:**

Drawback: Disabling schema enforcement compromises data governance and integrity. It allows inconsistent data, which can lead to errors in downstream analytics, contradicting the managed and unified approach of Unity Catalog.

Technical Impact: This could result in unexpected behavior in queries and analyses relying on Table1, especially in a shared catalog environment.

B. Use Row Filters to Exclude Records with New Columns:

Drawback: Excluding records with new columns results in data loss. This is particularly problematic if the new columns contain critical information for analytics.

Technical Limitation: Implementing row filters for dynamic schema changes can be cumbersome and may not scale well with frequently evolving source data.

D. Create a Separate Table for Each Schema Version:

Drawback: Managing multiple tables for each schema version introduces operational complexity, increases storage costs, and complicates analytics queries that need to aggregate data across versions.

Scalability Issue: As the number of schema versions grows, so does the management overhead, making this approach impractical for long-term data management in Azure Databricks with Unity Catalog.

Correct Action: C. Enable Schema Evolution

Action Steps for Implementation:

1. Navigate to your Azure Databricks Workspace.
2. Access the Delta Table **Table1** through the Unity Catalog interface or via Databricks CLI/SQL.
3. **Enable Schema Evolution** on **Table1**. This can typically be done through the table's properties/settings in the UI, or via a Spark SQL command like `ALTER TABLE Table1 SET TBLPROPERTIES ('delta.schemaEvolution' = 'true');` for existing tables, and ensuring the `spark.sql.sources.schemaEvolution` option is set to true in your Spark configuration for new tables.
4. Optionally, specify which actions are allowed (e.g., ADD COLUMN) and configure notifications or audits as required by your governance policies.

References:

1. **Azure Databricks - Schema Evolution in Delta Lake**<https://docs.databricks.com/delta-lake/concepts/schema-evolution.html>
2. **Apache Spark Documentation - Schema Evolution**<https://spark.apache.org/docs/latest/sql-programming-guide.html#schema-inference-and-schema-evolution>

Question: 48

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Orders.

You load the Orders table into an Apache Spark DataFrame named df.

You need to create a DataFrame that excludes rows where the order amount is null.

Solution: You run the following expression.

```
df.fillna(0, subset=['order_amount'])
```

Does this meet the goal?

- A. Yes
- B. No

Answer: B

Explanation:

Technical Justification

The proposed solution, `df.fillna(0, subset=['order_amount'])`, does **not** meet the goal of creating a DataFrame that excludes rows where the order amount is null. Here's why this solution is less suitable and an explanation of a more appropriate approach:

Why the Given Solution (A) is Incorrect (Correct Answer: B)

Intent vs. Outcome: The goal is to **exclude** rows with null `order_amount`, but `fillna(0, subset=['order_amount'])` **replaces** null values with 0, thus **retaining** all rows. This alters the data's semantic meaning without actually filtering out the desired rows.

Data Integrity Concern: Replacing nulls with a specific value (in this case, 0) can lead to incorrect analysis if the absence of a value (null) is distinct from the presence of a zero value in the business context.

A More Suitable Approach (Implicit, since the task is to justify the Correct Answer B)

Filtering: To **exclude** rows with null `order_amount`, a more appropriate operation would be to use a filter, for example: `df.filter(df.order_amount.isnotnull())` or equivalently, `df.dropna(subset=['order_amount'])`.

filter() with Condition: Explicitly filters out rows based on a condition (`isnotnull()`), directly achieving the goal.

dropna(): Specifically designed for dropping rows with null values, making the intent clear and aligned with the goal.

Key Points for Choosing the Correct Approach:

Goal Alignment: Ensure the operation (filter, replace, etc.) aligns with the stated objective.

Data Semantics: Consider the implications of replacing vs. filtering null values on data analysis and business decisions.

Code Clarity and Intent: Choose methods that clearly communicate their purpose (e.g., `dropna()` for removing nulls).

References

[Apache Spark Documentation - Filtering Data](#)

[Apache Spark Documentation - Dropping Rows with Null Values](#)

Question: 49

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Orders.

You load the Orders table into an Apache Spark DataFrame named `df`.

You need to create a DataFrame that excludes rows where the order amount is null.

Solution: You run the following expression.

```
df.filter(df.order_amount.isNotNull())
```

Does this meet the goal?

A. Yes

B. No

Answer: A

Explanation:

Technical Justification

Goal: Create a DataFrame excluding rows where the order_amount is null from the existing df loaded with the Orders Delta table.

Proposed Solution Analysis

Solution: `df.filter(df.order_amount.isNotNull())`

Assessment: A. Yes, this meets the goal.

Rationale for Correctness (Why A is Best)

Filtering Logic: The `filter()` method in Apache Spark is appropriately used to narrow down the rows based on a condition.

Null Check: `.isNotNull()` is the correct method to check for non-null values in Spark DataFrames, ensuring that only rows with non-null order_amount values are included in the resulting DataFrame.

Efficiency: This approach is efficient as it leverages Spark's optimized filtering capabilities without unnecessarily complicating the query or introducing performance bottlenecks.

Unity Catalog and Delta Table Compatibility: The solution is compatible with Unity Catalog (for metadata management) and Delta tables (for data storage), as the filtering operation does not depend on these features but works seamlessly with them.

Why Other Options are Less Suitable (Hypothetical Analysis since only one option is provided)

(No other options provided for comparison, but for educational purposes):

Incorrect Null Handling (e.g., using `!= null`): In Spark, `!= null` would not correctly filter out null values due to Spark's SQL handling of nulls. `.isNotNull()` is the correct approach.

Using `dropna()` without Specifications: While `dropna()` could be used, it would remove all rows with any null value, not just order_amount. The proposed solution is more targeted.

Complexity Introduction (e.g., unnecessary joins or subqueries): Unnecessary complexity would make the solution less efficient and harder to maintain, which the proposed solution avoids.

References

[Apache Spark API Documentation - DataFrame Filter Method](#)

[Delta Lake Documentation - Querying Delta Tables with Spark](#)

Question: 50

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Orders.

You load the Orders table into an Apache Spark DataFrame named df.

You need to create a DataFrame that excludes rows where the order amount is null.

Solution: You run the following expression.

```
df.dropna(subset=["order_amount"])
```

Does this meet the goal?

A. Yes

B. No

Answer: A

Explanation:

Technical Justification

Goal: Create a DataFrame that excludes rows where the order amount is null from the existing df loaded with the Orders Delta table.

Solution Analysis for `df.dropna(subset=["order_amount"])`

Why it meets the goal (A. Yes is correct):

`dropna()` is a Spark DataFrame method designed to remove rows with null values.

Specifying `subset=["order_amount"]` ensures that only rows with a null value in the `order_amount` column are excluded, leaving other columns' null values untouched if present in other columns, which aligns with the goal's specificity.

The operation returns a new DataFrame (as per Spark's immutable nature), which is what is desired without modifying the original df.

Why other hypothetical options might be less suitable (assuming for the sake of explanation, though not provided):

Using a generic `dropna()` without specifying a subset: Would remove rows with any null values across all columns, not just `order_amount`, potentially removing more data than intended.

Filtering with `.filter(order_amount != null)` or equivalent: Essentially achieves the same result as `dropna(subset=...)` for this specific case. However, `dropna` is more readable and efficient when the intent is explicitly to handle null values.

Using SQL Query on the Delta Table Directly (if considered an option): While viable (`SELECT * FROM Orders WHERE order_amount IS NOT NULL`), the question context implies working within the DataFrame (df) already loaded, making the provided solution more contextually appropriate.

Conclusion: The solution `df.dropna(subset=["order_amount"])` precisely meets the goal by efficiently and clearly excluding only the rows with null `order_amount` values from the DataFrame, without unnecessary data removal or complicating the workflow given the context.

References

1. **Apache Spark Documentation - DataFrame `dropna()`:**
<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrame.dropna.html>
2. **Azure Databricks - Working with Null Values:** <https://docs.databricks.com/data/data-sources/welcome-to-databricks#working-with-null-values> (Note: While not exclusively about `dropna()`, it covers handling nulls in the context of Databricks)

Question: 51

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Orders.

You load the Orders table into an Apache Spark DataFrame named df.

You need to create a DataFrame that excludes rows where the order amount is null.

Solution: You run the following expression.

```
df.filter(df.order_amount != None)
```

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification for Solution Evaluation

Goal: Create a DataFrame excluding rows where the order_amount is null from the existing Orders Delta table loaded into df.

Proposed Solution: `df.filter(df.order_amount != None)`

Evaluation and Justification:

Why the Proposed Solution is Less Suitable (Correct Answer: B. No):

In Apache Spark (and Python in general), None is used to represent the absence of a value. However, when comparing to None, the correct operator to use is is (identity operator), not != (inequality operator). The expression `df.order_amount != None` will not correctly filter out null values due to this operational mistake. The correct way to check for null (absence of any object) in Spark SQL or DataFrame API is to use `.isNull()` or the SQL keyword IS NULL when working with SQL queries.

Correct Approach (Not Provided as an Option but Included for Clarity):

Recommended Solution: `df.filter(df.order_amount.isNotNull())` or equivalently, `df.filter(df.order_amount != lit(None))` is not advisable due to the aforementioned None comparison issue; however, `.isNotNull()` clearly and correctly filters out null values.

Why Other Options (Implicitly Considered):

Although not provided, other incorrect options might include:

Using `df.order_amount == None` (incorrect operator for null check)

Omitting the filter altogether or applying an unrelated filter

Misinterpreting the null check with an equality check for a string 'None' or similar

Conclusion: Given the operational mistake in the comparison (`!= None` instead of `is None` or using `.isNull()`/`.isNotNull()`), the proposed solution **does not meet the goal**, making **B. No** the correct answer.

References:

1. **Apache Spark Documentation - Filtering Data:**

<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrame.filter.html>

2. **Azure Databricks - Working with Nulls in Databricks:** <https://docs.databricks.com/workshops/data-engineering-bad-data/null-values.html> (Note: While the link focuses on a workshop, the principles

Question: 52**Deep Dumps**

DRAG DROP -

You have an Azure Databricks workspace.

You have an Azure Data Lake Storage Gen2 account named account1.

You need to use a Databricks notebook to read CSV files from account1. The data from each new CSV file must be added to an existing Delta table named customers.

How should you complete the PySpark code segment? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

"csv"

"delta"

"append"

"overwrite"

Answer Area

```
basePath = "abfss://data@storage1.dfs.core.windows.net/"
df = (
    spark.read
        .format("csv")
        .option("header", "true")
        .load(basePath)
)
df.write
    . format( Value )
    . mode( Value )
    .save(basePath + "delta/customers")
```

Answer:**Values**

"csv"

"delta"

"append"

"overwrite"

Answer Area

```
basePath = "abfss://data@storage1.dfs.core.windows.net/"
df = (
    spark.read
        .format("csv")
        .option("header", "true")
        .load(basePath)
)
df.write
    . format( "delta" )
    . mode( "overwrite" )
    .save(basePath + "delta/customers")
```

Explanation:**format("delta")**

The save destination URI explicitly points to `basePath + "delta/customers"`. In Azure Databricks, to build a reliable, ACID-compliant Delta Lake table structure, the file format parameter must be set to "delta".

mode("overwrite")

Choose "overwrite" if your question text specifies that you need to completely replace or refresh the target data, or ensure no duplicate rows remain from previous testing cycles.

Question: 53**Deep Dumps****HOTSPOT -**

You have an Azure Databricks workspace that is enabled for Unity Catalog.
You plan to run the following PySpark code.

```
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "/cloud/schema")
  .option("cloudFiles.schemaEvolutionMode", "failOnNewColumns")
  .load("/data/source")
  .writeStream
  .option("checkpointLocation", "/data/checkpoint")
  .toTable("table1")
)
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
When a new element is found in the source data, the element will be written to table1 successfully.	☐	☐
The streaming process will resume from a failure without data loss.	☐	☐
Data will be written to a JSON file.	☐	☐

Answer:**Answer Area**

Statements	Yes	No
When a new element is found in the source data, the element will be written to table1 successfully.	☒	☐
The streaming process will resume from a failure without data loss.	☒	☐
Data will be written to a JSON file.	☐	☒

Explanation:**Statement 1: Yes**

Reasoning: With Auto Loader's default schema evolution mode enabled, when a new column or element is detected in the source JSON data, it will be automatically and successfully merged into table1.

Statement 2: Yes

Reasoning: Spark Structured Streaming uses checkpoints (checkpointLocation) to maintain state. If a failure occurs, the engine restarts from the last recorded offset, ensuring it resumes **without any data loss**.

Statement 3: No

Reasoning: The destination target is an optimized **Delta table format** (table1) stored in Parquet format, not an unindexed raw JSON file.

Question: 54

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a managed Delta table named Table1. Table1 stores customer profile data.

Business users must analyze how customer profile records change over time. They must also be able to query earlier versions of the table.

You need to implement a solution that:

Maintains persistent historical versions of customer profile records for long-term analysis.

Allows users to query earlier versions of the Delta table.

Minimizes maintenance effort.

What should you do? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

To record historical changes:

- Enable change data feed (CDF).
- Create a snapshot copy of Table1.
- Implement a Type 2 slowly changing dimension (SCD).

To support temporal analysis:

- Use checkpoint files.
- Use Delta Lake time travel.
- Implement Type 2 slowly changing dimension (SCD).

Answer:

Answer Area

To record historical changes:

- Enable change data feed (CDF).
- Create a snapshot copy of Table1.
- Implement a Type 2 slowly changing dimension (SCD).

To support temporal analysis:

- Use checkpoint files.
- Use Delta Lake time travel.
- Implement Type 2 slowly changing dimension (SCD).

Explanation:

To record historical changes: Implement a Type 2 slowly changing dimension (SCD).

A Type 2 SCD manages long-term, permanent row-level historical tracking by automatically splitting updating records into distinct rows mapped with specific validity timestamps (start_date, end_date), ensuring full audit-

trail integrity for dimensions.

To support temporal analysis: Use Delta Lake time travel.

[Delta Lake time travel](#) is the built-in feature designed specifically for point-in-time query auditing and temporal analysis, allowing users to query old versions of a table exactly as they appeared at a specific timestamp or transaction number.

Question: 55

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You need to ingest data from the CSV files stored in an external Azure Data Lake Storage Gen2 container into a managed Delta table. The solution must meet the following requirements:

Load data incrementally as new files arrive.

Minimize custom code and operational effort.

How should you complete the SQL statement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

▼ COPY INTO CREATE TABLE CREATE TABLE AS SELECT	main.sales.orders
▼ FILE FROM LOCATION	`abfss://raw@storage1.dfs.core.windows.net/orders/`
FILEFORMAT = CSV	

Answer:

Answer Area

▼ COPY INTO CREATE TABLE CREATE TABLE AS SELECT	main.sales.orders
▼ FILE FROM LOCATION	`abfss://raw@storage1.dfs.core.windows.net/orders/`
FILEFORMAT = CSV	

Explanation:

COPY INTO

The command COPY INTO is Databricks' optimized SQL command used to load data idempotently from a cloud storage file landing zone directly into a Delta Lake table.

FROM

According to the mandatory ANSI SQL syntax rules for the COPY INTO clause, the target table name is immediately followed by the keyword FROM to designate the external source URI location.

Question: 56

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Table1. Table1 is used as a dimension table and contains the following columns: id name address. You need to apply a Type 1 slowly changing dimension (SCD) approach when updates are processed in the name and address columns. What will occur when an update is processed?

- A. A new row will be added for each change to a name and address.
- B. The value of the id column will be updated.
- C. The existing name and address values will be overwritten.
- D. Historical versions of the row and its effective dates will be retained.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

When implementing a Type 1 Slowly Changing Dimension (SCD) approach, the primary objective is to always reflect the most current value for an attribute in the dimension table, without maintaining a history of previous values. In the context of Azure Databricks with Unity Catalog and a Delta table (Table1), here's why each option behaves as it does in relation to the SCD Type 1 requirement:

Correct Answer: C - The existing name and address values will be overwritten.

Justification for C: In a Type 1 SCD approach, updates to the dimension table simply overwrite the existing values with the new ones. Since Table1 is a Delta table, which supports ACID transactions and fine-grained updates, when an update is processed for the name and address columns, Delta Lake will efficiently update the existing row by overwriting the old values with the new ones. This ensures that only the latest version of the data is visible and used, aligning perfectly with the Type 1 SCD strategy.

Why Other Options are Less Suitable:

A. A new row will be added for each change to a name and address.

Inappropriateness for Type 1 SCD: This behavior describes a Type 2 SCD approach, where history is maintained by adding new rows. This would unnecessarily inflate the table size and contradict the requirement for a Type 1 SCD.

B. The value of the id column will be updated.

Impracticality and Incorrectness: The id column in a dimension table typically serves as a primary key or unique identifier. Updating it would break references and is not the focus of a Type 1 SCD update, which targets descriptive attributes (name and address in this case).

D. Historical versions of the row and its effective dates will be retained.

Alignment with Type 2 SCD: While Delta tables do support versioning and can retain historical versions

(viewable through DESCRIBE HISTORY or AS OF/USING TIMESTAMP), the explicit requirement for a Type 1 SCD means historical versions for update operations are not retained at the table's default view level. This option misinterprets the SCD type required.

References

For deeper dives into the technologies and concepts mentioned:

1. **Delta Lake Updates:** <https://docs.databricks.com/delta-lake/concepts/update-delete-merge.html>
2. **Azure Databricks Unity Catalog:** <https://docs.databricks.com/unified-catalog/index.html>

Question: 57

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Table1.

Table1 contains approximately 1.5 TB of data and the following columns: event_time event_type

Queries against Table1 frequently filter on ranges in event_time.

You need to implement a data layout strategy that improves query performance for filters on event_time, avoids over-partitioning, and remains efficient as the data volume grows over time.

What should you do?

- A. Implement liquid clustering on Table1.
- B. Create a fixed number of partitions based on the total table size.
- C. Partition Table1 by using the event_type column.
- D. Partition Table1 by using the event_time column.

Answer: A

Explanation:

Technical Justification for Correct Answer A

Why A (Implement liquid clustering on Table1) is the best option:

Improves Query Performance for Ranges in event_time: Liquid clustering in Azure Databricks optimizes data placement by clustering data around frequently queried columns, in this case, event_time. This ensures that data for similar event_time ranges are stored together, significantly improving query performance when filtering on ranges of event_time.

Avoids Over-Partitioning: Unlike explicit partitioning, liquid clustering does not create physical partitions that can lead to over-partitioning issues (e.g., too many small files), which can degrade performance. Instead, it dynamically manages data distribution.

Efficiency with Growing Data Volume: As the data volume grows, liquid clustering adapts by continuously optimizing the data layout based on query patterns, ensuring sustained query performance without manual intervention to adjust partitions.

Why Other Options are Less Suitable:

B. Create a Fixed Number of Partitions Based on the Total Table Size

Inflexibility: Fails to specifically optimize for event_time queries.

Guesswork: Determining the "right" fixed number of partitions for optimal performance across varying query patterns is challenging.

C. Partition Table1 by Using the event_type Column

Misaligned with Query Patterns: The primary query optimization need is for event_time ranges, not event_type.

Potential for Over-Partitioning: If event_type has many unique values, this could lead to over-partitioning.

D. Partition Table1 by Using the event_time Column

Risk of Over-Partitioning: Partitioning by a column with a high cardinality (like event_time, which could have a unique value for every row) leads to over-partitioning, resulting in many small files and degraded performance.

Less Efficient for Range Queries Compared to Liquid Clustering: While it targets the correct column, the dynamic nature of liquid clustering is more efficient for range queries on a large, growing dataset.

References

[Azure Databricks - Liquid Clustering](#)

[Azure Databricks - Best Practices for Table Partitioning](#)

Question: 58

Deep Dumps

HOTSPOT -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Overview -

Company Information -

Contoso, Inc. is a renewable energy provider that operates solar and wind farms across North America.

Existing Environment -

Azure Environment -

Contoso has a single Azure Databricks workspace named Workspace1 in the West US Azure region. Workspace1 is enabled for Unity Catalog.

Workspace1 contains all-purpose clusters for both development and production workloads.

The company's Azure environment contains:

In the West US, Central US, and East US Azure regions, Azure event hubs that stream telemetry data and an Azure Data Lake Storage Gen2 account in each region for each hub

A single Azure SQL database in the West US region that hosts enterprise resource planning (ERP) data

An Azure Database for PostgreSQL server in the West US region that stores operational maintenance data

Data Environment -

Contoso ingests the following operational and business data:

Telemetry data: More than 40,000 IoT sensors across 28 sites emit JSON telemetry events every few seconds. Each site sends the events to the nearest event hub, which writes the data into the corresponding Data Lake Storage Gen2 account. These files frequently experience schema drift.

Maintenance logs: Maintenance systems generate historical repair logs, daily incremental updates, technician notes, and unstructured attachments that are stored in the Data Lake Storage Gen2 accounts.

Operational maintenance data: Structured operational maintenance data is stored on the Azure Database for PostgreSQL server.

External weather data: Hourly weather forecasts are retrieved from a REST API and written to the Data Lake Storage Gen2 accounts.

ERP data: Daily CSV extracts of 50 to 100 GB contain equipment metadata, work orders, and purchase order information.

Problem Statements -

The company's existing analytics environment has several issues:

Ingestion -

Telemetry pipelines fall behind during peak loads.

Telemetry ingestion fails when schema drift occurs.

Streaming pipelines reprocess events after a pipeline restarts.

Compute -

Production and development workloads run on the same all-purpose clusters.

Production and development workloads do NOT support autoscaling or workload isolation.

Governance -

The ERP data is duplicated across systems and development teams.

Naming conventions are inconsistent across development teams, regions, and products.

Ownership of the IoT sensors changes over time, and analysts must track the full history of the ownership.

Occasionally, equipment manufacturers must correct data-entry mistakes in equipment names. Historical values are NOT required.

Pipeline operations -

Pipelines lack resiliency, alerting, and centralized scheduling.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement scalable data pipeline orchestration.

Create a managed analytics catalog in Unity Catalog.

Implement a consistent approach to creating curated datasets.

Establish a centralized governance model across ingestion, cleansed, and curated layers.

Grant data engineers access to the ERP tables by using minimal development effort.

Adopt a compute strategy that isolates production workloads and supports autoscaling.

Adopt a slowly changing dimension (SCD) approach to address current data modeling issues.

Technical Requirements -

Contoso identifies the following environment and compute requirements:

Ensure that production ingestion workloads run on compute clusters that can scale automatically during telemetry spikes.

Provide fast and consistent performance for business intelligence (BI) workloads.

Prevent development activity from affecting production pipelines.

Production ingestion workloads must run as scheduled, non-interactive pipelines rather than on shared interactive development clusters.

Contoso identifies the following data ingestion and processing requirements:

Auto-scale ingestion pipelines to handle bursty workloads.

Handle schema drift for the maintenance and telemetry data.

Ingest file-based telemetry data by using minimal operational effort.

Store all the ingested data in a format that supports incremental processing.

Support the continuous ingestion of telemetry data from the event hubs by using exactly-once semantics.

Support the ingestion of the structured maintenance data from the Azure Database for PostgreSQL server.

Build a new telemetry pipeline that ingests raw events from the event hubs, cleanses the data, and publishes curated tables to Unity Catalog.

Ensure that the Apache Spark Structured Streaming pipelines reading from the event hubs write the data into a managed Delta table named `telemetry.raw_events`. The pipelines must support schema drift and resume processing after failures without reprocessing the data.

Contoso identifies the following data modeling and optimization requirements:

Build curated tables that standardize business logic.

Overwrite equipment metadata attributes, such as name, manufacturer, model, and commissioning date, when the attributes change. Historical values are NOT required.

Contoso identifies the following pipeline deployment and operation requirements:

Orchestrate multi-step ingestion and transformation workflows.

Define a clear execution order and dependencies.

Automatically retry failed steps and notify operators.

Schedule ingestion and transformation workloads consistently.

Governance Requirements -

Contoso identifies the following governance requirements:

Centralize the metadata catalog.

Provide isolated development areas that follow standard naming conventions.

Establish a consistent structure for organizing raw, cleansed, and curated data.

Provide a read-only mechanism to reference the ERP data through a foreign catalog.

Business Requirements -

Contoso identifies the following business requirements:

Improve ingestion reliability and reduce operational effort.

Standardize data definitions across development teams.

You need to complete the PySpark code for the Spark Structured Streaming pipelines. The solution must meet the data ingestion and processing requirements.

How should you complete the code segment? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
telemetry_path = "abfss://telemetry@contosodata.dfs.core.windows.net/"
checkpoint_path = "abfss://telemetry@contosodata.dfs.core.windows.net/checkpoints/raw_events"
raw_events = "analytics.telemetry.raw_events"

df = (spark.readStream
      .format("json")
      .load(telemetry_path))

df.writeStream
  .format(
    ("CSV")
    ("delta")
    ("json")
    ("parquet")
  )
  .option(
    ("checkpointLocation", checkpoint_path)
    ("failOnDataLoss", "false")
    ("path", checkpoint_path)
    ("startingOffsets", "earliest")
  )
  .option("mergeSchema", "true")
  .trigger(availableNow=True)
  .table(raw_events)
```

Answer:

Answer Area

```
telemetry_path = "abfss://telemetry@contosodata.dfs.core.windows.net/"
checkpoint_path = "abfss://telemetry@contosodata.dfs.core.windows.net/checkpoints/raw_events"
raw_events = "analytics.telemetry.raw_events"

df = (spark.readStream
      .format("json")
      .load(telemetry_path))

df.writeStream
  .format(
    {"CSV"}
    {"delta"}
    {"json"}
    {"parquet"}
  )
  .option(
    {"checkpointLocation", checkpoint_path}
    {"failOnDataLoss", "false"}
    {"path", checkpoint_path}
    {"startingOffsets", "earliest"}
  )
  .option("mergeSchema", "true")
  .trigger(availableNow=True)
  .table(raw_events)
```

Explanation:

"delta"

In Azure Databricks, writing a stream directly to a catalog database table using `.table()` with schema evolution capabilities (`.option("mergeSchema", "true")`) requires the transactionally robust delta storage format.

"checkpointLocation", checkpoint_path

Spark Structured Streaming require a checkpoint directory to maintain processing state and metadata offsets. Since the variable checkpoint_path is explicitly defined at the top of the script, it must be supplied to the "checkpointLocation" parameter.

Question: 59

Deep Dumps

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Overview -

Company Information -

Contoso, Inc. is a renewable energy provider that operates solar and wind farms across North America.

Existing Environment -

Azure Environment -

Contoso has a single Azure Databricks workspace named Workspace1 in the West US Azure region. Workspace1 is enabled for Unity Catalog.

Workspace1 contains all-purpose clusters for both development and production workloads.

The company's Azure environment contains:

In the West US, Central US, and East US Azure regions, Azure event hubs that stream telemetry data and an Azure Data Lake Storage Gen2 account in each region for each hub

A single Azure SQL database in the West US region that hosts enterprise resource planning (ERP) data

An Azure Database for PostgreSQL server in the West US region that stores operational maintenance data

Data Environment -

Contoso ingests the following operational and business data:

Telemetry data: More than 40,000 IoT sensors across 28 sites emit JSON telemetry events every few seconds.

Each site sends the events to the nearest event hub, which writes the data into the corresponding Data Lake Storage Gen2 account. These files frequently experience schema drift.

Maintenance logs: Maintenance systems generate historical repair logs, daily incremental updates, technician notes, and unstructured attachments that are stored in the Data Lake Storage Gen2 accounts.

Operational maintenance data: Structured operational maintenance data is stored on the Azure Database for PostgreSQL server.

External weather data: Hourly weather forecasts are retrieved from a REST API and written to the Data Lake Storage Gen2 accounts.

ERP data: Daily CSV extracts of 50 to 100 GB contain equipment metadata, work orders, and purchase order information.

Problem Statements -

The company's existing analytics environment has several issues:

Ingestion -

Telemetry pipelines fall behind during peak loads.

Telemetry ingestion fails when schema drift occurs.

Streaming pipelines reprocess events after a pipeline restarts.

Compute -

Production and development workloads run on the same all-purpose clusters.

Production and development workloads do NOT support autoscaling or workload isolation.

Governance -

The ERP data is duplicated across systems and development teams.

Naming conventions are inconsistent across development teams, regions, and products.
Ownership of the IoT sensors changes over time, and analysts must track the full history of the ownership.
Occasionally, equipment manufacturers must correct data-entry mistakes in equipment names. Historical values are NOT required.

Pipeline operations -
Pipelines lack resiliency, alerting, and centralized scheduling.

Requirements -
Planned Changes -
Contoso plans to implement the following changes:
Implement scalable data pipeline orchestration.
Create a managed analytics catalog in Unity Catalog.
Implement a consistent approach to creating curated datasets.
Establish a centralized governance model across ingestion, cleansed, and curated layers.
Grant data engineers access to the ERP tables by using minimal development effort.
Adopt a compute strategy that isolates production workloads and supports autoscaling.
Adopt a slowly changing dimension (SCD) approach to address current data modeling issues.

Technical Requirements -
Contoso identifies the following environment and compute requirements:
Ensure that production ingestion workloads run on compute clusters that can scale automatically during telemetry spikes.
Provide fast and consistent performance for business intelligence (BI) workloads.
Prevent development activity from affecting production pipelines.
Production ingestion workloads must run as scheduled, non-interactive pipelines rather than on shared interactive development clusters.

Contoso identifies the following data ingestion and processing requirements:
Auto-scale ingestion pipelines to handle bursty workloads.
Handle schema drift for the maintenance and telemetry data.
Ingest file-based telemetry data by using minimal operational effort.
Store all the ingested data in a format that supports incremental processing.
Support the continuous ingestion of telemetry data from the event hubs by using exactly-once semantics.
Support the ingestion of the structured maintenance data from the Azure Database for PostgreSQL server.
Build a new telemetry pipeline that ingests raw events from the event hubs, cleanses the data, and publishes curated tables to Unity Catalog.
Ensure that the Apache Spark Structured Streaming pipelines reading from the event hubs write the data into a managed Delta table named `telemetry.raw_events`. The pipelines must support schema drift and resume processing after failures without reprocessing the data.

Contoso identifies the following data modeling and optimization requirements:
Build curated tables that standardize business logic.
Overwrite equipment metadata attributes, such as name, manufacturer, model, and commissioning date, when the attributes change. Historical values are NOT required.

Contoso identifies the following pipeline deployment and operation requirements:
Orchestrate multi-step ingestion and transformation workflows.
Define a clear execution order and dependencies.
Automatically retry failed steps and notify operators.
Schedule ingestion and transformation workloads consistently.

Governance Requirements -
Contoso identifies the following governance requirements:
Centralize the metadata catalog.
Provide isolated development areas that follow standard naming conventions.
Establish a consistent structure for organizing raw, cleansed, and curated data.
Provide a read-only mechanism to reference the ERP data through a foreign catalog.

Business Requirements -
Contoso identifies the following business requirements:
Improve ingestion reliability and reduce operational effort.
Standardize data definitions across development teams.
You need to develop the task logic for a new job in Lakeflow Jobs that processes telemetry data.
Each task must contain only the appropriate logic for its step in the pipeline. The solution must support the planned changes and meet the data ingestion and processing requirements.

What should you do?

- A. Use a single Databricks notebook task that performs ingestion, cleansing, and curation in one script.
- B. Create separate tasks for ingestion, cleansing, and curation.
- C. Create three tasks that each contains the identical logic and use task retries.
- D. Use a single SQL task that performs ingestion, cleansing, and curation by running MERGE commands.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the best option:

Modularity and Maintainability: Breaking down the pipeline into separate tasks for ingestion, cleansing, and curation (Option B) allows for modular development, easier maintenance, and targeted updates without affecting the entire pipeline.

Resiliency and Retry Capability: With separate tasks, if one step fails (e.g., cleansing), only that specific task needs to be retried, minimizing the impact on the overall pipeline and reducing reprocessing time.

Scalability and Resource Optimization: Separate tasks can be optimized for resource usage based on their specific requirements (ingestion might require more resources for handling high-volume telemetry data, while curation might need less), leading to better resource utilization.

Alignment with Requirements:

Pipeline Operations Requirements: Supports "Automatically retry failed steps" and "Schedule ingestion and transformation workloads consistently" through modular task management.

Data Ingestion and Processing Requirements: Facilitates "Handle schema drift" more effectively in a dedicated ingestion task and ensures "incremental processing" support by managing each stage separately.

Why other options are less suitable:

A. Single Databricks Notebook Task

Monolithic Approach: Increases complexity, making maintenance and updates cumbersome.

Single Point of Failure: Failure in any step requires re-running the entire script, wasting resources.

Does Not Support Retry for Specific Steps: Lacks the granularity for targeted retries.

C. Three Tasks with Identical Logic

Redundancy and Waste: Executes the same logic multiple times, wasting computational resources.

No Logical Separation: Does not address the need for separate ingestion, cleansing, and curation steps.

D. Single SQL Task with MERGE Commands

Inappropriate for Ingestion: SQL tasks are not suited for the initial ingestion of large, schema-drifting telemetry data from event hubs.

Scalability Concerns: Might not scale as effectively as separate, optimized tasks for each pipeline stage, especially under high throughput.

Limited Flexibility for Complex Cleansing Logic: MERGE commands are less flexible for complex data cleansing transformations compared to dedicated Spark or Python tasks.

References

[Azure Databricks - Lakehouse Architecture for Data Engineering](#)

[Apache Spark Structured Streaming for Real-Time Data Processing](#)

Question: 60

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that contains an all-purpose cluster named Cluster1.

You discover that out-of-memory (OOM) errors intermittently cause jobs running on Cluster1 to fail.

You need to identify the root cause of the failures by analyzing the runtime execution behavior.

What should you do? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

Diagnostic tool to use:

▼

Cluster event logs
The Apache Spark UI
The Databrick notebook output

Execution level to analyze:

▼

Executors
Jobs
Stages
Tasks

Answer:

Answer Area

Diagnostic tool to use:

▼

Cluster event logs
The Apache Spark UI
The Databrick notebook output

Execution level to analyze:

▼

Executors
Jobs
Stages
Tasks

Explanation:

Diagnostic tool to use: The Apache Spark UI

Reasoning: The Apache Spark UI is the core diagnostic monitoring dashboard built into Databricks. It provides visual access to query execution DAGs, event timelines, shuffle sizes, memory usage, and execution bottleneck metrics that are completely invisible inside standard cluster event logs or raw notebook text printouts.

Execution level to analyze: Tasks

Reasoning: A **Task** is the fundamental individual unit of execution that operates on a single partition of data inside a Spark executor cluster. When diagnosing granular execution behaviors or pinpointing why specific nodes are failing, analyzing the **Tasks** level allows engineers to spot data skew, structural stragglers, or out-of-memory errors on individual partitions.

Question: 61

Deep Dumps

You use Databricks Asset Bundles to manage two jobs and an app.

You need to deploy the bundle to development and production environments. The solution must meet the following requirements:

Deploy the app to both environments.

Deploy only one job to development.

Minimize administrative effort.

What should you use?

- A. separate databricks.yml files for each environment
- B. a variables node in a databricks.yml file
- C. a resources node in a databricks.yml file
- D. a targets node in a databricks.yml file

Answer: D

Explanation:

Technical Justification for Correct Answer D

Requirements Recap:

Deploy the app to both environments (development and production)

Deploy only one job to development

Minimize administrative effort

Why D (a targets node in a databricks.yml file) is the Best Choice:

Environment-Specific Deployments: A targets node in databricks.yml allows specifying different deployment targets (e.g., development, production) with their respective configurations in a single file. This directly addresses the need to deploy the app to both environments and selectively deploy jobs based on the target.

Conditional Deployment of Jobs: By leveraging the targets node, you can define which jobs are deployed to which target, fulfilling the requirement of deploying only one job to the development environment.

Administrative Efficiency: Managing a single databricks.yml file with a targets node minimizes administrative effort compared to maintaining separate files or more complex configurations.

Why Other Options are Less Suitable:

A. Separate databricks.yml files for each environment:

Increased Administrative Effort: Managing multiple files increases overhead.

Redundancy: Duplicate configurations across files can lead to inconsistencies.

B. A variables node in a databricks.yml file:

Limited Control: While useful for parameterization, variables alone don't directly control which resources

(jobs/app) are deployed to which environment.

Additional Logic Required: Would necessitate external scripting to achieve environment-specific deployments, adding complexity.

C. A resources node in a databricks.yml file:

Resource Definition, Not Deployment Control: The resources node is more about defining resources rather than controlling their deployment across different environments.

Lacks Target-Specific Logic: Does not inherently support deploying different subsets of resources to different targets without additional, external configuration.

Correct Answer Justification Summary: The targets node in databricks.yml (Option D) is the most suitable choice because it efficiently manages environment-specific deployments of jobs and apps in a single configuration file, directly meeting all specified requirements with minimal administrative overhead.

References

[Databricks Documentation: Databricks Assets - databricks.yml](#)

[Microsoft Learn: Deploy Databricks Assets to different environments](#)

Question: 62

Deep Dumps

You have an Azure Databricks workspace that contains a Delta table named Table1. Table1 has accumulated obsolete files. You need to reduce storage costs. The solution must preserve 30 days of time travel history. Which two actions should you perform? Each correct answer presents part of the solution. NOTE: Each correct selection is worth one point.

- A. Run the vacuum command on Table1.
- B. Set the delta.deletedFileRetentionDuration table property to 30 days.
- C. Set the delta.logRetentionDuration table property to 30 days.
- D. Run the OPTIMIZE command on Table1.
- E. Reduce the deleted file retention period to one day.

Answer: AB

Explanation:

Technical Justification for Correct Answer: AB

To reduce storage costs in Azure Databricks for Table1 (a Delta table with accumulated obsolete files) while preserving 30 days of time travel history, the following actions are justified:

A. Run the vacuum command on Table1

Why Correct: The VACUUM command in Delta Lake is used to remove files that are no longer needed after deletion or updates, which directly addresses the issue of accumulated obsolete files. Running VACUUM will free up storage by deleting unreferenced files.

Preserves Time Travel History: When used with the appropriately set retention periods, VACUUM ensures that files necessary for time travel within the specified window are retained.

B. Set the delta.deletedFileRetentionDuration table property to 30 days

Why Correct: This property controls how long Delta Lake keeps deleted files before they can be safely

removed by VACUUM. Setting it to 30 days **preserves 30 days of time travel history** by ensuring that enough historical data is retained for point-in-time queries or restores.

Alignment with Requirement: Directly meets the requirement to preserve 30 days of history.

Why Other Options Are Less Suitable

C. Set the delta.logRetentionDuration table property to 30 days

Incorrect for This Context: While important for operational logging and recovery, this property doesn't directly control the retention of deleted files for time travel in the context of reducing storage costs from obsolete files.

D. Run the OPTIMIZE command on Table1

Not Directly Relevant: OPTIMIZE is used to compact files for better query performance. While beneficial for Table1's query efficiency, it doesn't directly address the storage cost issue posed by obsolete files.

E. Reduce the deleted file retention period to one day

Contradicts Requirement: This would significantly reduce the time travel history window, violating the requirement to preserve 30 days of history.

Selected Actions for Solution:

A (Immediate removal of obsolete files)

B (Ensures 30 days of time travel history)

References

[Azure Databricks - Delta Table Vacuum](#)

[Azure Databricks - Delta Table Properties](#)

Question: 63

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.
You have a complex job named Job1 that contains eight tasks. Job1 takes multiple hours to complete.
During the last job run, the final task fails due to a transient issue.
You need to retry the last task without rerunning tasks that have already completed.
What should you do?

- A. Repair the current job run.
- B. Update the job parameters.
- C. Disable and reen able the job schedule.
- D. Restart Job1.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Why A (Repair the current job run) is the best option:Repairing the current job run in Azure Databricks is the most suitable action for retrying the last failed task without rerunning the already completed tasks. This feature allows for the restart of a failed task or a subset of tasks within a job run, leveraging the existing execution context and saving significant time by not redoing successful tasks. Since the failure was due to a

transient issue (implying no code changes are needed), repairing the job directly addresses the problem with minimal overhead.

Why other options are less suitable:

B (Update the job parameters): Updating job parameters would not directly address the issue of retrying a failed task due to a transient error. This action is more suited for changing input values or configurations for a future run, not for resolving a current run's failure.

C (Disable and reenable the job schedule): This option pertains to the scheduling mechanism and does not provide a direct way to retry a specific failed task within a job run. It might be useful for preventing immediate future runs but does not solve the problem of completing the current run.

D (Restart Job1): Restarting the entire job would rerun all tasks, including those that have already successfully completed. This is inefficient given the job's duration (multiple hours) and the fact that only the last task needs retries.

Precision on the Correct Action (A):

Action: Repair the current job run, specifically targeting the failed task.

Rationale: Efficient retry of only the failed task due to transient issues, preserving the resources and time spent on completed tasks.

Tooling/Feature: Leveraging Azure Databricks' job run repair feature, designed for such scenarios.

References

[Azure Databricks: Job Runs and Retry](#)

[Azure Databricks: Repairing Job Runs](#)

Question: 64

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1.

Job1 runs every hour.

Occasionally, the job run takes longer than one hour to complete. Overlapping runs must be prevented to avoid data corruption.

You need to configure the job scheduling behavior.

What should you configure? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Concurrency setting:

▼

Allow concurrent runs.
Limit concurrent runs to one.
Restart the job upon overlap.

Execution behavior:

▼

Cancel the new run.
Queue the new run.
Skip the failed run.

Answer:

Answer Area

Concurrency setting:

▼

Allow concurrent runs.
Limit concurrent runs to one.
Restart the job upon overlap.

Execution behavior:

▼

Cancel the new run.
Queue the new run.
Skip the failed run.

Explanation:

Limit concurrent runs to one.

This sets the maximum concurrency parameter (`max_concurrent_runs`) to exactly 1, preventing multiple job instances from overlapping or competing for the same resources/data writes.

Queue the new run.

Reasoning: When concurrency is capped at 1, setting the behavior to Queue the new run creates an orderly execution pipeline by holding subsequent triggers in a waiting state until the current active run finishes.

Question: 65

Deep Dumps

You have an Azure Databricks workspace that contains a Git folder and uses an Azure Repos Git repository. From the main branch, you create a branch named Branch1 and commit changes to Branch1. You need to incorporate the changes from Branch1 into main. The solution must preserve the commit history in the repository. Which Git operation should you use?

- A. rebase
- B. pull
- C. merge
- D. push

Answer: C

Explanation:

Technical Justification for Correct Answer: C (merge)

To incorporate changes from Branch1 into main while preserving the commit history in the Azure Repos Git repository within the Azure Databricks workspace, the most appropriate Git operation is **merge**. Here's why:

Preserve Commit History: Merging Branch1 into main creates a new merge commit in main that references the tip of Branch1, thereby preserving the entire commit history of both branches. This is crucial for maintaining a clear audit trail of changes.

Correct Operation:

C. merge: Directly addresses the requirement by combining the histories in a way that maintains all commits. You would typically use `git merge Branch1` while checked out on main.

Why Not Other Options:

A. rebase: Rewrites the commit history of Branch1 on top of main, which does not preserve the original commit history as required. It can also lead to conflicts and is generally discouraged when preserving history is a priority.

B. pull: Used to fetch and integrate changes from a remote repository into the current branch. Since the task involves integrating a local branch (Branch1) into main, pull is not directly applicable to the branch integration requirement described.

D. push: Updates the remote repository with changes from the local repository. It does not integrate changes from one branch into another, making it irrelevant to the task of merging Branch1 into main.

Key Technical Points for Choosing Merge:

Operation: `git merge Branch1` (while on main)

Outcome: Preserves commit history, creates a merge commit in main

Suitability for Requirement: High, as it directly meets the preservation of commit history and branch integration requirements.

References

[Microsoft Docs - Git Merge vs. Rebase](#)

[Azure Databricks - Git Version Control](#)

Question: 66

Deep Dumps

You have an Azure Databricks workspace named Workspace1 that uses a Git repository. The repository contains a Databricks notebook named Notebook1.

From the main branch, you create a feature branch named Branch1 and commit changes to Notebook1. Another user commits changes to Notebook1 in main.

When you attempt to merge Branch1 into main, the merge fails due to conflicts.

You need to merge Branch1 into the main branch. The solution must ensure that Notebook1 includes all the changes from both the branches.

What should you do?

- A. From Workspace1, clone the main branch as a new repository.
- B. Apply the main branch changes to Branch1 and resolve the conflicts.
- C. From Workspace1, clone Branch1 as a new repository.
- D. Apply the changes directly to the main branch.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the Best Option: To successfully merge Branch1 into the main branch while ensuring Notebook1 includes all changes from both branches, the most appropriate approach is to **apply the main branch changes to Branch1 and resolve the conflicts** (Option B). This methodology follows standard Git workflows (e.g., GitFlow, GitHub Flow) for handling conflicts in a controlled manner. By updating Branch1 with the latest changes from main, you can identify and resolve conflicts within the feature branch before merging, ensuring a clean and conflict-free merge into main. This approach also keeps the main branch untouched and stable until the merge is successfully resolved.

Why Other Options are Less Suitable:

A. Clone main branch as a new repository: This approach does not address the conflict resolution between Branch1 and main. Cloning creates a new, separate repository, which does not facilitate merging Branch1 into main. It introduces unnecessary complexity and divergence.

C. Clone Branch1 as a new repository: Similar to option A, cloning Branch1 doesn't resolve the merge conflict with main. It merely duplicates Branch1, moving away from the goal of integrating changes into the main branch.

D. Apply changes directly to the main branch: This bypasses the purpose of using a feature branch for isolated development and testing. Directly applying changes to main without conflict resolution in a feature branch risks instability in the main branch and contradicts best practices for collaborative Git workflows.

Resolution Steps for Correct Answer (B):

Checkout Branch1

Pull the latest changes from main into Branch1 (e.g., `git pull origin main` or equivalent in your Git tooling)

Resolve conflicts in Notebook1, ensuring all desired changes from both branches are included

Commit the resolved changes in Branch1

Merge Branch1 into main (now conflict-free)

References:

[Azure Databricks Documentation - Version Control with Git](#)

[Git Documentation - Resolving Merge Conflicts](#)

You have an Azure Databricks workspace that is enabled for Unity Catalog. You need to recommend a pipeline that ingests files from cloud storage, performs cleansing and enrichment transformations, and writes curated Delta tables for analytics. The solution must minimize development effort and provide built-in monitoring and automatic retries. What should you include in the recommendation?

- A. a Databricks notebook triggered by a scheduled job
- B. an Apache Spark Structured Streaming job
- C. a Lakeflow Spark Declarative Pipelines (SDP) pipeline
- D. an Azure Data Factory pipeline that uses data flows

Answer: C

Explanation:

Technical Justification for Correct Answer: C

Why C (Lakeflow Spark Declarative Pipelines (SDP) pipeline) is the Best Option:

Minimize Development Effort: Lakeflow SDP provides a declarative approach to pipeline definition, significantly reducing the coding effort required for data transformations and pipeline orchestration compared to imperative methods.

Built-in Monitoring and Automatic Retries: SDP is designed to integrate seamlessly with Azure Databricks monitoring tools and natively supports retry mechanisms for failed tasks, aligning perfectly with the requirements.

Unity Catalog Compatibility: As a solution built around Azure Databricks, Lakeflow SDP naturally supports Unity Catalog, ensuring secure and managed access to curated Delta tables for analytics.

End-to-End Solution: SDP can handle ingestion, transformations (cleansing and enrichment), and writing to Delta tables in a unified workflow, making it a comprehensive choice for the entire pipeline.

Why Other Options are Less Suitable:

A. Databricks Notebook triggered by a Scheduled Job:

Higher Development Effort: Requires manual coding for all pipeline aspects.

Monitoring and Retries: Not built-in; needs custom implementation.

Suitable for: Ad-hoc or simple workflows, not complex pipelines.

B. Apache Spark Structured Streaming Job:

Real-time Focus: Primarily designed for real-time data streams, not batch file ingestion scenarios.

Development Effort and Monitoring: Similar concerns as with notebooks.

Suitable for: Real-time data processing workloads.

D. Azure Data Factory (ADF) pipeline with Data Flows:

Additional Complexity: Introduces a new service (ADF) to manage, potentially increasing overall complexity.

Unity Catalog and Delta Tables: While possible, the integration is less native compared to a Databricks-centric solution like SDP.

Suitable for: Pipelines requiring integration across multiple Azure services.

Recommendation: Include a **Lakeflow Spark Declarative Pipelines (SDP) pipeline** in your recommendation for its declarative simplicity, built-in monitoring and retry capabilities, native Unity Catalog support, and comprehensive workflow management.

References

1. **Lakeflow Documentation - Spark Declarative Pipelines (SDP):**
https://lakeflow.io/docs/getting_started/
2. **Azure Databricks - Unity Catalog:** <https://docs.databricks.com/unified-catalog/index.html>

Question: 68

Deep Dumps

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1.

Job1 processes raw data files stored in Azure Storage.

New files arrive at unpredictable intervals.

You need to ensure that Job1 starts automatically when new files arrive and does NOT consume compute resources when no data is available.

Which type of job trigger should you use?

- A. manual
- B. file arrival
- C. continuous
- D. scheduled

Answer: B

Explanation:

Technical Justification for Choosing the Correct Option (B. file arrival)

Why B (file arrival) is the best option: The file arrival trigger is specifically designed for scenarios where new files are added to a storage location at unpredictable intervals. This trigger automatically starts the job (in this case, Job1) as soon as new files are detected in the specified Azure Storage location. Since the trigger is event-driven (based on file arrival), it ensures that compute resources are only consumed when there is actual data to process, aligning perfectly with the requirements.

Why other options are less suitable:

A. Manual: This option requires human intervention to start the job every time new files arrive, which is impractical for unpredictable intervals and defeats the purpose of automation.

C. Continuous: A continuous trigger would keep the job running indefinitely, leading to unnecessary consumption of compute resources during periods when no new files have arrived, thus wasting resources.

D. Scheduled: While scheduled triggers can reduce resource waste compared to continuous triggers, they might still consume resources during scheduled runs when no new files are present. Additionally, they cannot guarantee immediate processing of new files if the schedule interval is not frequent enough, leading to potential delays.

Key Characteristics Aligning B (file arrival) with the Requirements:

Event-Driven: Starts the job only when new files are detected.

Efficient Resource Utilization: Compute resources are consumed only when there is data to process.

Timely Processing: Ensures new files are processed promptly after arrival, regardless of the schedule.

References

[Azure Databricks: Job Triggers](#)

[Azure Databricks: File Arrival Trigger for Lakehouse Jobs](#)

Question: 69

You have an Azure Databricks workspace that contains multiple all-purpose clusters. You discover that some clusters remain idle for long periods after users finish their work. You need to reduce compute costs without affecting active workloads. What should you do?

- A. Enable autoscaling.
- B. Convert the clusters into job clusters.
- C. Use spot instances.
- D. Configure automatic termination.

Answer: D

Explanation:

Technical Justification for Correct Answer: D - Configure automatic termination

Why D is the best option: Configuring automatic termination is the most suitable solution to reduce compute costs without impacting active workloads in this scenario. By enabling this feature, idle clusters will automatically shut down after a specified period of inactivity, thereby minimizing unnecessary resource utilization and associated costs. This approach directly addresses the issue of clusters remaining idle for long periods without interfering with ongoing workloads when they are active.

Why other options are less suitable:

A. Enable autoscaling:

Autoscaling adjusts the number of nodes in a cluster based on workload demands, which helps in optimizing resource usage during active periods.

However, it does not address the issue of idle clusters after users finish their work, as it focuses on scaling rather than shutting down unused resources.

Cost Impact: May reduce costs during low usage but doesn't eliminate costs for completely idle clusters.

B. Convert the clusters into job clusters:

Job clusters are optimized for production workloads and batch processing, offering features like isolation and auto-scaling.

This conversion doesn't inherently reduce costs for idle periods, as the core issue of unused resources during extended idle times remains unaddressed.

Cost Impact: Similar to all-purpose clusters if left idle; potential for higher costs due to additional features if not utilized efficiently.

C. Use spot instances:

Spot instances can significantly reduce costs by utilizing unused cloud capacity.

However, their availability is not guaranteed (can be reclaimed by the cloud provider at any time), making them less suitable for workflows requiring predictable resource availability.

Cost Impact: Highly cost-effective but introduces operational risk due to potential interruptions.

Correct Answer Justification Summary:

Direct Issue Addressal: D directly solves the problem of idle clusters.

Cost Efficiency: D ensures no costs are incurred for idle periods.

No Operational Disruption: D does not interfere with active workloads or introduce new operational risks.

References:

1. **Azure Databricks Automatic Termination Documentation**
<https://docs.microsoft.com/en-us/azure/databricks/clusters/configure#automatic-termination>
2. **Azure Databricks Cluster Types and Pricing**
<https://docs.microsoft.com/en-us/azure/databricks/pricing#cluster-types-and-pricing-models>

Question: 70

Deep Dumps

You have an Azure Databricks workspace.

You have an Apache Spark Structured Streaming job named Job1 that processes data continuously and fails periodically due to transient errors.

You need to ensure that Job1 meets the following requirements:

Resumes processing from the point that Job1 failed

Minimizes how long it takes to restart Job1

Minimizes the costs to restart Job1

What should you do?

- A. Decrease the retry interval.
- B. Increase the minimum number of nodes in the cluster.
- C. Add an alert and manually restart Job1.
- D. Implement checkpointing.

Answer: D

Explanation:

Technical Justification for Correct Answer D: Implement Checkpointing

Why D (Implement Checkpointing) is the Best Option: Implementing checkpointing is the most suitable solution for meeting all the specified requirements for Apache Spark Structured Streaming Job1 in an Azure Databricks workspace. Here's why:

Resumes Processing from the Point of Failure: Checkpointing in Spark Structured Streaming allows the job to save the state of the streaming query at regular intervals. Upon failure, the job can resume from the last successful checkpoint, ensuring no data is lost or reprocessed unnecessarily.

Minimizes Restart Time: By having checkpoints, the job doesn't need to restart from the very beginning, significantly reducing the time it takes to get the job back to a processing state.

Minimizes Costs to Restart: Since the job resumes from the last checkpoint and doesn't require a full restart, the computational resources (and thus costs) associated with restarting from scratch are minimized.

Why Other Options are Less Suitable:

A. Decrease the Retry Interval:

Issue: Reducing the retry interval might lead to quicker restart attempts but does not address resuming from the point of failure or inherently minimize startup time/costs. It could also lead to increased costs if retries are frequent without resolving the root cause.

Does Not Meet: Resumes from the point of failure, Minimizes restart time (directly), Minimizes costs effectively.

B. Increase the Minimum Number of Nodes in the Cluster:

Issue: While this might reduce the likelihood of failures due to resource constraints, it does not directly address recovery from transient errors, restarting from the failure point, or minimizing restart time/costs. It increases baseline costs.

Does Not Meet: Resumes from the point of failure, Minimizes restart time, Minimizes costs to restart.

C. Add an Alert and Manually Restart Job1:

Issue: Manual intervention introduces significant delay, increases the time to restart, and does not guarantee resuming from the exact point of failure without additional complex setup. Human error and response time variability are additional drawbacks.

Does Not Meet: Minimizes how long it takes to restart, Minimizes costs to restart (due to manual labor and potential for more frequent, longer downtimes).

References

[Apache Spark Documentation - Structured Streaming: Checkpointing](#)

[Azure Databricks Documentation - Best Practices for Apache Spark Structured Streaming](#)

Question: 71

Deep Dumps

HOTSPOT -

You have an Azure Databricks job named Job1 that contains an ingestion task named Task1 and transformation task named Task2.

You need to ensure that if Task1 fails, the task retries automatically, and Task2 is prevented from running. How should you configure Job1? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Failure handling behavior:

▼

Block downstream tasks.

Continue to the next task.

Skip the failed task.

Retry configuration:

▼

Configure a retry policy for Job1.

Configure a retry policy for Task1.

Rely on the default retry policy of Task1.

Answer:

Answer Area

Failure handling behavior:

Block downstream tasks.
Continue to the next task.
Skip the failed task.

Retry configuration:

Configure a retry policy for Job1.
Configure a retry policy for Task1.
Rely on the default retry policy of Task1.

Explanation:

Failure handling behavior.

Block downstream tasks. This is the default Databricks DAG execution behavior. If Task1 enters a failed state, any subsequent tasks that list Task1 as a parent dependency will be automatically blocked from execution.

Retry configuration

Configure a retry policy for Task1.

In Databricks Workflows, retries are configured explicitly at the Task level, not globally at the Job level. You can set up specific number of retries, retry intervals, and back-off parameters for each individual task object (such as Task1).

Question: 72

Deep **Dumps**

You need to deploy Databricks Asset Bundles to a development environment. The solution must support automated and repeatable deployments across environments. What should you use?

- A. Git folders
- B. the Databricks CLI
- C. the Databricks SDK for Python
- D. the Jobs UI

Answer: B

Explanation:

Technical Justification for Correct Answer: B. the Databricks CLI

Why B. the Databricks CLI is the Best Option: The Databricks CLI is the most suitable choice for deploying

Databricks Asset Bundles to a development environment with automated and repeatable deployments across environments. Here's why:

Automation & Repetition: The CLI provides a command-line interface that can be easily scripted, enabling automated and repeatable deployments across different environments (dev, stage, prod, etc.).

Support for Asset Bundles: The Databricks CLI natively supports the deployment of Databricks Asset Bundles, which include notebooks, jobs, libraries, and other dependencies, ensuring a cohesive deployment.

Environment Management: It allows for clear environment isolation and management through workspace and resource group targeting, crucial for multi-environment setups.

Integration with CI/CD Pipelines: The CLI can be seamlessly integrated with Continuous Integration/Continuous Deployment (CI/CD) tools (e.g., Azure DevOps, Jenkins), further enhancing automation capabilities.

Why Other Options are Less Suitable:

A. Git Folders:

While Git is excellent for version control and can store asset bundles, it doesn't natively support the **deployment** of these bundles to Databricks environments.

Additional scripting would be required to deploy from Git, increasing complexity.

C. the Databricks SDK for Python:

Although the SDK offers programmatic control, using it for deployment would require writing custom Python scripts.

This approach adds development overhead and might not be as straightforward or widely supported as the CLI for deployment tasks.

D. the Jobs UI:

The Jobs UI is primarily designed for job scheduling and monitoring, not for deploying asset bundles.

It lacks the automation and scripting capabilities necessary for repeatable, cross-environment deployments.

Conclusion: Given the requirements for automated, repeatable deployments across environments, the Databricks CLI (Option B) is the most efficient, supported, and scalable choice for deploying Databricks Asset Bundles.

References:

1. **Databricks CLI Documentation** - <https://docs.databricks.com/administration-guide/command-line-interface/index.html>
2. **Deploying to Databricks with CI/CD Pipelines** - <https://docs.databricks.com/dev-tools/ci-cd/deploy-with-ci-cd.html>

Question: 73

Deep Dumps

HOTSPOT -

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1.

Job1 contains three tasks named Task1, Task2, and Task3.

If Task1 fails, Task2 and Task3 must be prevented from running. Successfully completed tasks must NOT rerun during recovery.

You need to configure Job1 to support controlled failure handling and recovery.

What should you configure? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Failure handling behavior:

Continue the job execution after Task1 fails.
Fail the job if Task1 fails.
Skip failed tasks automatically.

Recovery mechanism:

Configure a repair run.
Configure a job parameter.
Configure a retry policy for Task1.

Answer:

Answer Area

Failure handling behavior:

Continue the job execution after Task1 fails.
Fail the job if Task1 fails.
Skip failed tasks automatically.

Recovery mechanism:

Configure a repair run.
Configure a job parameter.
Configure a retry policy for Task1.

Explanation:

Fail the job if Task1 fails.

Reasoning: This is the standard operational standard for multi-task directed acyclic graphs (DAGs). If an isolated upstream root task (Task1) enters an error state, Databricks halts sequential dependency propagation, flags the entire active workflow run instance as failed, and skips any dependent child nodes to prevent dirty data states downstream.

Incorrect Choices:

Continue the job execution after Task1 fails: While this behavior can be artificially configured by modifying conditional execution blocks (like specifying Run if: All done on specific subsequent individual tasks), it is an override condition rather than a structural property applied directly as a global failure handling behavior

setting.

Skip failed tasks automatically: Databricks never blindly silently skips or masks failed tasks automatically; it explicitly changes their status to FAILED and blocks or skips the downstream dependent nodes (Upstream failed).

Recovery mechanism

Configure a retry policy for Task1.

Reasoning: To achieve automated fault tolerance for live processing execution pipelines, an automated **retry policy** must be defined directly inside the task properties layout. This allows the cluster engine to retry transient dropouts natively on the spot before marking the step as terminal.

Alternative Correct Context (Reactive): **Configure a repair run.** if your specific question text mentions **manual post-failure recovery**. A repair run allows engineers to explicitly rerun only the specific failed nodes of an already-completed job without starting the pipeline over from zero.

Incorrect Choice:

Configure a job parameter: Job parameters are global key-value string variables passed down into underlying workloads (e.g., passing a target date directory to notebooks) and have absolutely no logical control functionality over runtime platform recovery or retry automation logic.

Question: 74

Deep Dumps

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1. Job1 contains multiple tasks.

Failures of non-critical tasks must be logged but must NOT trigger notifications. Notifications must be triggered only when critical tasks have failed, and Job1 has completed.

You need to configure the job alerting behavior.

What should trigger a notification?

- A. task success
- B. a job failure
- C. a task failure
- D. job success

Answer: C

Explanation:

Technical Justification for Correct Answer (C: a task failure)

The correct configuration for triggering notifications in this scenario is **C: a task failure**. Here's why:

Why C is correct: The requirement specifies that notifications should be triggered only when **critical tasks** have failed, and the job has completed. Since the distinction between critical and non-critical tasks is made at the task level within the job, the notification trigger must also be set at the task level to allow for this differentiation. This ensures that only failures of critical tasks (once identified as such within the job's task configuration) will trigger notifications, aligning with the specified condition.

Why A is less suitable: A: task success is entirely contrary to the requirement, as the focus is on handling failures, not successes.

Why B is less suitable: B: a job failure is too broad and does not allow for the differentiation between critical and non-critical task failures. A job failure notification would trigger regardless of which task failed, not meeting the specific requirement for only critical task failures to trigger notifications.

Why D is less suitable: D: job success is irrelevant to the failure notification requirements outlined in the question. It pertains to successful job completion, not the handling of task failures.

Key Points for Configuration:

Trigger Specificity: The trigger must be specific to task failures to differentiate between critical and non-critical tasks.

Conditional Notification: Notifications are conditional upon the failure of specific (critical) tasks within the job.

Job Completion Context: The question implies that the distinction between critical and non-critical tasks' failure impact is assessed once the job has completed, allowing for a holistic view of task outcomes before notifying.

References

1. **Azure Databricks Documentation - Alerts and Notifications:**

<https://docs.databricks.com/workflows/job-alerts.html>(Section on Configuring Alerts for Jobs and Tasks provides insight into conditional notification setups)

2. **Microsoft Learn - Configure job alerting behavior in Azure Databricks:**

<https://learn.microsoft.com/en-us/learn/modules/configure-azure-databricks/#configure-job-alerting-behavior>(Covers best practices for alert configurations based on job and task states)

Question: 75

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You have a Lakeflow Spark Declarative Pipelines (SDP) pipeline that writes numerical data to a table named Table1 by using a data quality validation rule named rule1.

You need to modify rule1 to meet the following requirements:

Ensure that amount is always greater than 0.

Prevent an update to Table1 from being committed when data that violates rule1 is detected.

Which statement should you execute?

- A. `@dlt.expect_or_fail("rule1", "amount > 0")`
- B. `@dlt.expect("rule1", "amount > 0")`
- C. `@dlt.expect_all_or_drop("rule1": "amount > 0")`
- D. `@dlt.expect_or_drop("rule1", "amount > 0")`

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Why A is the Best Option:The correct answer is **A. `@dlt.expect_or_fail("rule1", "amount > 0")`** because it precisely meets both requirements set for modifying rule1:

Ensure that amount is always greater than 0: This is achieved by the condition "amount > 0".

Prevent an update to Table1 from being committed when data violating rule1 is detected: The `expect_or_fail` action ensures that if the expectation (in this case, amount > 0) is not met, the entire operation fails, thereby preventing the commit of invalid data to Table1.

Why Other Options are Less Suitable:

B. `@dlt.expect("rule1", "amount > 0")`:

Meets the first requirement (checks if amount > 0).

Fails the second requirement: `expect` only logs violations without preventing the commit of violating data to Table1.

C. `@dlt.expect_all_or_drop("rule1": "amount > 0")`:

Meets the first requirement (implicitly, by checking amount > 0).

Fails the second requirement in a less controlled manner: While it prevents commit by dropping rows that fail, it **does not fail the entire update** if violations are found; instead, it silently drops violating rows, which might not be the desired behavior for ensuring data integrity in this context.

D. `@dlt.expect_or_drop("rule1", "amount > 0")`:

Similar to C, meets the first requirement.

Fails the second requirement for the same reason as C: It drops violating rows without failing the update, which does not align with the requirement to prevent commitment of violating updates.

Correct Statement for the Requirement: Executing `@dlt.expect_or_fail("rule1", "amount > 0")` ensures that any update violating the amount > 0 rule fails entirely, protecting the integrity of Table1 by preventing the commitment of invalid data.

References

1. **Azure Databricks - Data Engineering with Lakehouse:** <https://docs.databricks.com/engineering-manual/lambda-architecture/lakehouse.html> Section on Lakeflow SDP and DLT Expectations provides context on expectation actions.
2. **Databricks Data Lineage & Expectations:** <https://docs.databricks.com/workflows/data-lineage/expectations.html> Detailed documentation on `expect_or_fail`, `expect`, `expect_all_or_drop`, and `expect_or_drop` actions.

Question: 76

Deep Dumps

You have a Lakeflow Spark Declarative Pipelines (SDP) pipeline in Azure Databricks. The pipeline ingests transaction data into a table named Table1.

You need to ensure that in the event of an invalid record, the pipeline continues to run. The solution must meet the following requirements:

Invalid records must NOT be written to Table1.

Invalid records must be preserved for review.

Minimize development effort.

What should you do?

- A. Add a CHECK constraint to Table1.
- B. Run WHERE clauses in downstream queries to filter out invalid records.
- C. Define a pipeline expectation.

D. Implement advanced logic to quarantine the invalid records.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

Why C (Define a pipeline expectation) is the best option:

Defining a pipeline expectation in Lakehouse Flow Spark Declarative Pipelines (SDP) is the most suitable solution for handling invalid records while meeting all the specified requirements. By setting up an expectation, you can define validation rules for your data (e.g., checking for required fields, data types, or specific value ranges). When an expectation fails (indicating an invalid record), the pipeline can be configured to:

Continue running without interruption, fulfilling the requirement for the pipeline to persist through invalid records.

Quarantine invalid records automatically, preserving them for review without writing them to Table1. This meets both the preservation and exclusion from Table1 requirements efficiently.

Minimize development effort by leveraging built-in SDP features designed for data validation and handling, reducing the need for custom, advanced logic development.

Why other options are less suitable:

A. Add a CHECK constraint to Table1:

Ineffective for existing invalid data if the table already contains invalid records.

Does not preserve invalid records for review as they would be rejected upon insertion attempt.

May halt the pipeline upon encountering an invalid record, depending on the insertion method's error handling.

B. Run WHERE clauses in downstream queries to filter out invalid records:

Fails to preserve invalid records for review since they are simply filtered out without storage.

Does not address the immediate ingestion issue, as invalid records would still be written to Table1 initially.

D. Implement advanced logic to quarantine invalid records:

Maximizes development effort, contradicting the requirement to minimize it.

Replicates functionality that can be more efficiently achieved through defining pipeline expectations in SDP.

Correct Action: C. Define a pipeline expectation to validate data, quarantine invalid records, and ensure the pipeline's continuity with minimal development overhead.

References

1. **Azure Databricks Documentation - Lakehouse Flow Pipelines:**
<https://docs.databricks.com/workflows/lakehouse-flow-pipelines.html>
2. **Azure Databricks Documentation - Data Validation with Expectations:**
<https://docs.databricks.com/workflows/lakehouse-flow-pipelines/validation.html>

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You have a Lakeflow Spark Declarative Pipelines (SDP) pipeline that writes records to a Delta table named Table1 by using a data quality rule named rule1.

You need to meet the following requirements:

Records that violate rule1 must NOT be written to Table1, but the pipeline must continue processing valid records.

Data engineers must be able to review expectation metrics by using minimal development effort.

What should you do? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Set action for rule1 to:

dlt.expect
dlt.expect_all
dlt.expect_or_drop
dlt.expect_or_fail

View expectation metrics in:

The pipeline event log
Azure Monitor metrics
System tables in Databricks
The Data quality tab of the pipeline

Answer:

Answer Area

Set action for rule1 to:

dlt.expect
dlt.expect_all
dlt.expect_or_drop
dlt.expect_or_fail

view expectation metrics in:

The pipeline event log
Azure Monitor metrics
System tables in Databricks
The Data quality tab of the pipeline

Explanation:

dlt.expect_or_drop: Use this if the goal is to quarantine or filter out bad data by discarding rows that violate the rule while letting the rest of the pipeline run.

The Data quality tab of the pipeline.

Databricks provides a dedicated, native **Data quality UI tab** directly inside the Delta Live Tables pipeline overview page. It serves as the primary visual space to monitor data quality trends, listing pass rates, drop percentages, and failure metrics graphically without needing to write custom queries.

Question: 78

Deep Dumps

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1. Job1 runs on a schedule and processes business-critical data. You need to ensure that you receive a notification if Job1 fails. What should you do?

- A. Add a SQL alert task to Job1.
- B. Create an Azure Monitor alert rule based on the metrics of Job1.
- C. Send the job run results of Job1 to Log Analytics.
- D. Configure a failure notification to be sent for the job runs of Job1.

Answer: D

Explanation:

Technical Justification for Correct Answer D

Why D is the best option:Configuring a failure notification to be sent for the job runs of Job1 (Option D) is the most direct and efficient approach to ensure notification upon job failure. This method leverages the native notification capabilities within Azure Databricks, specifically designed for job run statuses. By setting up failure notifications directly in the job configuration, you can specify recipients who will promptly receive alerts when Job1 fails, ensuring timely intervention for business-critical data processing.

Why other options are less suitable:

A. Add a SQL alert task to Job1: While adding a SQL alert task could theoretically notify of failures, it would require additional setup and dependency on a successful partial run to execute the alert task, which might not be reliable in all failure scenarios. Moreover, this approach mixes the concern of notification with the job's data processing logic, making the job more complex than necessary.

B. Create an Azure Monitor alert rule based on the metrics of Job1: This option, although viable for broader monitoring, involves more complexity in setup (requiring metrics configuration and threshold setting) and might introduce latency between the job failure and the notification, depending on how frequently the metric is evaluated. It's more suited for monitoring patterns over time rather than immediate job failure notifications.

C. Send the job run results of Job1 to Log Analytics: While logging job runs in Log Analytics is beneficial for auditing and post-failure analysis, it does not inherently provide a notification mechanism for immediate action upon failure. Additional setup would be required to alert on these logs, making this a less direct solution for the stated requirement.

References

For further details on configuring job notifications in Azure Databricks and Azure Monitor alert rules, refer to the following documentation:

1. **Azure Databricks Job Notifications:** [Microsoft Documentation - Configure job notifications](#)
2. **Azure Monitor Alert Rules:** [Microsoft Documentation - Create alert rules in Azure Monitor](#)

Question: 79

Deep Dumps

HOTSPOT -

You have a Declarative Automation Bundle.

You plan to use the Databricks CLI to deploy the bundle to an environment named dev.

How should you complete the CLI command? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

▼

databricks bundle

databricks workspace

az databricks workspace

▼

run dev

sync --target dev

deploy --target dev

Answer:

Answer Area

▼

databricks bundle

databricks workspace

az databricks workspace

▼

run dev

sync --target dev

deploy --target dev

Explanation:

databricks bundle.

The **databricks bundle** command group within the official Databricks CLI is specifically built to validate, deploy, and manage multi-resource software engineering bundles (DABs) as infrastructure-as-code.

deploy --target dev

The **deploy** subcommand uploads your project artifacts, orchestrates the asset configurations, and registers them in your workspace. Appending the **--target dev** flag directs the CLI to use the specific development environment properties defined inside your databricks.yml file.

Question: 80

Deep Dumps

You need to deploy Declarative Automation Bundles to a development environment. The solution must support automated and repeatable deployments across environments.

What should you use?

- A. the Databricks CLI
- B. Git folders

- C. the Azure Developer CLI (azd)
- D. the Azure Command-Line Interface (CLI)

Answer: A

Explanation:

Technical Justification for Correct Answer: A. the Databricks CLI

Why A (the Databricks CLI) is the Best Option:

Specificity to Databricks: The Databricks CLI is specifically designed for interacting with Azure Databricks. Deploying Declarative Automation Bundles (a Databricks feature for automating workspace configurations) is a task that directly benefits from the CLI's tailored functionality, ensuring precise control over the deployment process.

Automated and Repeatable Deployments: The Databricks CLI supports scripting, which is crucial for automating deployments. Scripts can be easily integrated into CI/CD pipelines, enabling repeatable deployments across different environments (dev, prod, etc.) with consistency and reliability.

Direct Support for Declarative Automation Bundles: The Databricks CLI has direct commands for working with Declarative Automation Bundles, streamlining the deployment process specifically for this type of asset.

Why Other Options are Less Suitable:

B. Git Folders:

Indirect Approach: While Git is excellent for version control and can trigger deployments via CI/CD pipelines, it doesn't directly deploy to Azure Databricks. An additional step or tool would be required for the actual deployment of Declarative Automation Bundles.

Lacks Direct Deployment Capability: Git itself doesn't deploy configurations to Azure services; it's more about versioning and triggering deployments.

C. the Azure Developer CLI (azd):

General-Purpose Azure Tool: The Azure Developer CLI (azd) is a general tool for Azure development, not specifically optimized for Databricks deployments. It might not offer the same level of direct support for Declarative Automation Bundles as the Databricks CLI.

Less Efficient for This Specific Task: Given its broader scope, using azd for a Databricks-specific task like deploying Declarative Automation Bundles could be less efficient and more prone to errors due to the lack of tailored functionality.

D. the Azure Command-Line Interface (CLI):

Too Broad in Scope: Similar to azd, the Azure CLI is a general-purpose tool for managing Azure resources. It lacks the specificity and direct commands for efficiently deploying Declarative Automation Bundles to Azure Databricks.

More Complex for Databricks Tasks: Achieving the desired deployment with the Azure CLI would likely involve more complex scripting and indirect methods, increasing the risk of errors.

Key Points in Bullet Form for Quick Reference:

Correct Choice (A. Databricks CLI):

- Specifically designed for Azure Databricks
- Supports automated, repeatable deployments via scripting
- Direct commands for Declarative Automation Bundles

Less Suitable Choices:

- B. Git Folders:** Indirect, lacks deployment capability
- C. Azure Developer CLI (azd):** Too general, less efficient
- D. Azure CLI:** Broad scope, complex for Databricks-specific tasks

References:

1. **Databricks CLI Documentation:** <https://docs.databricks.com/dev-tools/cli/index.html>
2. **Azure Databricks Declarative Automation Bundles:** <https://docs.databricks.com/administration-guide/cloud-ops/declarative-automation.html>

Question: 81

Deep Dumps

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1. Job1 contains multiple tasks.

Some tasks are business-critical, and others are not.

You need to configure alerting so that:

failures in non-critical tasks do not send notifications

notifications are sent only for failures in the critical parts of the workflow

What should trigger a notification?

- A. task success
- B. job success
- C. a task failure
- D. a job failure

Answer: C

Explanation:

Technical Justification for Correct Answer: C (a task failure)

To meet the specific alerting requirements for Job1 in the Azure Databricks workspace, particularly distinguishing between critical and non-critical tasks within the same job, the notification trigger must be granular enough to identify failures at the task level. Here's why **C (a task failure)** is the best option and why the others are less suitable:

Why C (a task failure) is correct:

Granularity: Triggering notifications on task failures allows for the distinction between critical and non-critical tasks based on their individual success or failure statuses.

Requirement Fulfillment: This option directly fulfills the requirement that notifications are sent only for failures in the critical parts of the workflow, assuming the critical tasks can be identified and alerted upon individually.

Flexibility in Configuration: Azure Databricks enables configuration of alerts on task-level outcomes, making it feasible to set up notifications for specific tasks (i.e., the critical ones) without affecting the others.

Why other options are less suitable:

A (task success):

Irrelevant to Failure Notification: Triggering on task success does not address the requirement of notifying on failures.

Unnecessary Notifications: Would send notifications for successful tasks, both critical and non-critical, which

is not required.

B (job success):

Lack of Granularity: Similar to A, it doesn't address failure notifications and ignores the distinction between task types within the job.

D (a job failure):

Overly Broad: A job failure notification would trigger regardless of which task failed (critical or not), leading to unnecessary notifications for non-critical task failures.

Insufficient Granularity for Requirements: Fails to meet the specific need of distinguishing between critical and non-critical task failures.

Conclusion: Given the need for granular control over notifications based on the criticality of tasks within Job1, triggering notifications **on a task failure (C)** is the most appropriate choice. This approach allows for targeted alert configuration for critical tasks while ignoring non-critical ones, aligning perfectly with the specified requirements.

References

1. **Azure Databricks Documentation - Jobs:** <https://docs.databricks.com/jobs/index.html> (Explore sections on Job Configuration and Alerts for detailed setup information)
2. **Microsoft Learn - Configure notifications for Azure Databricks jobs:** <https://learn.microsoft.com/en-us/azure/databricks/manage-jobs#configure-notifications> (Provides step-by-step guidance on setting up notifications based on job/task outcomes)

Question: 82

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You need to recommend a pipeline that ingests files from cloud storage, performs cleansing and enrichment transformations, and writes curated Delta tables for analytics. The solution must minimize development effort and provide built-in monitoring and automatic retries.

What should you include in the recommendation?

- A. a Databricks notebook triggered by a scheduled job
- B. an Apache Spark Structured Streaming job
- C. an MLflow pipeline
- D. a Lakeflow Spark Declarative Pipelines (SDP) pipeline

Answer: D

Explanation:

Technical Justification for Recommendation

Recommended Option: D. a Lakeflow Spark Declarative Pipelines (SDP) pipeline

Minimizes Development Effort: Lakeflow SDP provides a declarative configuration approach, allowing for concise pipeline definitions with less code, thus reducing development effort.

Built-in Monitoring: Lakeflow offers integrated monitoring capabilities, providing visibility into pipeline execution without requiring additional setup.

Automatic Retries: SDP includes features for retry mechanisms, ensuring robustness against transient failures without manual implementation.

Why Other Options are Less Suitable:

A. Databricks Notebook triggered by a Scheduled Job

Development Effort: Equivalent to writing custom code for transformations and handling retries, increasing effort.

Monitoring and Retries: Requires manual implementation of monitoring and retry logic.

B. Apache Spark Structured Streaming Job

Use Case Mismatch: Best suited for real-time streaming data processing, not necessarily optimized for batch file ingestion and curated Delta table writing.

Development Effort and Monitoring: Similar to A, requires custom code for monitoring and retries.

C. MLflow Pipeline

Primary Focus: Designed for Machine Learning workflow management, not generalized data engineering pipelines for data ingestion and transformation.

Monitoring and Retries: While MLflow provides some monitoring, it's more focused on ML experiments, and retries would need custom handling.

References

[Lakehouse Pipelines with Lakeflow | Azure Databricks](#)

[Unity Catalog Overview | Azure Databricks](#)

Question: 83

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a managed Delta table named Table1. Table1 stores customer data.

You need to implement a data lifecycle solution that meets the following requirements:

Deleted data must be recoverable for 30 days to support audits.

Data that is older than 30 days must be removed permanently.

The solution must minimize administrative effort.

The solution must be enforced consistently.

What should you configure?

- A. scheduled DELETE operations
- B. the data lineage
- C. Azure Storage lifecycle management
- D. data retention for Table1

Answer: D

Explanation:

Technical Justification

The correct option to implement the required data lifecycle solution for Table1 in Azure Databricks with Unity Catalog enabled is **D. data retention for Table1**. Here's why this option stands out and why the others are less suitable:

Why D (data retention for Table1) is the best choice:

Meets Recovery Requirement: Configuring data retention for Table1 directly addresses the need for deleted data to be recoverable for 30 days, leveraging Delta Lake's versioned storage capability.

Permanent Deletion: After the specified retention period (30 days), data older than this threshold is automatically and permanently removed, fulfilling the second requirement.

Minimizes Administrative Effort: This is a managed, automated process once configured, reducing the need for manual intervention or scheduling additional operations.

Consistent Enforcement: Being a property of the table itself within Azure Databricks, the retention policy is consistently applied across all interactions with Table1, ensuring audits and compliance are supported uniformly.

Why Other Options are Less Suitable:

A. Scheduled DELETE operations:

Higher Administrative Effort: Requires setup and management of additional scheduled jobs.

Recovery Complexity: Deleted data would not be recoverable through simple versioning; a more complex backup/restore process would be needed.

Inconsistency Risk: Manual or scheduled deletions might not consistently enforce the 30-day rule across all access paths.

B. Data Lineage:

Does Not Address Requirements Directly: Data lineage tracks data provenance and lineage, not retention or deletion policies.

No Automation for Deletion/Retention: Would require separate configuration for actual data management.

C. Azure Storage Lifecycle Management:

Scope Mismatch: Applies to storage at a broader level, not specifically tailored to the managed Delta table's needs within Databricks.

Less Direct Control for Table-Specific Policies: Managing at the storage level might not offer the same granularity for Table1's specific retention requirements.

References

For further reading on the recommended approach and understanding the capabilities of Azure Databricks and Unity Catalog:

1. **Azure Databricks Documentation - Delta Tables & Data Retention:** <https://docs.databricks.com/delta-lake/user-guide/data-lifecycle.html>
2. **Microsoft Docs - Azure Databricks Unity Catalog Overview:** <https://docs.microsoft.com/en-us/azure/databricks/security/access-control/unity-catalog-overview>

Question: 84

Deep Dumps

A data engineer receives periodic flat files in cloud storage and wants to load them into an existing Delta table using a SQL-based command. The workload is batch-oriented and does not require continuous streaming. Which command is most appropriate?

- A. COPY INTO
- B. MERGE INTO
- C. CREATE VIEW
- D. GRANT SELECT

Answer: A

Explanation:

A. COPY INTO Correct

COPY INTO is a SQL command in Databricks that efficiently loads batch data from files in cloud storage into an existing Delta table. It is ideal for periodic file ingestion and keeps track of previously loaded files to avoid duplicate loads.

B. MERGE INTO Incorrect

MERGE INTO is used to perform upserts (insert, update, and delete operations) between a source and a target table based on matching conditions. It is not primarily designed for loading flat files directly into a Delta table.

C. CREATE VIEW Incorrect

CREATE VIEW creates a virtual table based on a SQL query. It does not load or ingest data into a Delta table.

D. GRANT SELECT Incorrect

GRANT SELECT is a security command used to provide read permissions on a table or view. It has no role in data ingestion.

Final Answer: A. COPY INTO

Question: 85

Deep Dumps

A fact table is often filtered by customer ID, but customer ID has millions of distinct values. The team is considering partitioning by customer ID to speed up queries. What should the team do?

- A. Partition by customer ID because high cardinality always improves pruning
- B. Disable all data skipping features
- C. Convert the table to CSV before partitioning
- D. Avoid high-cardinality partitioning and consider clustering or another layout strategy

Answer: D

Explanation:

A. Partition by customer ID because high cardinality always improves pruning Incorrect

Partitioning on a column with millions of distinct values creates an excessive number of small partitions, leading to metadata overhead, slower performance, and inefficient file management.

B. Disable all data skipping features Incorrect

Data skipping features help optimize query performance by reducing the amount of data scanned. Disabling them would generally make queries less efficient.

C. Convert the table to CSV before partitioning Incorrect

Converting data to CSV provides no benefit for partitioning strategy and would eliminate many performance advantages of Delta Lake.

D. Avoid high-cardinality partitioning and consider clustering or another layout strategy Correct

For columns with very high cardinality, such as customer IDs with millions of unique values, partitioning is not recommended. Instead, use techniques such as clustering, Z-Ordering, liquid clustering, or other data layout

optimizations to improve query performance while avoiding excessive partitions.

Final Answer: D. Avoid high-cardinality partitioning and consider clustering or another layout strategy

Question: 86

Deep Dumps

A streaming pipeline reads events from Azure Event Hubs and writes them to a Delta table. After a compute failure, the pipeline must resume from the last committed progress point.

What should be configured?

- A. A table comment
- B. A Delta Sharing share
- C. A checkpoint location
- D. A cluster tag

Answer: C

Explanation:

A. A table comment Incorrect

A table comment is used to document or describe a table. It does not store streaming progress or recovery information.

B. A Delta Sharing share Incorrect

Delta Sharing is used to securely share data across organizations and platforms. It has no role in tracking streaming progress or fault recovery.

C. A checkpoint location Correct

A checkpoint location stores the streaming query's progress information, offsets, and state data. If a failure occurs, the streaming job can restart from the last successfully committed checkpoint instead of reprocessing all data.

D. A cluster tag Incorrect

Cluster tags are used for resource organization, monitoring, and cost tracking. They do not provide streaming recovery capabilities.

Question: 87

Deep Dumps

A scheduled Databricks workload must authenticate as a non-human identity to perform production data processing.

Which identity should be used?

- A. Local administrator
- B. Shared analyst account
- C. Personal user account
- D. Service principal

Answer: D

Explanation:

A. Local administrator Incorrect

A local administrator account is intended for system administration tasks and is not recommended for automated production workloads.

B. Shared analyst account Incorrect

Shared accounts reduce accountability, complicate auditing, and are considered a security risk for production environments.

C. Personal user account Incorrect

Personal user accounts are tied to individual users and should not be used for automated or production workloads because access may change or be revoked.

D. Service principal Correct

A service principal is a non-human identity designed for applications, automation, and scheduled workloads. It provides secure authentication, supports least-privilege access, and enables proper auditing and lifecycle management.

Question: 88

Deep Dumps

A Delta table receives frequent streaming writes that create many small files. Query performance is degrading because queries scan too many files. Which operation should be scheduled?

- A. DESCRIBE HISTORY
- B. CREATE RECIPIENT
- C. VACUUM only
- D. OPTIMIZE

Answer: D

Explanation:

A. DESCRIBE HISTORY Incorrect

DESCRIBE HISTORY displays the transaction history of a Delta table. It is useful for auditing and troubleshooting but does not improve query performance or reduce the number of files.

B. CREATE RECIPIENT Incorrect

CREATE RECIPIENT is used in Delta Sharing to define recipients of shared data. It has no impact on file management or query performance.

C. VACUUM only Incorrect

VACUUM removes old, unreferenced files from a Delta table to reclaim storage space. While useful for maintenance, it does not compact small active files that are causing query performance issues.

D. OPTIMIZE Correct

OPTIMIZE compacts many small files into fewer larger files, reducing file-scanning overhead and improving query performance. It is commonly scheduled for Delta tables that receive frequent streaming writes.

Question: 89

Deep Dumps

A streaming aggregation groups events into 10-minute windows. Events can arrive up to 30 minutes late. The pipeline must accept late data within that delay while preventing unbounded state growth. Which configuration should be used?

- A. Watermarking
- B. Z-ordering
- C. VACUUM
- D. Column masking

Answer: A

Explanation:

A. Watermarking Correct

Watermarking allows a streaming query to handle late-arriving data by specifying how long the engine should wait for delayed events. Events arriving within the watermark threshold (e.g., 30 minutes) are processed, while older events can be discarded. This helps prevent unbounded state growth and manages memory efficiently.

B. Z-ordering Incorrect

Z-ordering is a Delta Lake optimization technique used to improve query performance by colocating related data in files. It is unrelated to handling late-arriving streaming events.

C. VACUUM Incorrect

VACUUM removes obsolete files from Delta tables to reclaim storage space. It does not manage streaming state or late-arriving data.

D. Column masking Incorrect

Column masking is a data security feature used to hide sensitive information from certain users. It has no role in streaming aggregations or state management.

Question: 90

Deep Dumps

A data engineering job requires a specific Python wheel package. The package should be available only to that job and should not affect other workloads in the workspace. Where should the package be configured?

- A. In the Unity Catalog metastore
- B. In an Azure Key Vault secret
- C. As a job or compute library
- D. As a table property

Answer: C

Explanation:**A. In the Unity Catalog metastore Incorrect**

Unity Catalog manages data governance, access control, and metadata for data assets. It is not used to install or manage Python wheel packages for jobs.

B. In an Azure Key Vault secret Incorrect

Azure Key Vault stores secrets, credentials, certificates, and keys. It does not provide Python package installation or dependency management.

C. As a job or compute library Correct

Installing the wheel as a job library or compute library makes the package available only to the specific job or cluster where it is needed. This isolates dependencies and prevents impact on other workloads in the workspace.

D. As a table property Incorrect

Table properties store metadata and configuration settings for tables. They cannot be used to install or manage Python packages.

Question: 91**Deep Dumps**

A group of business analysts needs to query governed Unity Catalog tables by using SQL. They do not need to run notebooks or manage Spark cluster settings. Which compute option is most appropriate?

- A. SQL warehouse
- B. All-purpose compute
- C. Single-user cluster
- D. Cluster pool

Answer: A**Explanation:****A. SQL warehouse Correct**

SQL warehouses are designed specifically for running SQL queries, dashboards, and BI workloads against data in Unity Catalog. They provide a managed SQL experience without requiring users to manage clusters or run notebooks.

B. All-purpose compute Incorrect

All-purpose compute is intended for interactive workloads such as notebooks, data exploration, and development. It provides more functionality than business analysts require and requires cluster management.

C. Single-user cluster Incorrect

A single-user cluster is dedicated to one user and is typically used for development or specialized workloads, not for shared SQL analytics by business users.

D. Cluster pool Incorrect

A cluster pool is a resource optimization feature that reduces cluster startup times. It is not a compute option for directly querying data.

Question: 92

Deep Dumps

A source JSON field named amount sometimes arrives as an integer and sometimes as a string. The silver table requires a decimal type for accurate calculations.

What should the transformation logic do?

- A. Store the value only in a table comment
- B. Keep the value as a string permanently
- C. Cast and validate the field during transformation
- D. Disable schema enforcement

Answer: C

Explanation:

A. Store the value only in a table comment Incorrect

Table comments are used for documentation and metadata. They do not transform or validate data types.

B. Keep the value as a string permanently Incorrect

Storing numeric values as strings makes calculations, aggregations, and comparisons less efficient and more error-prone.

C. Cast and validate the field during transformation Correct

The transformation process should convert the incoming value to a decimal type and validate that the data is in a valid numeric format. This ensures consistent schema enforcement and accurate calculations in the silver table.

D. Disable schema enforcement Incorrect

Schema enforcement helps maintain data quality and consistency. Disabling it could allow invalid or inconsistent data types into the table.

Question: 93

Deep Dumps

A data lake already contains Delta files in an ADLS Gen2 path. The company wants to register the data in Unity Catalog while keeping the data in the existing storage location.

Which table type should be created?

- A. Managed table
- B. Temporary view
- C. Materialized view
- D. External table

Answer: D

Explanation:

A. Managed table Incorrect

Managed tables store and manage data within the storage location controlled by Unity Catalog. They are not intended for registering data that already exists in an external ADLS Gen2 location.

B. Temporary view Incorrect

A temporary view provides a session-scoped logical view of data and does not register or manage existing Delta files as a cataloged table.

C. Materialized view Incorrect

A materialized view stores the results of a query for performance optimization. It is not used to register existing Delta files in external storage.

D. External table Correct

An external table allows Unity Catalog to register and govern data that already exists in an external storage location such as ADLS Gen2. The data remains in its current location while Unity Catalog manages access, metadata, and governance.

Question: 94

Deep Dumps

A batch process updates a Delta table incorrectly. The team needs to identify which operation introduced the bad data before deciding how to recover.

Which command or capability should be used first?

- A. OPTIMIZE
- B. DESCRIBE HISTORY
- C. COPY INTO
- D. CREATE CATALOG

Answer: B

Explanation:

A. OPTIMIZE Incorrect

OPTIMIZE compacts small files to improve query performance. It does not provide information about which operation introduced incorrect data.

B. DESCRIBE HISTORY Correct

DESCRIBE HISTORY displays the transaction history of a Delta table, including operation types, timestamps, users, and other metadata. This helps identify the specific operation that introduced the bad data before deciding on a recovery method such as time travel or restore.

C. COPY INTO Incorrect

COPY INTO is used to load data from files into a table. It does not help investigate historical table changes.

D. CREATE CATALOG Incorrect

CREATE CATALOG creates a new catalog in Unity Catalog and is unrelated to troubleshooting data quality issues.

Question: 95**Deep Dumps**

After identifying a bad write to a Delta table, the team wants to revert the table to a previous known-good version. Which operation should be used?

- A. VACUUM
- B. CREATE RECIPIENT
- C. ANALYZE TABLE only
- D. RESTORE

Answer: D**Explanation:****A. VACUUM Incorrect**

VACUUM removes obsolete files that are no longer referenced by a Delta table. It does not revert data to a previous version.

B. CREATE RECIPIENT Incorrect

CREATE RECIPIENT is used in Delta Sharing to define data-sharing recipients. It has no role in data recovery.

C. ANALYZE TABLE only Incorrect

ANALYZE TABLE collects statistics to improve query optimization. It does not restore or recover previous table versions.

D. RESTORE Correct

RESTORE allows a Delta table to be reverted to a previous version or timestamp using Delta Lake's transaction log. This is the appropriate operation after identifying a bad write and selecting a known-good version from the table history.

Question: 96**Deep Dumps**

A company wants to publish curated business metrics for executive dashboards. The data should be cleaned, aggregated, and optimized for consumption. Which medallion layer is most appropriate?

- A. Bronze
- B. Gold
- C. Landing
- D. Raw

Answer: B**Explanation:****A. Bronze Incorrect**

The Bronze layer contains raw, ingested data with minimal or no transformations. It is not intended for direct business consumption or executive reporting.

B. Gold Correct

The Gold layer contains curated, business-ready data that has been cleaned, transformed, and often aggregated. It is optimized for reporting, dashboards, analytics, and executive decision-making.

C. Landing Incorrect

The Landing layer (or landing zone) is typically a temporary storage area where data arrives before processing. It is not part of the final curated analytics layer.

D. Raw Incorrect

Raw data is unprocessed source data and is not suitable for executive dashboards that require trusted and aggregated business metrics.

Question: 97

Deep Dumps

A Lakeflow Spark Declarative Pipeline loads order data into a curated table. If order_id is null, the record must not be allowed into the target table, and the failed quality condition must be visible to the pipeline operator. Which feature should be used?

- A. Secret scope
- B. Pipeline expectation
- C. Cluster pool
- D. Delta Sharing provider

Answer: B

Explanation:

A. Secret scope Incorrect

Secret scopes are used to securely store and access credentials, API keys, and other sensitive information. They do not perform data quality validation.

B. Pipeline expectation Correct

Pipeline expectations allow you to define data quality rules (for example, order_id IS NOT NULL). Records that fail the expectation can be dropped, quarantined, or cause the pipeline to fail, and the results are visible to pipeline operators through monitoring and quality metrics.

C. Cluster pool Incorrect

A cluster pool helps reduce cluster startup times and improve resource utilization. It does not enforce data quality rules.

D. Delta Sharing provider Incorrect

A Delta Sharing provider is used to share data with external recipients. It is unrelated to data validation within a pipeline.

Question: 98**Deep Dumps**

A data engineer wants to create a new Delta table from the result of a SELECT query in Databricks SQL. Which statement should be used?

- A. CREATE TABLE AS SELECT
- B. VACUUM
- C. RESTORE TABLE
- D. GRANT SELECT

Answer: A**Explanation:****A. CREATE TABLE AS SELECT Correct**

CREATE TABLE AS SELECT (CTAS) creates a new table and populates it with the results of a SELECT query in a single operation. It is commonly used to create Delta tables from transformed or filtered data.

B. VACUUM Incorrect

VACUUM removes obsolete files from a Delta table to reclaim storage space. It does not create new tables.

C. RESTORE TABLE Incorrect

RESTORE TABLE reverts a Delta table to a previous version or timestamp. It is used for recovery, not table creation.

D. GRANT SELECT Incorrect

GRANT SELECT is a permission statement used to provide read access to a table. It does not create tables.

Question: 99**Deep Dumps**

A data engineer maintains a derived reporting table used by dashboards. The table must be rebuilt from a SELECT query during deployment, but downstream users should continue referencing the same table name. Which statement is most appropriate?

- A. CREATE OR REPLACE TABLE AS SELECT
- B. MERGE INTO without a matching key
- C. VACUUM the reporting table
- D. GRANT SELECT on the source table only

Answer: A**Explanation:****A. CREATE OR REPLACE TABLE AS SELECT Correct**

CREATE OR REPLACE TABLE AS SELECT (CRTAS) recreates the table using the results of a SELECT query while keeping the same table name. This allows downstream dashboards and users to continue referencing the existing table name without any changes.

B. MERGE INTO without a matching key Incorrect

MERGE INTO is intended for upsert operations based on matching conditions. Without a proper matching key, it is not an appropriate method for rebuilding an entire reporting table.

C. VACUUM the reporting table Incorrect

VACUUM removes obsolete files from a Delta table to reclaim storage space. It does not rebuild or refresh table contents.

D. GRANT SELECT on the source table only Incorrect

Granting permissions affects access control but does not create, replace, or rebuild a reporting table.

Question: 100

Deep Dumps

A pipeline reads CSV files from cloud storage. Some rows contain unexpected extra columns or malformed values. The engineering team wants to retain problematic data for later review instead of silently losing it. Which capability is most relevant?

- A. Rescued data handling
- B. Delta Sharing
- C. SQL warehouse caching
- D. Cluster policy inheritance

Answer: A

Explanation:

A. Rescued data handling Correct

Rescued data handling captures unexpected columns, schema mismatches, and malformed values in a special rescued data column (commonly `_rescued_data`) instead of discarding them. This allows engineers to review and troubleshoot problematic records later while preserving the original data.

B. Delta Sharing Incorrect

Delta Sharing is used to securely share data across organizations and platforms. It does not handle malformed records or schema issues during ingestion.

C. SQL warehouse caching Incorrect

SQL warehouse caching improves query performance by caching data and results. It has no role in retaining problematic ingestion records.

D. Cluster policy inheritance Incorrect

Cluster policies govern cluster configuration and permissions. They do not manage malformed CSV data or schema rescue behavior.

Question: 101

Deep Dumps

A Delta table has obsolete files from previous versions. The team wants to reduce storage cost but must preserve the required time travel retention period. Which operation should be used carefully?

- A. VACUUM
- B. RESTORE
- C. MERGE
- D. CLONE

Answer: A

Explanation:

A. VACUUM Correct

VACUUM removes obsolete data files that are no longer referenced by the Delta table. While this helps reduce storage costs, it must be used carefully because removing files before the required retention period can limit or eliminate the ability to use Delta Lake time travel for older versions.

B. RESTORE Incorrect

RESTORE reverts a Delta table to a previous version. It relies on historical files being available and does not remove obsolete files or reduce storage costs.

C. MERGE Incorrect

MERGE is used for upserts (insert, update, and delete operations) between tables. It does not manage historical files or retention periods.

D. CLONE Incorrect

CLONE creates a copy of a Delta table (shallow or deep clone). It does not remove obsolete files or directly reduce storage costs.

Question: 102

Deep Dumps

A governance team wants to apply access policies based on tags such as data classification, business unit, and sensitivity level.

Which Unity Catalog governance approach best matches this requirement?

- A. Attribute-based access control
- B. Auto Loader schema inference
- C. Photon query acceleration
- D. Delta table vacuuming

Answer: A

Explanation:

A. Attribute-based access control Correct

Attribute-Based Access Control (ABAC) allows access decisions to be made based on attributes or tags associated with data, users, or resources. Tags such as data classification, business unit, and sensitivity level can be used to define and enforce governance policies centrally in Unity Catalog.

B. Auto Loader schema inference Incorrect

Auto Loader schema inference automatically detects and evolves schemas during data ingestion. It is

unrelated to access control and governance policies.

C. Photon query acceleration Incorrect

Photon is Databricks' high-performance query engine that improves execution speed. It does not provide governance or access-control capabilities.

D. Delta table vacuuming Incorrect

VACUUM removes obsolete files from Delta tables to reclaim storage space. It has no role in applying access policies.

Question: 103

Deep Dumps

A data engineer needs to rebuild a derived table from a SELECT query while preserving the object name used by downstream queries.

Which statement is most appropriate?

- A. GRANT SELECT ON TABLE
- B. DESCRIBE HISTORY
- C. VACUUM
- D. CREATE OR REPLACE TABLE AS SELECT

Answer: D

Explanation:

A. GRANT SELECT ON TABLE Incorrect

GRANT SELECT ON TABLE is used to provide read permissions to users or groups. It does not rebuild or recreate a table.

B. DESCRIBE HISTORY Incorrect

DESCRIBE HISTORY displays the transaction history of a Delta table. It is useful for auditing and troubleshooting but does not rebuild a table.

C. VACUUM Incorrect

VACUUM removes obsolete files from a Delta table to reclaim storage space. It does not recreate or refresh table contents.

D. CREATE OR REPLACE TABLE AS SELECT Correct

CREATE OR REPLACE TABLE AS SELECT (CRTAS) recreates the table using the results of a SELECT query while keeping the same table name. This ensures that downstream queries continue to reference the same object name without modification.

Question: 104

Deep Dumps

A company has an Azure Databricks job that runs every hour to process landing-zone files and write curated Delta tables.

Each run should use fresh isolated compute and terminate when the workload finishes.

Which compute option should be used?

- A. All-purpose compute
- B. SQL warehouse
- C. Job compute
- D. Interactive shared cluster

Answer: C

Explanation:

A. All-purpose compute Incorrect

All-purpose compute is designed for interactive development, notebooks, and ad hoc analysis. It is typically long-running and shared among users, making it less suitable for scheduled production jobs.

B. SQL warehouse Incorrect

SQL warehouses are optimized for SQL analytics, dashboards, and BI workloads. They are not intended for running ETL or data engineering jobs.

C. Job compute Correct

Job compute creates a dedicated cluster for each job run, providing fresh and isolated resources. The cluster automatically terminates when the workload completes, making it the recommended and cost-effective option for scheduled production workloads.

D. Interactive shared cluster Incorrect

Interactive shared clusters are intended for collaborative development and exploratory workloads. They do not provide the same isolation and automatic lifecycle management as job compute.

Question: 105

Deep Dumps

A team is building a bronze-silver-gold architecture. The bronze layer must support replay, auditing, and troubleshooting of source data issues. Which design is most appropriate?

- A. Store only final aggregated metrics
- B. Store only masked reporting tables
- C. Store raw or minimally transformed source data
- D. Store only materialized views

Answer: C

Explanation:

A. Store only final aggregated metrics Incorrect

Final aggregated metrics belong in the Gold layer, which is optimized for reporting and business consumption, not for replay or troubleshooting.

B. Store only masked reporting tables Incorrect

Masked reporting tables are curated outputs intended for end users and governance requirements, not for

retaining original source data.

C. Store raw or minimally transformed source data Correct

The Bronze layer is designed to retain raw or minimally transformed source data. This enables data replay, auditing, lineage tracking, debugging, and recovery from downstream processing issues.

D. Store only materialized views Incorrect

Materialized views are derived data structures used for performance optimization and reporting, not for preserving source data history.

Question: 106

Deep Dumps

A large Delta table is queried by different teams using changing filter patterns. The team wants to improve data skipping and layout without maintaining a rigid static partition design. Which feature should be considered?

- A. Query result cache only
- B. Random repartitioning on every write
- C. Liquid clustering
- D. Delta Sharing

Answer: C

Explanation:

A. Query result cache only Incorrect

Query result caching can improve performance for repeated identical queries, but it does not optimize the underlying data layout or improve data skipping for changing query patterns.

B. Random repartitioning on every write Incorrect

Random repartitioning does not organize data in a way that improves filtering or data skipping and can introduce unnecessary processing overhead.

C. Liquid clustering Correct

Liquid clustering provides a flexible data layout strategy that improves data skipping and query performance without requiring a fixed partition scheme. It adapts more easily to changing access patterns and avoids many of the limitations of static partitioning.

D. Delta Sharing Incorrect

Delta Sharing is used for securely sharing data across organizations and platforms. It does not optimize table layout or query performance.

Question: 107

Deep Dumps

A company wants to share several governed tables with an external organization through Delta Sharing. Which two objects are most relevant? Choose two.

- A. Share

- B. Cluster pool
- C. Secret scope
- D. Recipient

Answer: A,D

Explanation:

A. Share Correct

A **Share** is a Delta Sharing object that contains the tables, views, or other data assets that will be made available to external consumers.

B. Cluster pool Incorrect

A cluster pool is used to reduce cluster startup times and optimize compute resource usage. It is unrelated to Delta Sharing.

C. Secret scope Incorrect

Secret scopes are used to securely store credentials and sensitive information. They are not required to define or manage Delta Sharing recipients.

D. Recipient Correct

A **Recipient** represents the external organization, user, or system that will receive access to the shared data. Shares are granted to recipients.

Final Answer: A. Share and D. Recipient

Question: 108

Deep Dumps

An administrator wants to prevent users from creating clusters with unsupported runtimes, oversized node types, or missing auto-termination settings.

What should the administrator configure?

- A. Table ACLs
- B. Delta Sharing recipients
- C. External locations
- D. Compute policies

Answer: D

Explanation:

A. Table ACLs Incorrect

Table ACLs control access permissions to tables and views. They do not govern cluster creation or configuration settings.

B. Delta Sharing recipients Incorrect

Delta Sharing recipients are used to grant external organizations access to shared data. They are unrelated to cluster configuration.

C. External locations Incorrect

External locations define and secure access to cloud storage locations in Unity Catalog. They do not enforce cluster settings.

D. Compute policies Correct

Compute policies (formerly cluster policies) allow administrators to enforce rules and restrictions on cluster configurations, such as approved Databricks runtimes, allowed node types, auto-termination requirements, and other governance controls.

Question: 109

Deep Dumps

A workflow must copy files between Azure services, call an external API, and then trigger Azure Databricks processing. The team wants one orchestration layer for the cross-service process. Which tool is most appropriate?

- A. Delta table history
- B. Column mask
- C. Azure Data Factory
- D. Query profile

Answer: C

Explanation:

A. Delta table history Incorrect

Delta table history is used to view the transaction history of a Delta table for auditing and troubleshooting. It does not orchestrate workflows across services.

B. Column mask Incorrect

Column masking is a data governance feature used to protect sensitive data. It is unrelated to workflow orchestration.

C. Azure Data Factory Correct

Azure Data Factory (ADF) is a cloud-based data integration and orchestration service. It can copy data between Azure services, invoke external APIs, schedule and monitor workflows, and trigger Azure Databricks jobs as part of an end-to-end pipeline.

D. Query profile Incorrect

A query profile provides execution details and performance metrics for SQL queries. It is not an orchestration tool.

Question: 110

Deep Dumps

A production job must access ADLS Gen2 data governed by Unity Catalog. The security team does not want storage account keys or secrets stored in notebooks. Which approach is most appropriate?

- A. Use a managed identity or service principal through a Unity Catalog storage credential

- B. Store the storage key in a notebook variable
- C. Grant users direct access to the storage account and bypass Unity Catalog
- D. Save the connection string in a Delta table comment

Answer: A

Explanation:

A. Use a managed identity or service principal through a Unity Catalog storage credential Correct

Unity Catalog storage credentials allow secure access to ADLS Gen2 using a managed identity or service principal. This avoids storing storage account keys or secrets in notebooks while maintaining centralized governance, auditing, and access control.

B. Store the storage key in a notebook variable Incorrect

Storing storage keys in notebook code is a security risk because credentials can be exposed to users, source control systems, or logs.

C. Grant users direct access to the storage account and bypass Unity Catalog Incorrect

Bypassing Unity Catalog removes centralized governance, auditing, and fine-grained access control, which goes against security best practices.

D. Save the connection string in a Delta table comment Incorrect

Table comments are intended for documentation and metadata. Storing secrets or connection strings in comments is insecure and not an approved authentication method.

Question: 111

Deep Dumps

A CSV source stores transaction_date as a string. Downstream users must filter by date ranges and calculate aging accurately.

What should the transformation logic do?

- A. Store the date only in a comment
- B. Keep the value as a string to avoid schema enforcement
- C. Move the value to a secret scope
- D. Cast the value to an appropriate date type

Answer: D

Explanation:

A. Store the date only in a comment Incorrect

Comments are used for documentation and metadata. They do not enable filtering, calculations, or data type enforcement.

B. Keep the value as a string to avoid schema enforcement Incorrect

Storing dates as strings makes date-range filtering, sorting, and aging calculations less efficient and more error-prone.

C. Move the value to a secret scope Incorrect

Secret scopes are used to securely store credentials and sensitive information, not business data such as transaction dates.

D. Cast the value to an appropriate date type Correct

The transformation process should convert the string value to a proper DATE (or TIMESTAMP if needed) data type. This enables efficient date filtering, accurate aging calculations, schema consistency, and better query optimization.

Question: 112

Deep Dumps

A sales table contains records for multiple regions. Sales managers should only see rows for the regions they are authorized to access.
Which Unity Catalog feature should be applied?

- A. Column mask
- B. Materialized view
- C. Storage credential
- D. Row filter

Answer: D

Explanation:

A. Column mask Incorrect

Column masking hides or obfuscates sensitive values within columns, but it does not restrict access to specific rows of data.

B. Materialized view Incorrect

A materialized view stores the results of a query for performance optimization. It is not a security feature for restricting row-level access.

C. Storage credential Incorrect

Storage credentials are used to securely access cloud storage locations. They do not control which rows users can see in a table.

D. Row filter Correct

Row filters provide row-level security in Unity Catalog by restricting which records a user can view based on defined conditions, such as region, department, or user attributes. This ensures sales managers only see data for the regions they are authorized to access.

Question: 113

Deep Dumps

A customer dimension must preserve every status change. Analysts need to query both the current status and the status that was valid on a previous date.
Which design should be implemented?

- A. SCD Type 2 with effective date ranges

- B. SCD Type 1 overwrite
- C. Query result caching
- D. Append-only raw storage only

Answer: A

Explanation:

A. SCD Type 2 with effective date ranges Correct

Slowly Changing Dimension (SCD) Type 2 preserves the full history of changes by creating a new record whenever an attribute changes. Effective start and end dates (or current-record indicators) allow analysts to query both the current status and the status that was valid at any point in time.

B. SCD Type 1 overwrite Incorrect

SCD Type 1 overwrites existing values and does not retain historical information. Previous statuses would be lost.

C. Query result caching Incorrect

Query result caching improves performance for repeated queries but does not preserve historical dimension data.

D. Append-only raw storage only Incorrect

Raw append-only storage may retain historical records, but it does not provide the structured dimension management, effective dates, and querying capabilities required for historical analysis.

Question: 114

Deep Dumps

A company uses one Unity Catalog metastore across multiple workspaces. It needs clear separation between development, test, and production data while preserving centralized governance. Which two design choices are appropriate? Choose two.

- A. Create separate catalogs for dev, test, and prod
- B. Store all environments in one schema and rely on table names
- C. Use group-based grants for each environment
- D. Use notebook folders as the main governance boundary

Answer: A,C

Explanation:

A. Create separate catalogs for dev, test, and prod Correct

Using separate catalogs for development, test, and production provides strong logical isolation between environments while maintaining centralized governance through a single Unity Catalog metastore.

B. Store all environments in one schema and rely on table names Incorrect

Relying on naming conventions alone does not provide strong isolation and increases the risk of accidental access or modification across environments.

C. Use group-based grants for each environment Correct

Group-based permissions allow administrators to control access to each environment according to user roles and responsibilities, supporting least-privilege access and centralized governance.

D. Use notebook folders as the main governance boundary Incorrect

Notebook folders organize workspace content but are not a governance or security boundary for data access in Unity Catalog.

Question: 115

Deep Dumps

A curated table must support updates, deletes, schema enforcement, transaction reliability, and rollback of recent changes.

Which storage format should be used?

- A. Delta
- B. CSV
- C. Plain text
- D. Raw JSON only

Answer: A

Explanation:

A. Delta Correct

Delta Lake provides ACID transactions, support for updates and deletes, schema enforcement, schema evolution, time travel, and reliable transaction management. These features make it ideal for curated tables that require data quality, consistency, and rollback capabilities.

B. CSV Incorrect

CSV is a simple file format that does not support ACID transactions, updates, deletes, schema enforcement, or rollback capabilities.

C. Plain text Incorrect

Plain text files provide no transactional guarantees, schema management, or support for updates and deletes.

D. Raw JSON only Incorrect

JSON files are useful for storing semi-structured data but do not provide ACID transactions, schema enforcement, or rollback features by themselves.

Question: 116

Deep Dumps

A transaction source produces immutable daily files. Each file contains only new transactions and never updates previously delivered records. The pipeline should avoid unnecessary key matching.

Which load pattern is usually most appropriate?

- A. RESTORE
- B. Append

- C. MERGE with update clauses
- D. DROP TABLE before each load

Answer: B

Explanation:

A. RESTORE Incorrect

RESTORE is used to revert a Delta table to a previous version. It is a recovery operation and not a data-loading pattern.

B. Append Correct

When source files contain only new records and never modify existing data, an append-only load pattern is the simplest and most efficient approach. It avoids the overhead of matching keys and checking for updates that will never occur.

C. MERGE with update clauses Incorrect

MERGE is useful when records may be inserted, updated, or deleted. For immutable transaction files containing only new records, the key-matching overhead is unnecessary.

D. DROP TABLE before each load Incorrect

Dropping and recreating the table for every load is inefficient, removes history, and can disrupt downstream users and processes.

Question: 117

Deep Dumps

A team has several Databricks tasks that must run in sequence with retries, parameters, and alerts. The workflow does not need to orchestrate non-Databricks services.

Which option should be used?

- A. Lakeflow Jobs
- B. Manual notebook execution
- C. Table comments
- D. External table registration only

Answer: A

Explanation:

A. Lakeflow Jobs Correct

Lakeflow Jobs is designed for orchestrating Databricks workflows. It supports task dependencies, sequential execution, retries, parameters, scheduling, monitoring, and alerting, making it ideal for managing multiple Databricks tasks within a single workflow.

B. Manual notebook execution Incorrect

Manual execution requires human intervention and does not provide automated scheduling, retries, dependency management, or alerting.

C. Table comments Incorrect

Table comments are used for documentation and metadata purposes. They do not orchestrate workflows or execute tasks.

D. External table registration only Incorrect

External table registration governs access to data stored outside Databricks-managed storage. It does not provide workflow orchestration capabilities.

Question: 118

Deep Dumps

A product dimension currently overwrites old attribute values during each load. The business now needs to compare previous and current product attributes across time. What should the data engineer change?

- A. Implement a historical versioning pattern
- B. Store the table as a volume
- C. Increase SQL warehouse size only
- D. Remove all primary key logic

Answer: A

Explanation:

A. Implement a historical versioning pattern Correct

The current design behaves like **SCD Type 1**, which overwrites historical values. To compare previous and current product attributes over time, the engineer should implement a historical versioning approach such as **SCD Type 2**, which preserves changes by creating new records with effective date ranges or version indicators.

B. Store the table as a volume Incorrect

Volumes are used for managing non-tabular files in Unity Catalog. They do not provide historical tracking of dimension changes.

C. Increase SQL warehouse size only Incorrect

Increasing compute resources may improve query performance but does not preserve historical attribute values.

D. Remove all primary key logic Incorrect

Primary keys help identify unique business entities. Removing them does not solve the need for tracking historical changes.

Question: 119

Deep Dumps

A pipeline ingests semi-structured JSON files into a governed Delta table. The design must support scalable incremental ingestion and compatible schema drift. Which two capabilities are most relevant? Choose two.

- A. Auto Loader

- B. Query result caching
- C. Schema evolution
- D. Delta Sharing provider creation

Answer: A,C

Explanation:

A. Auto Loader Correct

Auto Loader is designed for scalable and efficient ingestion of new files from cloud storage. It incrementally processes arriving files, making it ideal for large-scale JSON ingestion pipelines.

B. Query result caching Incorrect

Query result caching improves performance for repeated queries but does not help with file ingestion or handling schema changes.

C. Schema evolution Correct

Schema evolution allows Delta tables to adapt to compatible schema changes, such as newly added fields in incoming JSON data, without requiring manual schema updates.

D. Delta Sharing provider creation Incorrect

A Delta Sharing provider is used to share data with external organizations and is unrelated to ingestion or schema drift management.

Final Answer: A. Auto Loader and C. Schema evolution

Question: 120

Deep Dumps

A team uses a SQL warehouse for short periods throughout the day. They want fast startup and minimal infrastructure management.
Which option best fits this requirement?

- A. Manually started classic cluster
- B. Single-node all-purpose compute
- C. Dedicated job cluster
- D. Serverless SQL warehouse

Answer: D

Explanation:

A. Manually started classic cluster Incorrect

Classic clusters require manual management, startup time, and infrastructure administration, which does not meet the requirement for minimal operational overhead.

B. Single-node all-purpose compute Incorrect

All-purpose compute is designed for interactive development and notebooks, not optimized SQL analytics workloads with rapid startup requirements.

C. Dedicated job cluster Incorrect

Job clusters are created for scheduled or triggered jobs and terminate when the job completes. They are not intended for interactive SQL analytics throughout the day.

D. Serverless SQL warehouse Correct

Serverless SQL warehouses provide near-instant startup, automatic scaling, and minimal infrastructure management. They are optimized for BI, dashboards, and ad hoc SQL workloads that run intermittently throughout the day.

Question: 121

Deep Dumps

A machine learning team needs to store governed non-tabular files, including JSON configuration files and model input files, under Unity Catalog permissions. Which Unity Catalog object should be used?

- A. External table
- B. Volume
- C. Materialized view
- D. SQL warehouse

Answer: B

Explanation:

A. External table Incorrect

External tables are used to register and govern structured data (such as Delta, Parquet, or CSV data) that is queried as tables. They are not intended for managing arbitrary non-tabular files.

B. Volume Correct

Unity Catalog **Volumes** are designed to store and govern non-tabular data such as JSON files, configuration files, machine learning artifacts, images, documents, and other files. They provide centralized governance and access control through Unity Catalog.

C. Materialized view Incorrect

A materialized view stores the results of a query for performance optimization. It cannot be used to store non-tabular files.

D. SQL warehouse Incorrect

A SQL warehouse is a compute resource for running SQL queries. It is not a storage object.

Question: 122

Deep Dumps

A reporting team needs a table with one row per customer and separate columns for monthly spending values. The source has one row per customer per month. Which transformation best fits this requirement?

- A. Union

- B. Anti join
- C. Pivot
- D. VACUUM

Answer: C

Explanation:

A. Union Incorrect

A union combines rows from two or more datasets into a single dataset. It does not transform row values into separate columns.

B. Anti join Incorrect

An anti join returns rows from one dataset that do not have matching rows in another dataset. It is not used for reshaping data into columns.

C. Pivot Correct

A pivot transformation converts unique values from rows (such as months) into separate columns. This produces one row per customer with individual columns for each month's spending, which is ideal for reporting and dashboarding.

D. VACUUM Incorrect

VACUUM removes obsolete files from a Delta table to reclaim storage space. It does not transform data.

Question: 123

Deep Dumps

A customer dimension must preserve every address change. Analysts need to query which address was valid for a customer on a specific historical date. Which design should be used?

- A. SCD Type 1 overwrite
- B. SCD Type 2 with effective dates
- C. A temporary view
- D. A full table VACUUM after each load

Answer: B

Explanation:

A. SCD Type 1 overwrite Incorrect

SCD Type 1 overwrites existing values when changes occur. Historical addresses are lost, making it impossible to determine which address was valid on a past date.

B. SCD Type 2 with effective dates Correct

SCD Type 2 preserves history by creating a new record whenever an address changes. Effective start and end dates (or a current-record flag) allow analysts to determine the address that was valid for any specific historical date.

C. A temporary view Incorrect

A temporary view is a logical query object and does not maintain historical versions of dimension data.

D. A full table VACUUM after each load Incorrect

VACUUM removes obsolete files from Delta tables to reclaim storage space. It does not provide historical tracking of address changes.

Question: 124

Deep Dumps

You have an Azure Databricks workspace that is attached to a Unity Catalog metastore named metastore1, metastore1 contains a catalog named catalog1.

You need to create a new schema named schema2 that meets the following requirements:

- Is contained in catalog1
- Uses abfss://container@storageaccount.dfs.core.windows.net/data as the managed location.

Which SQL statement should you execute?

- A. `CREATE SCHEMA catalog1 schema2LOCATION 'abfss://container@storageaccount.dfs.core.windows.net/data';`
- B. `CREATE SCHEMA catalog1 .schema2MANAGED LOCATION 'abfss://container@storageaccount.dfs.core.windows.net/data';`
- C. `CREATE CATALOG schema2MANAGED LOCATION 'abfss://container@storageaccount.dfs.core.windows.net/data';`
- D. `CREATE SCHEMA catalog1 schema2WITH DBPROPERTIES(LOCATION- 'abfss://container@storageaccount.dfs.core.windows.net/data');`

Answer: B

Explanation:

A. LOCATION Incorrect

LOCATION specifies the location for an external schema, not a managed location in Unity Catalog.

B. MANAGED LOCATION Correct

In Unity Catalog, to create a schema with a specific managed storage location, you use:

CREATE SCHEMA catalog1.schema2

MANAGED LOCATION 'abfss://container@storageaccount.dfs.core.windows.net/data';

This creates schema2 inside catalog1 and stores managed tables in the specified ADLS Gen2 path.

C. CREATE CATALOG Incorrect

The requirement is to create a schema, not a catalog.

D. WITH DBPROPERTIES Incorrect

DBPROPERTIES stores metadata properties and cannot be used to define the managed storage location.

Question: 125

Deep Dumps

A Delta table receives new fields in incoming JSON data. You want to automatically adapt without breaking pipelines. What should you enable?

- A. Auto Loader with schema inference and evolution
- B. Disable schema enforcement
- C. Use overwrite mode
- D. Manually update schema each time

Answer: A

Explanation:

A. Auto Loader with schema inference and evolution Correct

Auto Loader can automatically detect new fields in incoming JSON files through **schema inference** and update the target schema using **schema evolution**. This allows pipelines to continue running without manual intervention when the JSON structure changes.

B. Disable schema enforcement Incorrect

Disabling schema enforcement removes schema validation but does not automatically detect or add new columns to the Delta table. It can also lead to inconsistent and unreliable data.

C. Use overwrite mode Incorrect

Overwrite mode replaces existing data in the table. It does not handle schema changes automatically and may result in data loss if used improperly.

D. Manually update schema each time Incorrect

Manually modifying the schema whenever new fields arrive is time-consuming, error-prone, and does not provide the automated adaptability required by the scenario.

Question: 126

Deep Dumps

What happens if incoming data violates Delta table schema?

- A. Data is automatically cast
- B. Data is rejected
- C. Table is overwritten
- D. Data is appended with nulls

Answer: B

Explanation:

A. Data is automatically cast Incorrect

Delta Lake does not automatically cast incompatible data types that violate the table schema.

B. Data is rejected Correct

Delta Lake enforces **schema enforcement** by default. If incoming data does not match the table schema (for example, missing required columns or incompatible data types), the write operation fails and the data is

rejected, preventing bad data from being written.

C. Table is overwritten Incorrect

A schema violation does not cause the table to be overwritten.

D. Data is appended with nulls Incorrect

Delta Lake does not automatically append invalid data with null values when a schema violation occurs.

Question: 127

Deep Dumps

You have an Azure Databricks workshop that user serverless compute. You need to ingest data by using Lakeflow jobs. New records must be processed as soon as they become available. Which type of job trigger should you use for the ingestion?

- A. manual
- B. file arrival
- C. scheduled
- D. continuous

Answer: B

Explanation:

A. Manual Incorrect

A manual trigger requires a user to start the job and does not provide automatic processing of new data.

B. File arrival Correct

A **file arrival trigger** starts the Lakeflow job automatically when new files arrive in the monitored storage location. This enables near real-time ingestion and ensures that new records are processed as soon as they become available.

C. Scheduled Incorrect

A scheduled trigger runs at predefined intervals (for example, every hour) and may introduce delays between file arrival and processing.

D. Continuous Incorrect

Continuous execution is used for long-running streaming workloads, but for Lakeflow Jobs that need to react to newly arriving files, the **file arrival trigger** is the appropriate choice.

Question: 128

Deep Dumps

Which layer contains cleaned and conformed data in Databricks Lakehouse architecture?

- A. Bronze
- B. Silver
- C. Gold
- D. Raw

Answer: B

Explanation:

A. Bronze Incorrect

The **Bronze** layer contains **raw or minimally transformed** source data. It is used for ingestion, replay, and auditing.

B. Silver Correct

The **Silver** layer contains **cleaned, validated, and conformed data**. Typical transformations include:

Data cleansing and deduplication

Type casting and standardization

Handling missing values

Joining and enriching data

Applying business rules

This layer prepares data for downstream analytics and reporting.

C. Gold Incorrect

The **Gold** layer contains **business-level, aggregated, and curated datasets** optimized for dashboards, reporting, and machine learning.

D. Raw Incorrect

"Raw" is not a standard medallion layer. Raw data is typically stored in the **Bronze** layer.

Question: 129

Deep **Dumps**

You have an Azure Databricks workspace that contains an all-purpose compute cluster named Cluster1. Cluster1 is used for interactive development.

You need to configure Cluster1 to meet the following requirements:

- Automatically add and remove worker nodes based on workload demand.
- Automatically shut down when the cluster has been idle for a specific period.

What should you configure for each requirement? To answer, drag the appropriate options to the correct requirements. Each option may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Options

- Auto termination
- Autoscaling
- Max workers
- Min workers
- Photon acceleration
- Spot instances
- The driver node type

Answer Area

Automatically add and remove worker nodes:

Automatically shut down:

Option

Option

Answer:

Options	Answer Area
Auto termination	Automatically add and remove worker nodes:
Autoscaling	
Max workers	Automatically shut down:
Min workers	
Photon acceleration	Autoscaling
Spot instances	Auto termination
The driver node type	

Explanation:

Detailed Target Breakdown

Automatically add and remove worker nodes based on workload demand: Autoscaling

Reasoning: Turning on the **Autoscaling** option allows Azure Databricks to dynamically adjust the allocation of resources assigned to your cluster. It assigns additional worker instances when execution workloads (like heavy notebook cells or parallel Spark jobs) spike, and releases them back to the cloud pool when demands settle down, helping you balance performance against runtime compute costs.

Automatically shut down when the cluster has been idle for a specific period: Auto termination

Reasoning: All-purpose interactive clusters are prone to staying on accidentally when developers walk away. The **Auto termination** setting lets you specify a custom inactivity duration limit (e.g., 20 minutes). If no active operations or command cells are executed within that window, Databricks automatically de-allocates the driver and worker virtual machines to stop unnecessary infrastructure charges.

Question: 130

Deep Dumps

Drag and Drop Question

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a catalog named finance, finance contains two schemas named default and procurement.

You need to create a table named assets in the procurement schema, assets must contain the following columns:

- asset_id
- asset_type
- asset_name

How should you complete the SQL statement? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Answer Area

Values

assets

CATALOG finance

CATALOG hive_metastore

finance

SCHEMA procurement

TABLE assets

```
USE  ;  
USE  ;  
CREATE  (  
    asset_id INT,  
    asset_name STRING,  
    asset_type STRING  
);
```

Answer:

Answer Area

Values

assets

CATALOG finance

CATALOG hive_metastore

finance

SCHEMA procurement

TABLE assets

```
USE  ;  
USE  ;  
CREATE  (  
    asset_id INT,  
    asset_name STRING,  
    asset_type STRING  
);
```

Explanation:

Detailed Target Breakdown

Target 1 (First USE statement): CATALOG finance

Reasoning: In Unity Catalog's three-tier namespace (catalog.schema.table), you must first establish your current working context at the highest layer. The keyword syntax USE CATALOG followed by the catalog name (finance) sets this context.

Target 2 (Second USE statement): SCHEMA procurement

Reasoning: Once inside the active catalog, you drill down into your targeted data organization compartment using the USE SCHEMA syntax followed by the schema name (procurement).

Target 3 (CREATE statement prefix): TABLE assets

Reasoning: To instantiate a new data entity structure within your isolated schema context, you use standard DDL syntax: CREATE TABLE followed by your desired table identity name (assets). Using just the value assets

would create a syntax error because the structural type keyword (TABLE) is completely missing from the base statement template.

Question: 131

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two Delta tables named Table1 and Table2 of the same data type.

Table1 contains a column named Column1. Table2 contains a column named Column2.

You run the following query.

```
SELECT Column1
FROM Table1
GROUP BY Column1
HAVING COUNT(*) > 1
INTERSECT
SELECT Column2
FROM Table2
GROUP BY Column2
HAVING COUNT(*) > 1;
```

What occurs when you run the query?

- A. Values appear in both tables more than once.
- B. Values appear in either table more than once.
- C. Values appear in Table2 but NOT Table 1.
- D. Values appear in Table1 more than once.

Answer: A

Explanation:

Answer: A. Values appear in both tables more than once. Correct

First query:

```
SELECT Column1
FROM Table1
GROUP BY Column1
HAVING COUNT(*) > 1
```

Returns values that occur more than once in Table1.

Second query:

```
SELECT Column2
FROM Table2
GROUP BY Column2
HAVING COUNT(*) > 1
```

Returns values that occur more than once in Table2.

INTERSECT

Returns only the values that are present in both result sets.

Therefore, the final result contains values that:

Appear more than once in Table1, and

Appear more than once in Table2.

Option Analysis

A. Values appear in both tables more than once. Correct

B. Values appear in either table more than once. Incorrect

This would require a UNION, not an INTERSECT.

C. Values appear in Table2 but NOT Table1. Incorrect

This would require an EXCEPT operation.

D. Values appear in Table1 more than once. Incorrect

The query also requires the values to appear more than once in Table2.

Question: 132

Deep Dumps

What ensures failure recovery in Databricks Structured Streaming?

- A. Auto scaling
- B. Checkpointing
- C. Partition pruning
- D. Broadcast joins

Answer: B

Explanation:

A. Auto scaling Incorrect

Auto scaling adjusts cluster resources based on workload demand, but it does not provide fault tolerance or recover streaming state after failures.

B. Checkpointing Correct

Checkpointing stores streaming metadata, offsets, and state information in reliable storage. If a streaming query fails or restarts, Structured Streaming uses the checkpoint data to resume processing from the last successful state, ensuring **fault tolerance and exactly-once processing semantics**.

C. Partition pruning Incorrect

Partition pruning improves query performance by reading only relevant partitions. It does not provide failure recovery.

D. Broadcast joins Incorrect

Broadcast joins optimize join performance by sending small tables to all worker nodes. They are unrelated to recovery from failures.

Question: 133**Deep Dumps**

You have an Azure Databricks workspace that contains multiple all-purpose clusters. You discover that some clusters remain idle for long periods after users finish their work. You need to reduce compute costs without affecting active workloads. What should you do?

- A. Enable autoscaling.
- B. Convert the clusters into job clusters.
- C. Use spot instances.
- D. Configure automatic termination.

Answer: D**Explanation:****A. Enable autoscaling Incorrect**

Autoscaling adjusts the number of worker nodes based on workload demand, but it does not shut down idle clusters. The cluster can still incur costs while idle.

B. Convert the clusters into job clusters Incorrect

Job clusters are suitable for scheduled or automated jobs, not for interactive all-purpose clusters used by users.

C. Use spot instances Incorrect

Spot instances can reduce costs but may be interrupted and do not address the issue of clusters remaining idle.

D. Configure automatic termination Correct

Automatic termination (auto-termination) shuts down all-purpose clusters after a specified period of inactivity. This reduces compute costs without affecting active workloads because running clusters remain available while users are actively working.

Question: 134**Deep Dumps**

Which feature provides end-to-end tracking of data movement in Databricks?

- A. Unity Catalog lineage
- B. Spark logs
- C. Cluster metrics
- D. Query history only

Answer: A**Explanation:****A. Unity Catalog lineage Correct**

Unity Catalog lineage provides **end-to-end data lineage tracking** in Azure Databricks. It automatically

captures how data moves across tables, views, notebooks, jobs, and dashboards, allowing you to trace data origins and transformations for governance, auditing, and impact analysis.

B. Spark logs Incorrect

Spark logs contain execution details, errors, and debugging information about Spark jobs. They do not provide a comprehensive, visual, end-to-end view of data movement and dependencies.

C. Cluster metrics Incorrect

Cluster metrics monitor resource utilization such as CPU, memory, and network usage. They are used for performance monitoring, not for tracking data lineage.

D. Query history only Incorrect

Query history shows previously executed SQL queries and their execution details. It does not provide complete end-to-end lineage across datasets, notebooks, jobs, and downstream dependencies.

Question: 135

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two managed Delta tables named `sales.schema1.table1` and `sales.schema1.table2`. `sales.schema1.table1` contains sales data from the current year. `sales.schema1.table2` contains historical data.

You need to load all the rows from `sales.schema1.table1` into `sales.schema1.table2`. The solution must preserve any existing data in `sales.schema1.table2` and minimize processing effort.

Which command should you run?

- A. `INSERT OVERWRITE sales.schema1.table2 SELECT * FROM sales.schema1.table1;`
- B. `CREATE TABLE sales.schema1.table2 AS SELECT * FROM sales.schema1.table1;`
- C. `INSERT INTO sales.schema1.table2 SELECT * FROM sales.schema1.table1;`
- D. `CREATE OR REPLACE TABLE sales.schema1.table2 AS SELECT * FROM sales.schema1.table1;`

Answer: C

Explanation:

C. `INSERT INTO sales.schema1.table2 SELECT * FROM sales.schema1.table1;` Correct

`INSERT INTO` appends rows from `sales.schema1.table1` to `sales.schema1.table2` while preserving the existing historical data in `table2`. Since the requirement is to load current-year data into the historical table without deleting existing rows and with minimal processing, `INSERT INTO` is the best choice.

`INSERT INTO sales.schema1.table2`

`SELECT *`

`FROM sales.schema1.table1;`

A. `INSERT OVERWRITE sales.schema1.table2 SELECT * FROM sales.schema1.table1;` Incorrect

`INSERT OVERWRITE` replaces the existing data in `table2` with the data from `table1`. This would delete the historical data, violating the requirement to preserve existing data.

B. `CREATE TABLE sales.schema1.table2 AS SELECT * FROM sales.schema1.table1;` Incorrect

`CREATE TABLE AS SELECT (CTAS)` creates a new table. Since `table2` already exists, this command would fail because the table already exists and would not preserve the historical data.

D. CREATE OR REPLACE TABLE sales.schema1.table2 AS SELECT * FROM sales.schema1.table1;

Incorrect

CREATE OR REPLACE TABLE drops and recreates table2 using only the data from table1, removing all historical data in the process.

Question: 136

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.

You have a Lakeflow Spark Declarative Pipelines (SDP) pipeline that writes numerical data to a table named Table 1 by using a data quality validation rule named rule1.

You need to modify rule1 to meet the following requirements:

- Ensure that amount is always greater than 0.
- Prevent an update to table1 from being committed when data that violates rule1 is detected.

Which statement should you execute?

- A. `@dlt.expect_or_fail("rule1", "amount > 0")`
- B. `@dlt.expect("rule1", "amount > 0")`
- C. `@dlt.expect_all_or_drop(f"rule1": "amount > 0" )`
- D. `@dlt.expect_or_drop("rule1", "amount > 0")`

Answer: A

Explanation:

A. `@dlt.expect_or_fail("rule1", "amount > 0")` Correct

`@dlt.expect_or_fail` enforces a data quality rule and fails the pipeline update if any record violates the expectation. Since the requirement is to ensure amount is always greater than 0 and prevent the update to Table1 from being committed when invalid data is detected, this is the correct option.

`@dlt.expect_or_fail("rule1", "amount > 0")`

B. `@dlt.expect("rule1", "amount > 0")` Incorrect

`@dlt.expect` only records metrics about data quality violations. It does not drop invalid rows or fail the pipeline update.

C. `@dlt.expect_all_or_drop("rule1": "amount > 0")` Incorrect

`@dlt.expect_all_or_drop` drops records that violate the expectations and continues processing. The requirement is to prevent the update from being committed, not simply discard invalid rows.

D. `@dlt.expect_or_drop("rule1", "amount > 0")` Incorrect

`@dlt.expect_or_drop` removes rows that fail the rule and allows the pipeline to continue. It does not fail the update or stop the commit.

Question: 137

Deep Dumps

What improves join performance for small lookup tables?

- A. Shuffle join

- B. Broadcast join
- C. Cartesian join
- D. Sort merge join

Answer: B

Explanation:

B. Broadcast join Correct

A Broadcast Join improves join performance when one of the tables (typically a lookup or dimension table) is small enough to fit in memory. Spark broadcasts the small table to all worker nodes, avoiding expensive data shuffling across the cluster and significantly speeding up the join operation.

large_df.join(broadcast(small_lookup_df), "key")

A. Shuffle join Incorrect

A Shuffle Join requires data to be redistributed (shuffled) across the cluster based on the join key. This can be expensive and is not the optimal choice for small lookup tables.

C. Cartesian join Incorrect

A Cartesian Join produces the cross product of both tables, generating every possible row combination. It is extremely expensive and generally avoided unless specifically required.

D. Sort merge join Incorrect

A Sort Merge Join requires both datasets to be shuffled and sorted by the join key. It works well for large datasets but is less efficient than a broadcast join when one table is small.

Question: 138

Deep Dumps

What is the best way to reduce Databricks compute cost for intermittent workloads?

- A. Use all-purpose clusters
- B. Enable autoscaling and auto-termination
- C. Increase cluster size
- D. Use single-node clusters only

Answer: B

Explanation:

A. Use all-purpose clusters Incorrect

All-purpose clusters can remain running and incur costs even when idle.

B. Enable autoscaling and auto-termination Correct

Autoscaling adjusts the number of worker nodes based on workload demand, and auto-termination shuts down clusters after a period of inactivity. Together, they minimize compute costs for intermittent workloads.

C. Increase cluster size Incorrect

Increasing cluster size generally increases costs and does not address intermittent usage.

D. Use single-node clusters only Incorrect

Single-node clusters may be suitable for small workloads but are not the best general solution for intermittent workloads.

Question: 139

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two catalogs named Catalog1 and Catalog2.

An external application uses a service principal named SP1 to connect to a SQL warehouse.

You need to ensure that SP1 can query the data in Catalog1 and Catalog2. The solution must follow the principle of least privilege.

Which permissions should you grant to SP1 for the catalogs?

- A. USE SCHEMA and SELECT
- B. USE CATALOG and SELECT
- C. USE CATALOG, USE SCHEMA, and SELECT
- D. USE CATALOG and USE SCHEMA

Answer: C

Explanation:

C. USE CATALOG, USE SCHEMA, and SELECT Correct

In Unity Catalog, to query data in a table, a principal (such as a service principal) needs the following permissions:

1. **USE CATALOG** – Allows access to the catalog.
2. **USE SCHEMA** – Allows access to the schema within the catalog.
3. **SELECT** – Allows reading data from the tables.

Without all three permissions, SP1 cannot successfully query the data. This follows the **principle of least privilege** because only the minimum permissions required for read access are granted.

A. USE SCHEMA and SELECT Incorrect

USE SCHEMA and SELECT are insufficient because the service principal also needs **USE CATALOG** to access objects within the catalog.

B. USE CATALOG and SELECT Incorrect

USE CATALOG and SELECT are insufficient because the service principal also requires **USE SCHEMA** to access schemas and their tables.

D. USE CATALOG and USE SCHEMA Incorrect

These permissions allow navigation to the catalog and schema, but without **SELECT**, SP1 cannot read data from the tables.

Question: 140

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem.

Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace named Workspace1 that contains a lakehouse and is enabled for Unity Catalog.

You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

- The schemas and tables can be queried in Databricks.
- The schemas and tables appear alongside other Unity Catalog objects.
- The data is NOT copied into Databricks-managed storage.

Solution: You create a Lakeflow Connect pipeline and connect it to DB1.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

A. **Yes** Incorrect

Lakeflow Connect copies data into Databricks-managed storage, violating the requirement.

B. **No** Correct

The solution does **not** meet the goal because it does not satisfy the requirement that the data remain in the external SQL Server database and not be copied into Databricks.

Question: 141

Deep Dumps

You have an Azure Databricks workspace that contains a Delta table named Table1.

Table1 has accumulated obsolete files.

You need to reduce storage costs. The solution must preserve 30 days of time travel history.

Which two actions should you perform? Each correct answer presents part of the solution.

NOTE: Each correct selection is worth one point.

- A. Run the vacuum command on Table1.
- B. Set the delta.deletedFileRetentionDuration table property to 30 days.
- C. Set the delta.logRetentionDuration table property to 30 days.
- D. Run the OPTIMIZE command on Table1.
- E. Reduce the deleted file retention period to one day.

Answer: AB

Explanation:

A. **Run the VACUUM command on Table1.** Correct

VACUUM removes obsolete data files that are no longer referenced by the Delta table, thereby reducing storage costs.

VACUUM Table1;

However, VACUUM only deletes files older than the configured deleted file retention period.

B. Set the delta.deletedFileRetentionDuration table property to 30 days. Correct

delta.deletedFileRetentionDuration specifies how long deleted data files are retained for time travel and rollback operations.

Setting it to 30 days ensures that:

30 days of time travel history are preserved.

VACUUM removes only files older than 30 days.

Example:

```
ALTER TABLE Table1
```

```
SET TBLPROPERTIES (
```

```
'delta.deletedFileRetentionDuration' = 'interval 30 days'
```

```
);
```

C. Set the delta.logRetentionDuration table property to 30 days. Incorrect

delta.logRetentionDuration controls how long transaction log entries are retained, not the retention of deleted data files required for time travel.

Changing only the log retention period does not guarantee 30 days of time travel.

D. Run the OPTIMIZE command on Table1. Incorrect

OPTIMIZE compacts small files to improve query performance. It does not remove obsolete files or reduce storage costs by itself.

E. Reduce the deleted file retention period to one day. Incorrect

Reducing the retention period to one day would allow VACUUM to delete files needed for time travel, violating the requirement to preserve 30 days of history.

Question: 142

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a Delta table named Orders.

You load the Orders table into an Apache Spark DataFrame named df.

You need to create a DataFrame that excludes rows where the order amount is null.

Solution: You run the following expression. `df.fillna(0, subset=['order_amount'])`

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

A. Yes Incorrect

`df.fillna(0, subset=['order_amount'])` does not exclude rows with null values. Instead, it replaces null values in the `order_amount` column with 0.

B. No Correct

The requirement is to exclude rows where `order_amount` is null. To remove such rows, you should use:

`df.dropna(subset=['order_amount'])`

Question: 143

Deep Dumps

You have an Azure Databricks workspace named `Workspace1` that uses a Git repository. The repository contains a Databricks notebook named `Notebook1`.

From the main branch, you create a feature branch named `Branch1` and commit changes to `Notebook1`.

Another user commits changes to `Notebook1` in main.

When you attempt to merge `Branch1` into main, the merge fails due to conflicts.

You need to merge `Branch1` into the main branch. The solution must ensure that `Notebook1` includes all the changes from both the branches.

What should you do?

- A. From `Workspace1`, clone the main branch as a new repository.
- B. From `Workspace1`, clone `Branch1` as a new repository.
- C. Apply the main branch changes to `Branch1` and resolve the conflicts.
- D. Apply the changes directly to the main branch.

Answer: C

Explanation:

C. Apply the main branch changes to `Branch1` and resolve the conflicts. Correct

When changes are made to `Notebook1` in both `Branch1` and main, a merge conflict occurs. To ensure that `Notebook1` contains all changes from both branches, you should first bring the latest changes from main into `Branch1`, resolve the conflicts, commit the resolved notebook, and then merge `Branch1` back into main.

A. From `Workspace1`, clone the main branch as a new repository. Incorrect

Cloning the main branch only creates another copy of the repository and does not resolve the merge conflict or combine changes from both branches.

B. From `Workspace1`, clone `Branch1` as a new repository. Incorrect

Cloning `Branch1` also creates another copy of the repository and does not help in resolving merge conflicts or merging the changes.

D. Apply the changes directly to the main branch. Incorrect

Applying changes directly to main can overwrite existing changes and does not properly resolve the conflict while preserving modifications from both branches.

Question: 144**Deep Dumps**

You have an Azure Databricks workspace.

You are creating a Lakeflow Spark Declarative Pipelines (SDP) pipeline that scales automatically.

You need to configure compute for the pipeline. The solution must minimize operational costs and effort.

What should you use?

- A. a job cluster that uses autoscaling
- B. an all-purpose cluster that uses autoscaling
- C. a single-node, all-purpose cluster
- D. the existing SQL warehouse

Answer: A**Explanation:**

A. a job cluster that uses autoscaling Correct

A **job cluster with autoscaling** is created only when the Lakeflow Spark Declarative Pipelines (SDP) pipeline runs and is terminated after completion. Autoscaling automatically adjusts the number of workers based on workload requirements, minimizing both **operational costs and management effort**.

B. an all-purpose cluster that uses autoscaling Incorrect

An **all-purpose cluster** is designed for interactive development and analysis. Even with autoscaling, it is typically kept running longer and is more expensive than job clusters for scheduled pipeline workloads.

C. a single-node, all-purpose cluster Incorrect

A **single-node all-purpose cluster** does not provide automatic scaling and may not handle varying workloads efficiently. It also requires more manual management.

D. the existing SQL warehouse Incorrect

A **SQL warehouse** is designed for executing SQL queries and BI workloads. It is not intended to run Lakeflow Spark Declarative Pipelines (SDP) processing jobs.

Question: 145**Deep Dumps**

You need to configure compute for the ingestion of telemetry data. The solution must meet the data ingestion and processing requirements.

What should you do?

- A. Move the ingestion pipelines to shared compute.
- B. Increase an all-purpose cluster to a larger fixed node type.
- C. Enable Photon acceleration for a job compute cluster.
- D. Disable autoscaling for a job compute cluster.

Answer: C**Explanation:**

C. Enable Photon acceleration for a job compute cluster. Correct

Photon is Databricks' high-performance query engine that accelerates data processing workloads. Enabling Photon on a **job compute cluster** improves the performance of telemetry data ingestion and processing while maintaining efficient and scalable compute usage.

A. Move the ingestion pipelines to shared compute. Incorrect

Shared compute is intended for collaborative and interactive workloads. It is not the optimal choice for high-throughput telemetry ingestion pipelines.

B. Increase an all-purpose cluster to a larger fixed node type. Incorrect

Using a larger fixed cluster increases costs and lacks the flexibility and efficiency needed for varying ingestion workloads.

D. Disable autoscaling for a job compute cluster. Incorrect

Disabling autoscaling can lead to under-provisioning or over-provisioning of resources, increasing costs or reducing performance for telemetry ingestion workloads.

Question: 146

Deep Dumps

Hotspot Question

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains a managed Delta table named Table1.

Table1 is written by batch jobs every hour and is queried frequently by filtering two columns named Customerid and EventDate.

You expect Table 1 to grow significantly over time.

The rows in Table 1 are frequently updated and deleted to support compliance requests.

You need to keep query performance consistent as Table1 grows. The solution must minimize update and deletion effort.

What should you include in the solution? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

To optimize query performance, implement:

Liquid clustering
Partitioning
Z-ordering

To optimize updates and deletions:

Disable deletion vectors.
Enable deletion vectors.
Run the VACUUM command after each batch load.

Answer:

Answer Area

To optimize query performance, implement:

Liquid clustering
Partitioning
Z-ordering

To optimize updates and deletions:

Disable deletion vectors.
Enable deletion vectors.
Run the VACUUM command after each batch load.

Explanation:

1. To optimize query performance, implement:

Correct Selection: Liquid clustering

Explanation: Databricks Liquid clustering replaces older layout optimization techniques like manual partitioning and Z-ordering. Because your query load filters on multiple columns (CustomerID and EventDate) and the table is growing significantly over time, Liquid clustering dynamically adapts data layouts as data is written without suffering from over-partitioning problems. It maintains highly consistent query read execution performance automatically.

2. To optimize updates and deletions:

Correct Selection: Enable deletion vectors.

Explanation: Compliance requests (like GDPR/CCPA deletes) usually modify or remove individual rows scattered randomly across your storage blocks. By default, Delta Lake uses a "Copy-on-Write" strategy, meaning modifying a single row requires rewriting the entire data file. Enabling deletion vectors switches the table to a "Merge-on-Read" strategy. Instead of rewriting massive parquet blocks every hour during compliance runs, it writes tiny, highly efficient tracking markers (vectors) that record exactly which rows were removed, reducing update and deletion write costs dramatically.

Question: 147

Deep Dumps

You have an Azure Databricks workspace.

You have an Apache Spark Structured Streaming job named Job1 that processes data continuously and fails periodically due to transient errors.

You need to ensure that Job1 meets the following requirements:

- Resumes processing from the point that Job1 failed
- Minimizes how long it takes to restart Job 1
- Minimizes the costs to restart Job1

What should you do?

- A. Decrease the retry interval.
- B. Increase the minimum number of nodes in the cluster.
- C. Add an alert and manually restart Job1.
- D. Implement checkpointing.

Answer: D

Explanation:

D. Implement checkpointing. Correct

Checkpointing stores the progress and state information of an Apache Spark Structured Streaming job, including offsets and metadata. If Job1 fails because of transient errors, it can **resume processing from the last checkpoint instead of reprocessing all data**. This minimizes both **restart time and restart costs**.

A. Decrease the retry interval. Incorrect

Reducing the retry interval may restart the job sooner, but it does not preserve the processing state or allow the job to resume from where it failed.

B. Increase the minimum number of nodes in the cluster. Incorrect

Increasing cluster size may improve performance, but it does not help the streaming job recover from failures or resume from the point of failure. It can also increase costs.

C. Add an alert and manually restart Job1. Incorrect

Manual restarts increase operational effort and downtime. Without checkpointing, the job may need to reprocess data, increasing both time and cost.

Question: 148**Deep Dumps**

Note: This section contains one or more sets of questions with the same scenario and problem.

Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have an Azure Databricks workspace named Workspace1 that contains a lakehouse and is enabled for Unity Catalog.

You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

- The schemas and tables can be queried in Databricks.
- The schemas and tables appear alongside other Unity Catalog objects.
- The data is NOT copied into Databricks-managed storage.

Solution: You create a new native catalog in Unity Catalog.

Does this meet the goal?

A. Yes

B. No

Answer: B**Explanation:**

A. Yes Incorrect

Creating a native catalog does not expose the external SQL Server objects and does not provide direct access to DB1.

B. No Correct

A native catalog alone does not satisfy the requirement to query SQL Server tables without copying the data into Databricks.

Question: 149**Deep Dumps**

A data engineer notices slow query performance on a large Delta table in Azure Databricks. The table has frequent updates and deletes. Which action best improves query performance?

- A. Increase cluster size
- B. Enable Delta caching only
- C. Run OPTIMIZE and ZORDER BY on frequently filtered columns
- D. Convert table to Parquet format

Answer: C**Explanation:****C. Run OPTIMIZE and ZORDER BY on frequently filtered columns Correct**

OPTIMIZE compacts small files created by frequent updates and deletes, reducing file fragmentation and improving read efficiency. ZORDER BY colocates related data in the same files based on frequently filtered columns, improving data skipping and significantly enhancing query performance on large Delta tables.

OPTIMIZE table_name**ZORDER BY (frequently_filtered_column);****A. Increase cluster size Incorrect**

Increasing cluster size may provide more compute resources, but it does not address the underlying issues of file fragmentation and inefficient data layout in the Delta table.

B. Enable Delta caching only Incorrect

Delta caching can improve performance for repeatedly accessed data, but it does not resolve problems caused by frequent updates, deletes, and many small files.

D. Convert table to Parquet format Incorrect

Converting a Delta table to Parquet would remove Delta Lake features such as ACID transactions, time travel, and efficient handling of updates and deletes. It is not the recommended solution for improving performance.

Question: 150**Deep Dumps**

You have an Azure Databricks workspace that is enabled for Unity Catalog. You have a complex job named Job1 that contains eight tasks. Job1 takes multiple hours to complete. During the last job run, the final task fails due to a transient issue. You need to retry the last task without rerunning tasks that have already completed. What should you do?

- A. Repair the current job run.
- B. Update the job parameters.
- C. Disable and reenabling the job schedule.
- D. Restart Job1.

Answer: A

Explanation:

A. Repair the current job run. Correct

The **Repair Run** feature in Azure Databricks allows you to rerun only the failed tasks and any dependent tasks in a job. Since the final task failed due to a transient issue, repairing the current run retries only that task and **does not rerun the seven tasks that already completed**, saving both time and compute costs.

B. Update the job parameters. Incorrect

Updating job parameters changes the configuration of the job but does not retry only the failed task or resume the previous run.

C. Disable and reenable the job schedule. Incorrect

Disabling and reenabling the schedule affects when the job runs in the future and does not help retry the failed task from the previous run.

D. Restart Job1. Incorrect

Restarting Job1 launches a completely new job run and reruns all eight tasks, including those that already completed successfully, increasing execution time and costs.

Question: 151

Deep **Dumps**

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1.

Job1 processes raw data files stored in Azure Storage.

Jew files arrive at unpredictable intervals.

You need to ensure that Job1 starts automatically when new files arrive and does NOT consume compute resources when no data is available.

Which type of job trigger should you use?

- A. manual
- B. file arrival
- C. continuous
- D. scheduled

Answer: B

Explanation:

B. file arrival Correct

A **file arrival trigger** automatically starts Job1 when new files are detected in Azure Storage. Since files arrive at unpredictable intervals, this trigger ensures that the job runs only when data is available and **does not consume compute resources when no new files exist**, minimizing costs.

A. manual Incorrect

A manual trigger requires a user to start the job explicitly and does not automatically process newly arrived files.

C. continuous Incorrect

A continuous trigger keeps the job running continuously to check for new data, which can consume compute resources even when no files are available.

D. **scheduled** Incorrect

A scheduled trigger runs the job at fixed times regardless of whether new files have arrived, potentially wasting compute resources.

Question: 152

Deep **Dumps**

Which operation guarantees ACID compliance in Delta Lake?

- A. INSERT OVERWRITE
- B. Direct file append
- C. Delta transaction log
- D. Spark RDD transformation

Answer: C

Explanation:

C. **Delta transaction log** Correct

The **Delta transaction log** (`_delta_log`) is the core mechanism that provides **ACID (Atomicity, Consistency, Isolation, Durability)** guarantees in Delta Lake. Every change to a Delta table is recorded in the transaction log, enabling reliable transactions, concurrency control, and recovery.

A. **INSERT OVERWRITE** Incorrect

INSERT OVERWRITE is a data modification operation on a Delta table, but it does not itself guarantee ACID compliance. The ACID guarantees come from the underlying **Delta transaction log**.

B. **Direct file append** Incorrect

Directly appending files to the storage location bypasses Delta Lake's transaction management and can lead to inconsistent data states. It does not provide ACID guarantees.

D. **Spark RDD transformation** Incorrect

RDD transformations are Spark processing operations and are unrelated to transaction management or ACID compliance in Delta Lake.

Question: 153

Deep **Dumps**

You have an Azure Databricks workspace that contains an all-purpose cluster named Cluster1.

You need to configure Cluster1 to meet the following requirements:

- The cluster must scale up automatically when workloads increase.
- The cluster must scale down automatically when workloads decrease.

The solution must minimize costs.

Which two actions should you perform? Each correct answer presents part of the solution.

NOTE: Each correct selection is worth one point.

- A. Disable Photon acceleration.

- B. Enable autoscaling for Cluster1.
- C. Apply a compute policy that enables users to manage the cluster settings.
- D. Specify a fixed number of workers.
- E. Configure Cluster1 to terminate after 30 minutes of inactivity.

Answer: BE

Explanation:

B. Enable autoscaling for Cluster1. Correct

Autoscaling automatically increases the number of workers when workloads grow and decreases the number of workers when demand drops. This satisfies the requirements for both scaling up and scaling down while optimizing resource usage and costs.

E. Configure Cluster1 to terminate after 30 minutes of inactivity. Correct

Auto-termination shuts down the cluster when it is idle for a specified period, preventing unnecessary compute charges and helping minimize costs.

A. Disable Photon acceleration. Incorrect

Disabling Photon does not contribute to automatic scaling or cost optimization. Photon often improves performance and can even reduce costs by completing workloads faster.

C. Apply a compute policy that enables users to manage the cluster settings. Incorrect

Allowing users to manage cluster settings does not automatically scale the cluster or minimize costs.

D. Specify a fixed number of workers. Incorrect

A fixed number of workers prevents automatic scaling and may lead to over-provisioning or under-provisioning of resources.

Question: 154

Deep Dumps

You have a Lakeflow Spark Declarative Pipelines (SP) pipeline in Azure Databricks. The pipeline ingests transaction data into a table named Table1.

You need to ensure that in the event of an invalid record, the pipeline continues to run. The solution must meet the following requirements:

- Invalid records must NOT be written to Table 1.
- Invalid records must be preserved for review.
- Minimize development effort.

What should you do?

- A. Add a CHECKconstraint to Table1.
- B. Run WHEREclauses in downstream queries to filter out invalid records.
- C. Define a pipeline expectation.
- D. Implement advanced logic to quarantine the invalid records.

Answer: C

Explanation:

C. Define a pipeline expectation. Correct

Pipeline expectations in Lakeflow Spark Declarative Pipelines (SDP) allow you to define data quality rules and specify how invalid records should be handled. You can configure expectations to **drop invalid records from Table1 while preserving them for review in expectation metrics**, and the pipeline continues running. This approach requires minimal development effort.

A. Add a CHECK constraint to Table1. Incorrect

A CHECK constraint enforces data integrity but can cause write operations to fail when invalid records are encountered, potentially stopping the pipeline and not preserving invalid records for review.

B. Run WHERE clauses in downstream queries to filter out invalid records. Incorrect

Filtering invalid records in downstream queries means the invalid data is already written to Table1, violating the requirement that invalid records must not be stored in the table.

D. Implement advanced logic to quarantine the invalid records. Incorrect

Quarantining invalid records can satisfy the requirements, but it requires additional custom development and does **not minimize development effort** compared to built-in pipeline expectations.

Question: 155

Deep Dumps

You have an Azure Databricks workspace that contains a job in Lakeflow Jobs named Job1. Job1 contains multiple tasks.

Failures of non-critical tasks must be logged but must NOT trigger notifications. Notifications must be triggered only when critical tasks have failed, and Job1 has completed.

You need to configure the job alerting behavior.

What should trigger a notification?

- A. task success
- B. a job failure
- C. a task failure
- D. job success

Answer: B

Explanation:

B. a job failure Correct

A **job failure notification** is triggered only after the entire job has completed and its final status is **Failed**. This allows non-critical task failures to be logged without sending notifications, while notifications are sent only when failures in critical tasks cause Job1 to fail.

A. task success Incorrect

Task success notifications are unrelated to failure scenarios and do not meet the requirement.

C. a task failure Incorrect

A task failure notification is sent immediately when any task fails, including non-critical tasks. This violates the requirement that non-critical task failures should not trigger notifications.

D. job success Incorrect

Job success notifications are sent only when the entire job completes successfully and do not notify about

critical task failures.

Question: 156

Deep Dumps

You need to ingest real-time IoT data into Delta Lake with exactly-once guarantees. Which approach should you use?

- A. Copy activity with retry policy
- B. Manual ingestion using notebooks
- C. Structured Streaming with checkpointing
- D. Batch ingestion using ADF

Answer: C

Explanation:

C. Structured Streaming with checkpointing **Correct**

Structured Streaming with checkpointing provides exactly-once processing guarantees in Delta Lake. Checkpointing stores offsets and state information, enabling the streaming job to recover from failures and continue processing without duplicating or losing data. This makes it ideal for real-time IoT data ingestion.

```
stream_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/checkpoints/iot") \
    .start("/delta/iot_data")
```

A. Copy activity with retry policy **Incorrect**

A retry policy can re-execute failed activities, potentially causing duplicate data processing. It does not provide exactly-once guarantees for real-time streaming.

B. Manual ingestion using notebooks **Incorrect**

Manual notebook-based ingestion requires custom logic for fault tolerance and state management and does not inherently provide exactly-once processing.

D. Batch ingestion using ADF **Incorrect**

Azure Data Factory batch ingestion is designed for periodic data loads, not continuous real-time streaming with exactly-once guarantees.

Question: 157

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog. You need to recommend a pipeline that ingests files from cloud storage, performs cleansing and enrichment transformations, and writes curated Delta tables for analytics. The solution must minimize development effort and provide built-in monitoring and automatic retries. What should you include in the recommendation?

- A. a Lakeflow Spark Declarative Pipelines (SP) pipeline

- B. a Databricks notebook triggered by a scheduled job
- C. an Azure Data Factory pipeline that uses data flows
- D. an Apache Spark Structured Streaming job

Answer: A

Explanation:

A. a Lakeflow Spark Declarative Pipelines (SP) pipeline Correct

Lakeflow Spark Declarative Pipelines (SDP) are designed for ingesting data, performing cleansing and enrichment transformations, and writing curated Delta tables. They provide built-in monitoring, automatic retries, and simplified pipeline development, which minimizes both development effort and operational overhead.

B. a Databricks notebook triggered by a scheduled job Incorrect

A notebook-based solution requires custom code for monitoring, retries, and error handling. It involves more development and maintenance effort than Lakeflow Spark Declarative Pipelines.

C. an Azure Data Factory pipeline that uses data flows Incorrect

Azure Data Factory can orchestrate data pipelines, but it does not provide the same native Databricks declarative pipeline capabilities and built-in monitoring and retry features required by the scenario.

D. an Apache Spark Structured Streaming job Incorrect

Structured Streaming is primarily intended for real-time streaming workloads and requires additional custom development for orchestration, monitoring, and retry logic.

Question: 158

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog.
You have an Apache Spark Structured Streaming job that writes data to a Delta table.
After the cluster restarts, the streaming job reprocesses previously ingested data.
You need to prevent the streaming job from reprocessing the data after the cluster restarts.
What should you do?

- A. Configure a checkpoint location for the streaming query.
- B. Configure a watermark for the streaming query.
- C. Enable change data feed (CDF) for the target table.
- D. Increase the trigger interval of the streaming query.

Answer: A

Explanation:

A. Configure a checkpoint location for the streaming query. Correct

A **checkpoint location** stores the streaming query's progress information, including offsets and state data. After a cluster restart, Structured Streaming reads the checkpoint information and resumes processing from the last committed point, preventing previously ingested data from being reprocessed.

B. Configure a watermark for the streaming query. Incorrect

A **watermark** is used to handle late-arriving data and manage state in aggregations. It does not track processing progress or prevent data from being reprocessed after a restart.

C. **Enable change data feed (CDF) for the target table.** Incorrect

Change Data Feed (CDF) tracks row-level changes in a Delta table. It does not store streaming offsets or prevent reprocessing after cluster restarts.

D. **Increase the trigger interval of the streaming query.** Incorrect

The **trigger interval** controls how often the streaming query processes data. Changing it does not preserve processing state or prevent reprocessing after a restart.

Question: 159

Deep Dumps

You have an Azure Databricks workspace named Workspace1 that contains a takehouse and is enabled for Unity Catalog. You have a connection to a Microsoft SQL Server database named DB1.

You need to expose the schemas and tables of DB1 to meet the following requirements:

The schemas and tables can be queried in Databricks. The schemas and tables appear alongside other Unity Catalog objects.

The data is NOT copied into Databricks-managed storage.

Solution: You create a new native catalog in Unity Catalog.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

A. **Yes** Incorrect

Creating a native catalog does not provide access to external SQL Server objects without copying data.

B. **No** Correct

A native catalog does not meet the requirement of querying external SQL Server tables without copying the data into Databricks.

Question: 160

Deep Dumps

You have an Azure Databricks workspace. You are creating a Lakeflow Spark Declarative Pipelines (SDP) pipeline that scales automatically. You need to configure compute for the pipeline.

The solution must minimize operational costs and effort.

What should you use?

A. an all-purpose cluster that uses autoscaling

B. the existing SQL warehouse

C. a job cluster that uses autoscaling

D. a single-node, all-purpose cluster

Answer: C

Explanation:

C. a job cluster that uses autoscaling Correct

A **job cluster with autoscaling** is created only when the Lakeflow Spark Declarative Pipelines (SDP) pipeline runs and is terminated after the run completes. Autoscaling automatically adjusts the number of workers based on workload requirements, minimizing both operational effort and costs.

A. an all-purpose cluster that uses autoscaling Incorrect

An all-purpose cluster is designed for interactive development and analysis. Even with autoscaling, it is generally more expensive than a job cluster for pipeline workloads.

B. the existing SQL warehouse Incorrect

A SQL warehouse is designed for SQL analytics and BI workloads. It is not intended to run Lakeflow Spark Declarative Pipelines.

D. a single-node, all-purpose cluster Incorrect

A single-node all-purpose cluster does not provide automatic scaling and is not suitable for varying pipeline workloads.

Question: 161

Deep Dumps

You use Databricks Asset Bundles to manage two jobs and an app. You need to deploy the bundle to development and production environments.

The solution must meet the following requirements:

Deploy the app to both environments.

Deploy only one job to development.

Minimize administrative effort.

What should you use?

- A. a resources node in a databricks.yml file
- B. separate databricks.yml files for each environment
- C. a variables node in a databricks.yml file
- D. a targets node in a databricks.yml file

Answer: D

Explanation:

A. a resources node in a databricks.yml file Incorrect

The resources node defines jobs, pipelines, and apps, but it does not provide environment-specific deployment behavior.

B. separate databricks.yml files for each environment Incorrect

This would increase administrative effort because you would have to maintain multiple configuration files.

C. a variables node in a databricks.yml file Incorrect

Variables allow parameterization of values but do not control which resources are deployed to specific

environments.

D. a targets node in a databricks.yml file Correct

The targets node is designed for environment-specific deployments (such as dev and prod). It allows you to override configurations and control which resources are deployed in each environment while maintaining a single bundle configuration file, minimizing administrative effort.

Question: 162

Deep Dumps

You have an Azure Databricks workspace that is enabled for Unity Catalog and contains two Delta tables named Table1 and Table2 of the same data type. Table1 contains a column named Column1. Table2 contains a column named Column2. You run the following query: `SELECT Column1 FROM Table1 GROUP BY Column1 HAVING COUNT(*) > 1 INTERSECT SELECT Column2 FROM Table2 GROUP BY Column2 HAVING COUNT(*) > 1`.

What occurs when you run the query?

- A. Values appear in either table more than once.
- B. Values appear in both tables more than once.
- C. Values appear in Table more than once.
- D. Values appear in Table2 but NOT Table1.

Answer: B

Explanation:

A. Values appear in either table more than once. Incorrect

This would be the result of a UNION, not an INTERSECT.

B. Values appear in both tables more than once. Correct

INTERSECT returns only values that are duplicated in both tables.

C. Values appear in Table1 more than once. Incorrect

The first query alone identifies duplicates in Table1, but INTERSECT further restricts the result to values also duplicated in Table2.

D. Values appear in Table2 but NOT Table1. Incorrect

This would require an EXCEPT operation, not INTERSECT.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured
100% Pass Guarantee



24/7 Customer Support
Mail us - support@deepdumps.com



Verified Updates
Lifetime Updates!

Total: **162 Questions**

Link: <https://deepdumps.com/papers/microsoft/dp-750>